Title Page

SBT-DF204 - Investigating Rhion Possession Evidence

Student Name: Athanasius Alekwe

Student ID: 2025/FWSD/11230

Course Name: Computer Forensics Case Study

Instructor Name: Aminu Idris

Date of Submission: 26/10/2026

Introduction

This case study involved a forensic examination of the supplied Rhion evidence package using an isolated Kali Linux environment. The investigation focused on a USB disk image and three network packet captures supplied for the exercise. The purpose was to determine what the available artefacts could establish about recovered files, possible concealment, FTP transfers, a protected archive and later HTTP activity.

The examination was carried out using working copies rather than directly analysing the preserved source evidence. Hash values, file sizes and file types were recorded throughout the investigation so that important artefacts could be traced back to their source. The analysis combined filesystem examination, file carving, metadata review, steganography checks, network reconstruction and cross-evidence comparison.

Objectives

The objectives of the investigation were to:

Preserve the supplied evidence and create verified working copies before analysis.

Record SHA-256 hashes, file sizes, source locations and working-copy locations.

Identify the filesystem and layout of the supplied USB disk image.

Examine filesystem records and recover files that were no longer directly visible through normal directory listings.

Identify recovered file types and determine whether any recovered files were duplicates.

Examine recovered media and document metadata for relevant information.

Test recovered images for evidence of hidden or embedded content using the authorised forensic tools.

Analyse the FTP capture to identify sessions, transfer direction, filenames, TCP streams and server responses.

Reconstruct and validate files transferred through FTP.

Examine the protected ZIP archive and compare its contents with USB-recovered evidence.

Analyse both HTTP captures and reconstruct relevant network objects.

Correlate findings across the USB, FTP and HTTP evidence while avoiding unsupported person-level attribution.

Laboratory / Tools Used

The investigation was completed inside a Kali Linux virtual machine. The main tools and utilities used during the examination included:

sha256sum for integrity verification.

stat for file size and metadata checks.

file for identifying actual file formats.

unzip for examining and extracting the supplied evidence archive.

Sleuth Kit tools including img_stat, mmls, fsstat, fls, istat and icat.

PhotoRec 7.2 for file carving from the USB image.

ExifTool 13.25 for metadata examination.

Antiword for safely extracting text from the recovered Microsoft Word document.

Stegseek 0.6 for Steghide-related hidden-content testing.

Binwalk for embedded-signature examination.

cmp for byte-for-byte file comparison.

Tshark/Wireshark for FTP and HTTP packet analysis.

7-Zip for examining and validating the reconstructed protected archive.

John the Ripper and zip2john for the authorised offline archive-password recovery stage.

Executive Conclusion

The examination established a strong technical relationship between the USB evidence and the network activity contained in the supplied packet captures.

The USB image was identified as a FAT16 filesystem. Although the deleted-directory search did not display deleted entries, PhotoRec recovered 132 files from free space. The recovered material included JPEG and GIF images and a Microsoft Word document. Hash and byte comparison showed that two carved JPEG filenames represented the same binary file, leaving eight unique recovered image files.

The recovered Word document contained text referring to Rhino photographs, hiding photographs and reformatting a USB device. This provided useful contextual evidence, but the statements alone could not prove that a specific person wrote the document or carried out the actions described. Stegseek did not identify a valid Steghide seed in the unique JPEG files, while Binwalk showed the expected image signatures without additional embedded-file signatures in the recorded results.

FTP analysis showed that 137.30.122.253 transferred files to FTP server 137.30.120.40 using the STOR command. The transferred items included rhino1.jpg, rhino3.jpg and contraband.zip. The reconstructed contraband.zip measured 230,566 bytes, matching the amount reported by the FTP server. The archive was encrypted using ZipCrypto and was successfully validated through the authorised course-only offline recovery procedure.

The archive contained rhino2.jpg. Its SHA-256 value matched USB-recovered f0106409.jpg, establishing that both artefacts contained exactly the same binary data. This was the strongest cross-source correlation found during the investigation.

The HTTP captures also showed requests for Rhino-related material, including rhino4.jpg and rhino.exe. The reconstructed rhino.exe was identified as a Windows PE32 executable but was not executed.

Overall, the evidence provides high confidence in a technical connection between the recovered USB material and the recorded FTP and HTTP activity. It does not, by itself, prove who physically used the device or computer at the relevant times.

Investigation Methodology

Evidence Preservation

The supplied DFRWS2005-RODEO.zip package was first recorded and hashed. A working copy was produced and its SHA-256 value was checked against the original record. The extracted evidence files—RHINOUSB.dd, rhino.log, rhino2.log and rhino3.log—were individually hashed and copied into the working evidence directory. Successful OK results confirmed that the working files matched the recorded source data.

USB Examination

The USB image was examined using file, img_stat, mmls, fsstat and fls. The image was identified as a FAT16 filesystem beginning at sector zero. The root directory contained gumbo1.txt and gumbo2.txt. A deleted-entry search did not reveal deleted directory records, so file carving was used to examine recoverable content beyond the active directory structure.

File Recovery and Metadata Review

PhotoRec was run against free space in the FAT16 image and recovered 132 files. The recovered files were classified with file and hashed with SHA-256. Duplicate-hash analysis identified two JPEG files containing identical binary content.

Recovered images were examined with ExifTool, while the recovered Word document was examined using ExifTool and Antiword. This preserved the original recovered document while allowing its metadata and readable text to be reviewed.

Hidden-Content Examination

Unique recovered JPEG files were tested using Stegseek. The tool did not find a valid Steghide seed in the tested files. Binwalk was also used to inspect the unique images for additional embedded signatures. The recorded output showed the normal JPEG or GIF signatures rather than separate embedded objects.

These negative results weakened the suggestion that the examined files contained Steghide or simple appended-file concealment, but they could not rule out every possible steganographic method.

FTP Examination

rhino.log was analysed with Tshark/Wireshark. FTP control commands, responses and associated TCP streams were identified. The presence of STOR commands established that the material was being sent from the client toward the FTP server.

Data streams associated with rhino1.jpg, two transfers of rhino3.jpg, and contraband.zip were reconstructed. File sizes, types and SHA-256 hashes were then recorded.

Protected Archive Examination

The reconstructed contraband.zip was identified as an encrypted ZIP archive using ZipCrypto. An authorised offline password-recovery procedure was used only against the supplied course archive. The archive successfully passed validation and contained rhino2.jpg.

The SHA-256 hash of the extracted rhino2.jpg matched the SHA-256 hash of USB-recovered f0106409.jpg, demonstrating binary equality across the FTP and USB evidence sources.

HTTP Examination

rhino2.log and rhino3.log were analysed for HTTP requests and responses. Tshark was used to identify request paths, hosts, streams, response codes, content types and object sizes.

Relevant HTTP objects were exported from the packet captures rather than retrieved from live systems. rhino4.jpg was identified as a JPEG, while rhino.exe was identified as a PE32 Windows executable. The executable was preserved and hashed but was not executed.

Findings

1. Evidence Preservation

The supplied evidence was preserved through recorded SHA-256 hashes and verified working copies. All four main evidence files produced successful integrity checks before analysis continued.

2. USB Image Before Recovery

The disk image contained a FAT16 filesystem beginning at sector zero, using 512-byte sectors and 4,096-byte clusters. The root directory included gumbo1.txt and gumbo2.txt. The deleted-entry search returned no deleted directory records.

3. Recoverable Files

PhotoRec recovered 132 files from free space. The material included text files, JPEG images, GIF images and a Microsoft Word document. Duplicate checking showed that f0105873.jpg and f0335081.jpg were byte-for-byte identical.

4. Hidden-Content Evidence

The recovered document contained language referring to hiding Rhino photographs. However, Stegseek returned no valid seed for the tested unique JPEG files, while Binwalk displayed only normal image signatures in the recorded output.

The document therefore provides contextual evidence suggesting possible concealment, but the technical image tests performed did not independently confirm hidden Steghide content.

5. FTP Evidence

FTP traffic showed communication between client 137.30.122.253 and server 137.30.120.40. STOR commands were observed for rhino1.jpg, rhino3.jpg and contraband.zip, establishing upload direction toward the server.

The server returned successful 226 Transfer complete responses for the material transfers. The reconstructed contraband.zip was exactly 230,566 bytes, matching the amount reported by the server.

6. Protected Archive

contraband.zip was encrypted with ZipCrypto. Following the authorised offline recovery procedure, the archive validated successfully and rhino2.jpg was extracted.

The extracted image had the same SHA-256 value as USB-recovered f0106409.jpg. This establishes that the two files were binary identical.

7. HTTP Evidence

The first HTTP capture included a request for /~gnome/rhino4.jpg and a successful JPEG response. The reconstructed rhino4.jpg measured 53,191 bytes.

The second HTTP capture contained a request for /~gnome/rhino.exe. The successful response contained 145,920 bytes and the reconstructed object was identified as a PE32 Windows executable. It was not executed during the investigation.

Timeline and Correlation

Stage	UTC Timestamp	Evidence	Observed Event
Evidence preservation	30 Oct 2026, 06:42:20 UTC	Evidence package	Original archive details and SHA-256 were recorded before working-copy analysis.
FTP activity	26 Apr 2004, approximately 22:21:49 UTC	rhino.log	Client issued STOR rhino1.jpg.
FTP activity	26 Apr 2004, approximately 22:22:08 UTC	rhino.log	First STOR rhino3.jpg transfer occurred in ASCII mode.
FTP activity	26 Apr 2004, approximately 22:22:16 UTC	rhino.log	A second rhino3.jpg transfer was made after binary mode was selected.
FTP activity	26 Apr 2004, approximately 22:26:46 UTC	rhino.log	Client issued STOR contraband.zip; the server later reported a successful transfer.
HTTP activity	28 Apr 2004, approximately 21:08:38 UTC	rhino2.log	HTTP request was made for /~gnome/rhino4.jpg.
HTTP activity	28 Apr 2004, approximately 21:16:23 UTC	rhino3.log	HTTP request was made for /~gnome/rhino.exe.

The recovered Word document also contained embedded 2005 application metadata, but its source timezone was not established. For that reason, it should not be presented as a fully normalised UTC event.

Evidence Log

Evidence ID	Tool / Method	Source	Main Finding
E01	SHA-256 and working-copy verification	Supplied archive and four extracted evidence files	Integrity was recorded and all working copies passed verification.
E02	file, fsstat, fls	RHINOUSB.dd	FAT16 filesystem identified; root contained gumbo1.txt and gumbo2.txt.
E03	PhotoRec, SHA-256, file, cmp	USB free space	132 files recovered; duplicate JPEG pair confirmed.
E04	ExifTool, Antiword, Stegseek, Binwalk	Recovered document and images	Document discussed hidden photographs; image tests did not confirm Steghide content.
E05	Tshark/Wireshark	rhino.log	FTP uploads of Rhino material and contraband.zip identified.
E06	7-Zip, John, SHA-256	Reconstructed contraband.zip	Archive contained rhino2.jpg, whose hash matched USB-recovered f0106409.jpg.
E07	Tshark HTTP export and file	rhino2.log, rhino3.log	rhino4.jpg and rhino.exe were reconstructed and validated by file type and hash.

Lab Folder Structure and Evidence Preparation

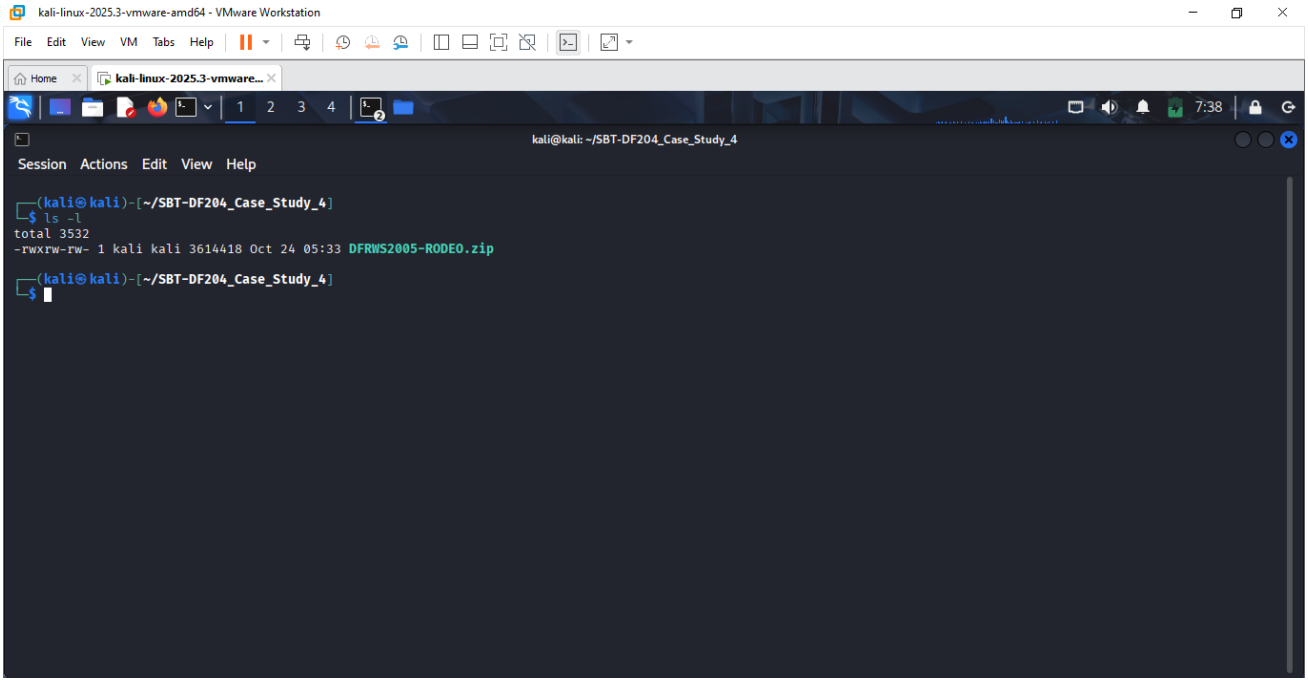


Fig 1.0 Forensic Investigation Directory Structure.

Explanation

The screenshot shows the case working folder created in Kali Linux and the supplied evidence archive placed inside it. This establishes the organised location used to keep the investigation material separate from other files.

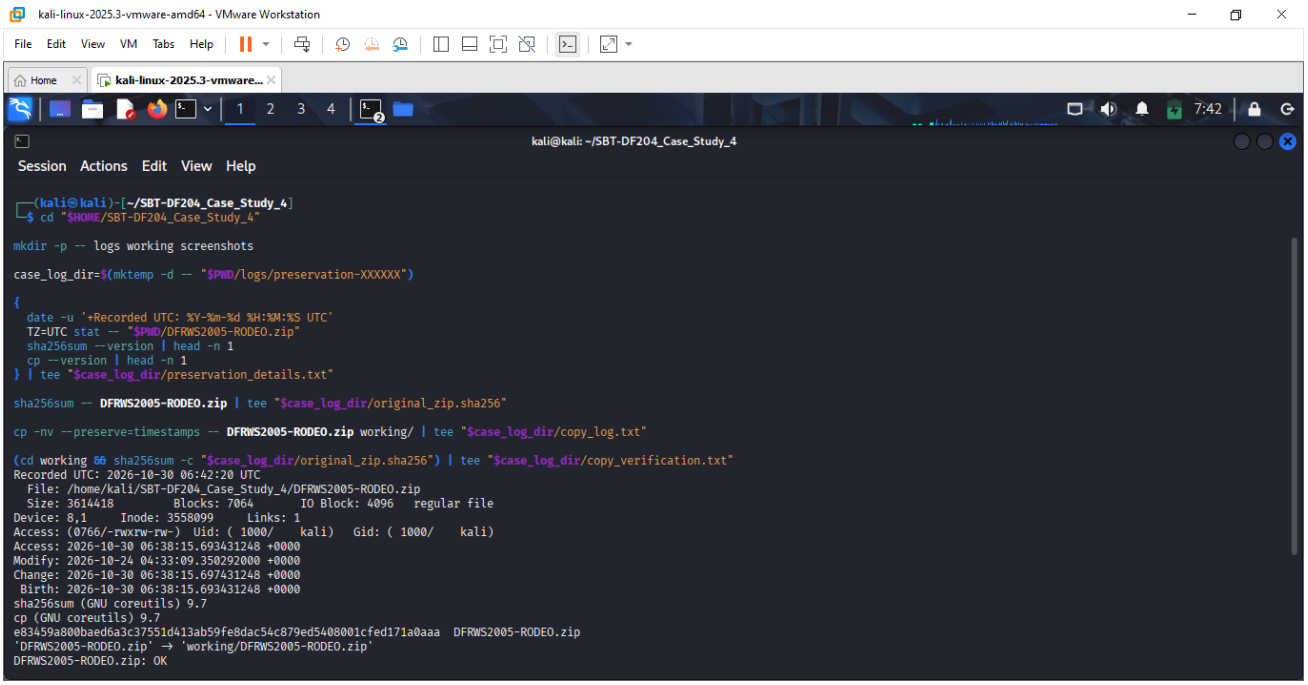


Fig 1.1 Original Archive Hash and Working Copy Verification.

Explanation

The output records the original ZIP metadata and SHA-256 value before confirming the copied archive with

an OK verification result. This demonstrates that the working archive matched the recorded source.

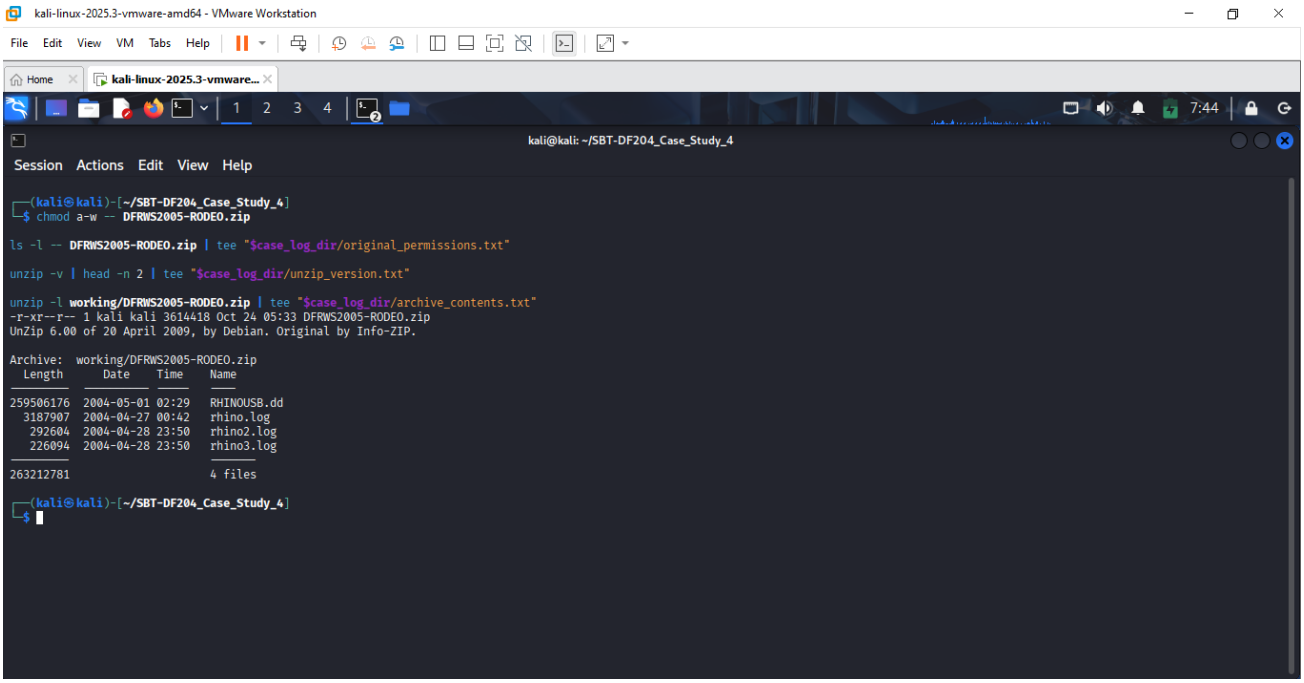


Fig 1.2 Evidence Archive Contents.

Explanation

The screenshot lists RHINOUSB.dd, rhino.log, rhino2.log and rhino3.log inside the supplied package. It establishes the exact evidence files and their sizes before extraction.

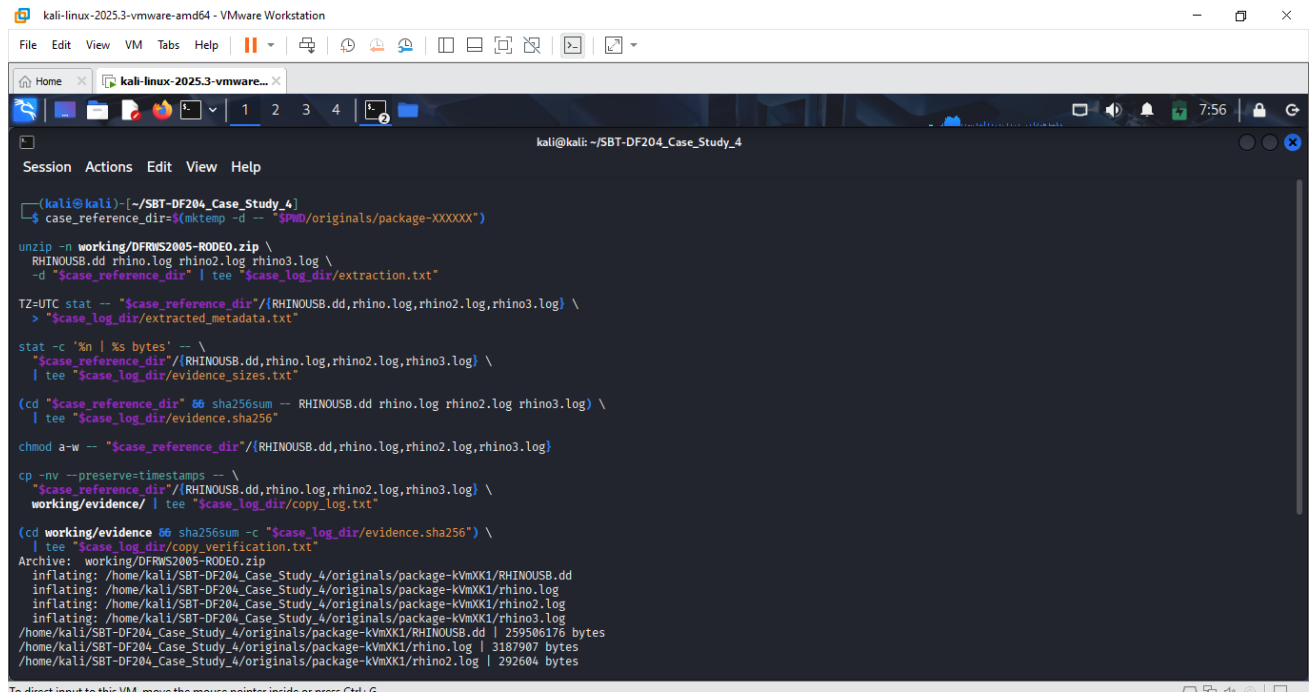


Fig 1.3 Extracted Evidence Hashes and Working Copy Verification.

Explanation

The output shows the evidence files being extracted into the preserved evidence location and their sizes

being recorded. SHA-256 hashing was performed before the separate analysis copies were used.

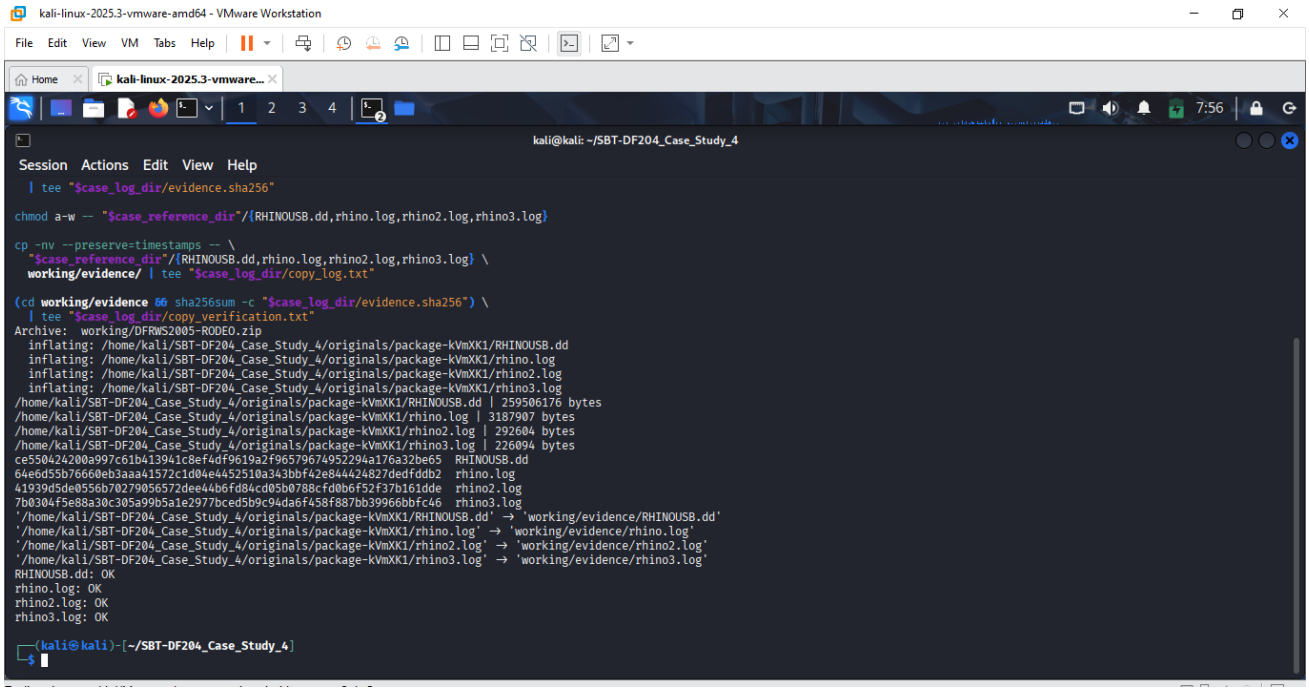


Fig 1.4 Extracted Evidence Hashes and Working Copy Verification page 1.

Explanation

The screenshot shows the extracted evidence copied into the working directory and all four files returning OK during hash verification. This confirms that the analysis copies remained identical to the recorded evidence files.

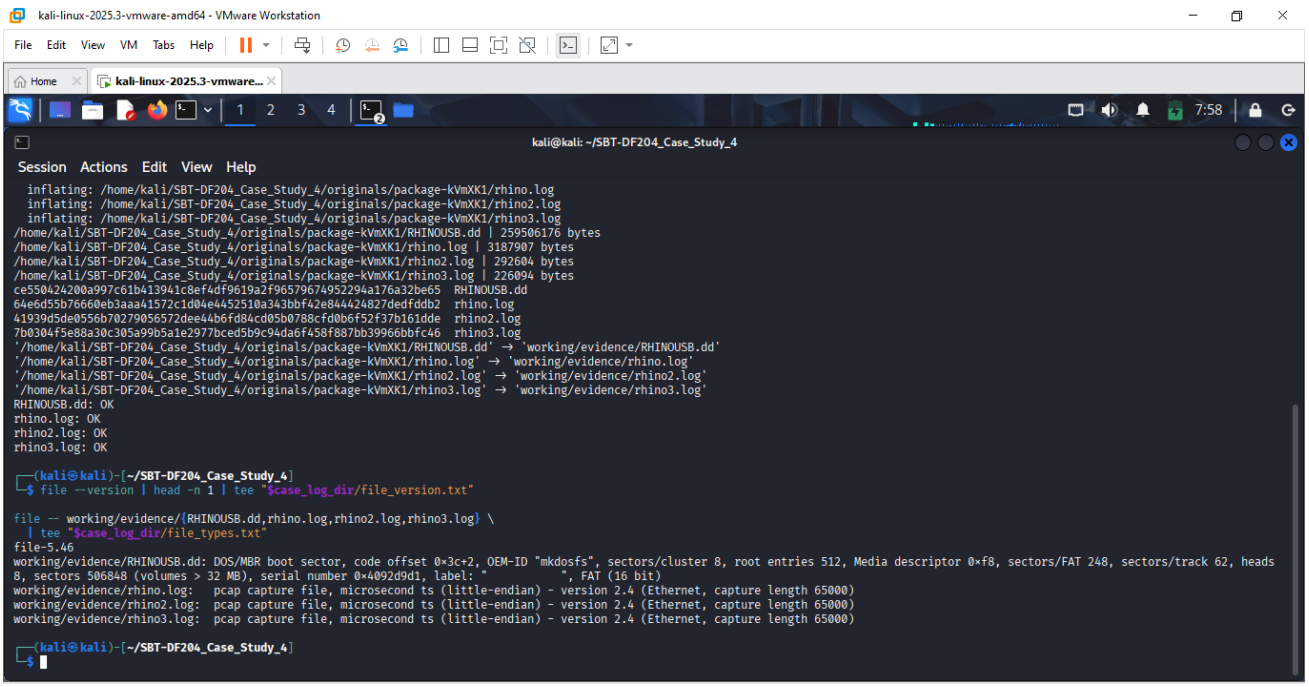


Fig 1.5 Evidence File Type Identification.

Explanation

The file utility identifies RHINOUSB.dd as FAT-related disk data and the three .log files as packet captures. This confirms the real formats without relying only on filename extensions.

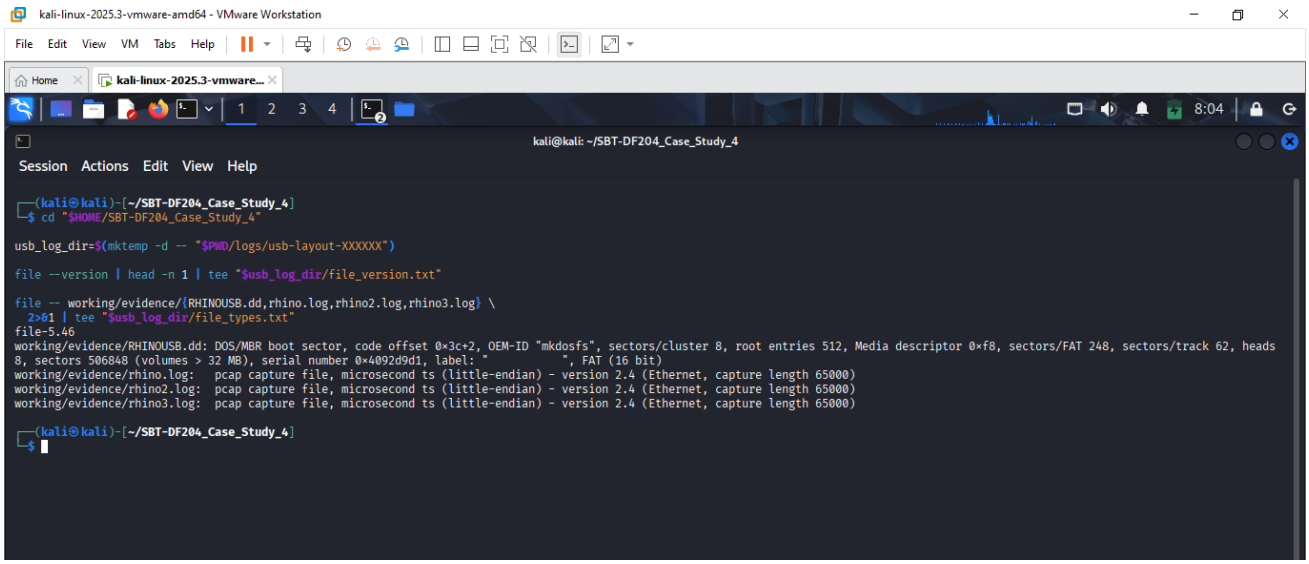


Fig 1.6 Evidence File Type Identification page 1.

Explanation

This continuation provides the complete format-identification results for the disk image and packet captures. The findings were used to select the appropriate disk and network forensic tools.

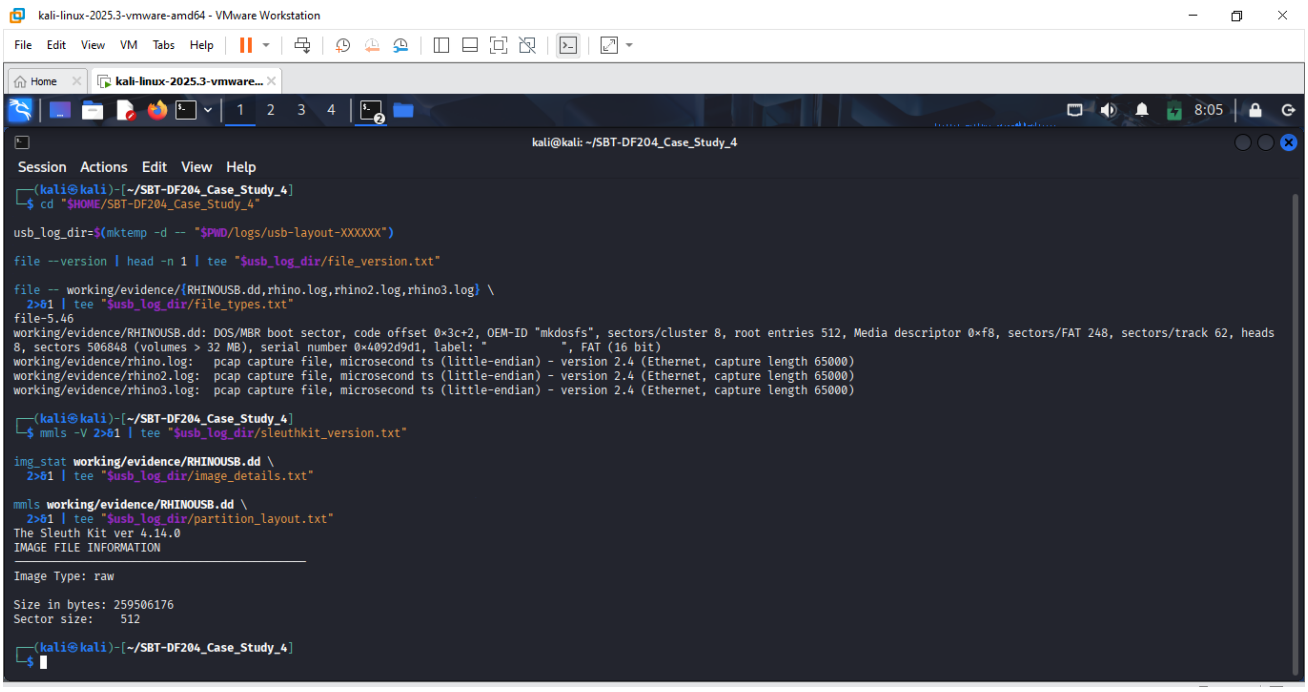


Fig 1.7 USB Disk Image Details and Partition Layout.

Explanation

img_stat records the raw image size and sector size while mmls does not display a conventional partition table. The results supported checking for a filesystem starting directly at sector zero.

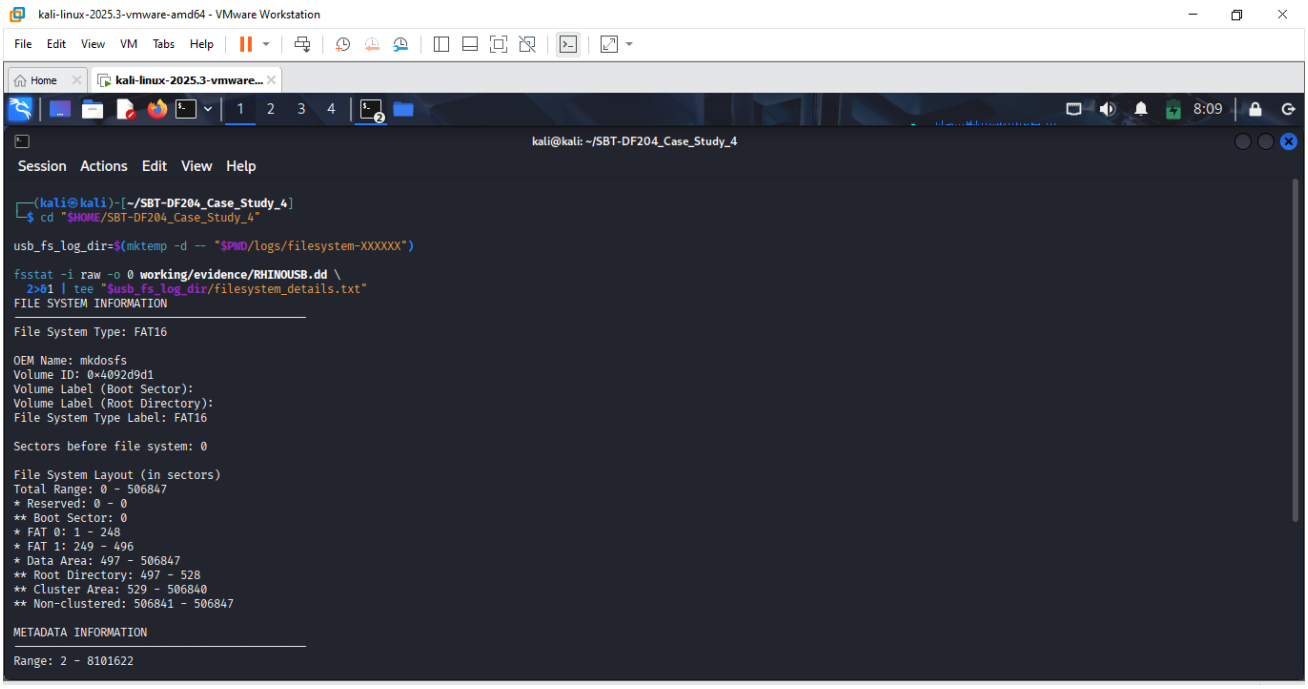


Fig 1.8 USB FAT16 File System Details.

Explanation

fsstat identifies the USB image as FAT16 and shows its main filesystem layout. The output records the boot sector, FAT regions, root-directory area and data area.

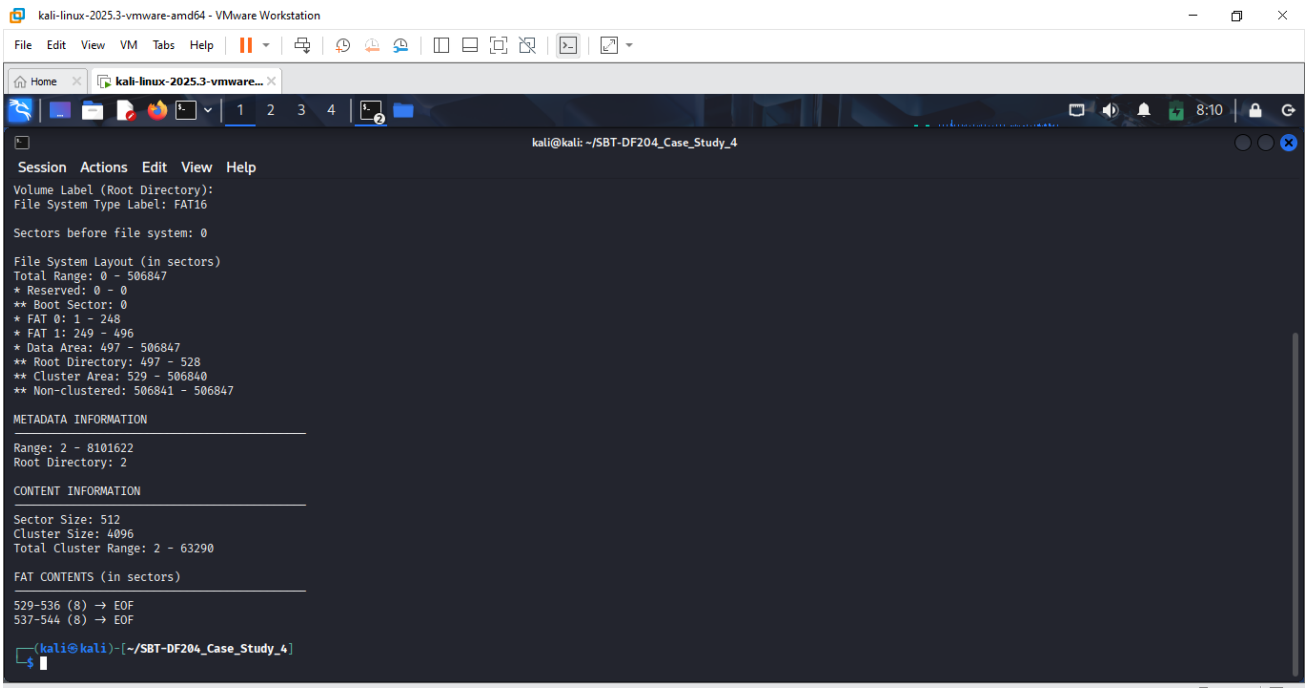


Fig 1.9 USB FAT16 File System Details page 1.

Explanation

The continuation records the 512-byte sector size, 4,096-byte cluster size and FAT content ranges. These details describe how data is organised inside the image.

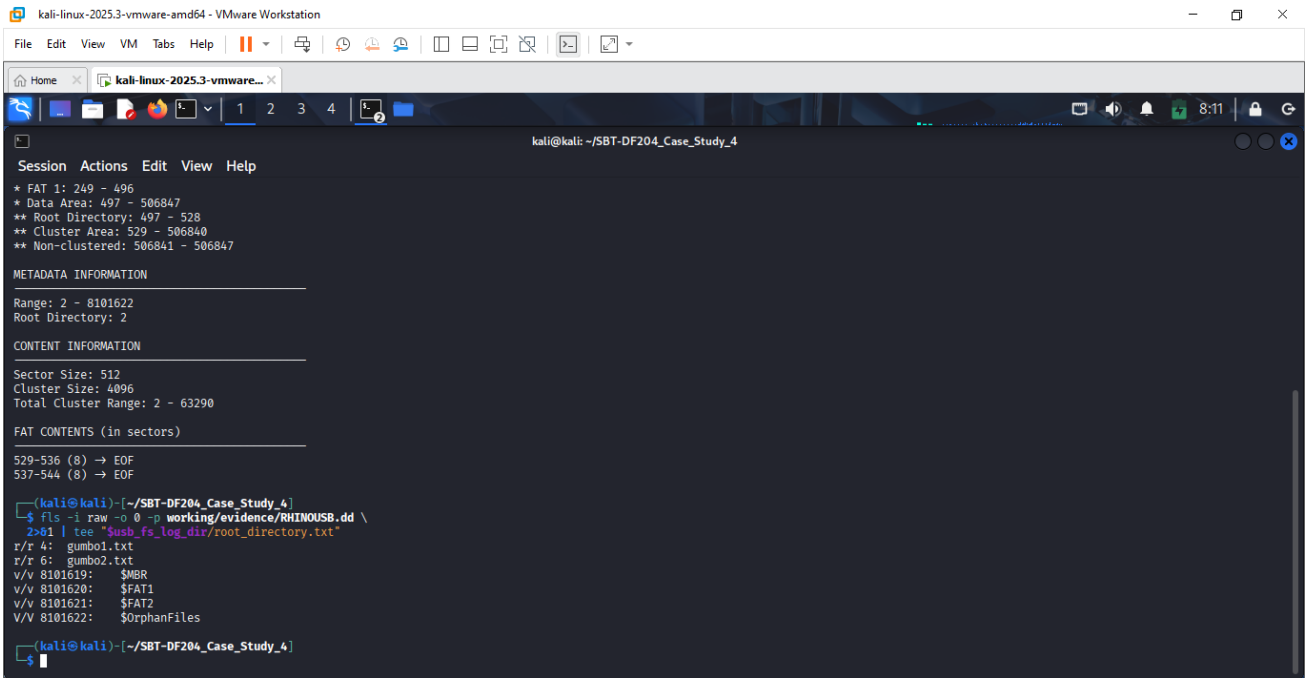


Fig 2.0 USB Root Directory Listing.

Explanation

fls displays the active root-directory entries, including gumbo1.txt and gumbo2.txt. The listing provides the metadata record numbers used for subsequent examination.

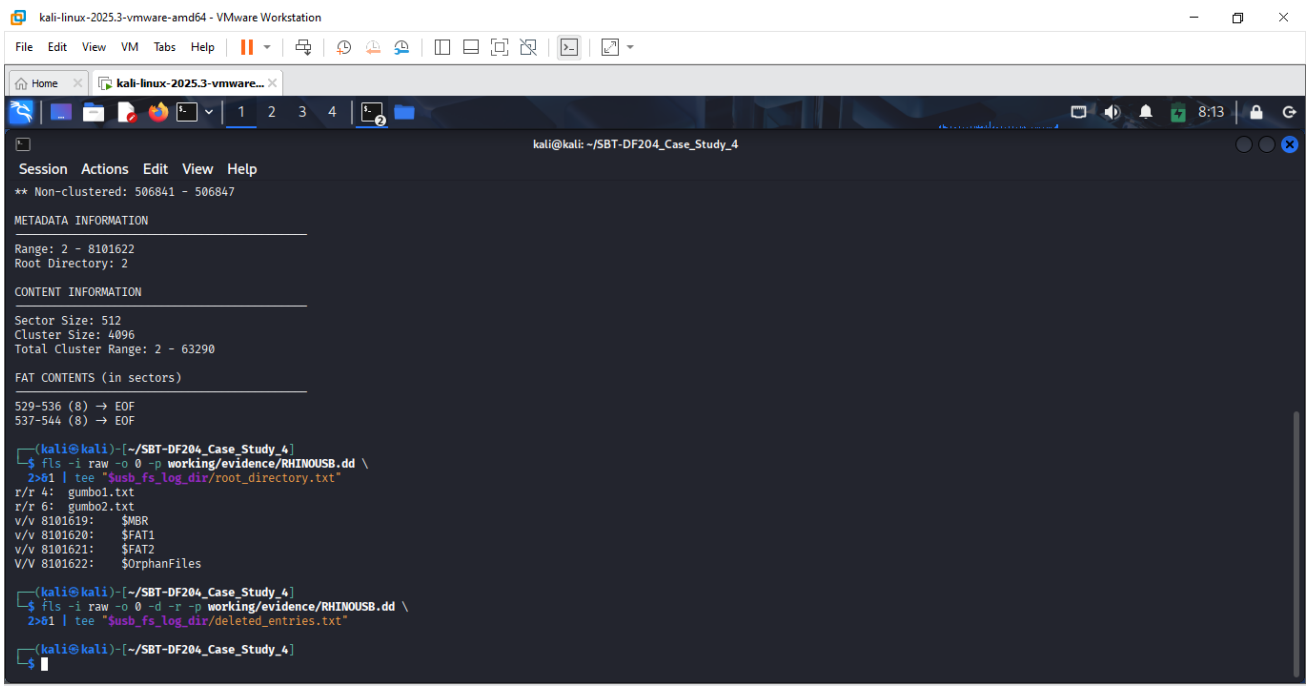


Fig 2.1 Deleted File and Directory Entries.

Explanation

The recursive deleted-entry command completed without displaying deleted directory records. Because this did not exclude recoverable data in free space, file carving was used afterwards.

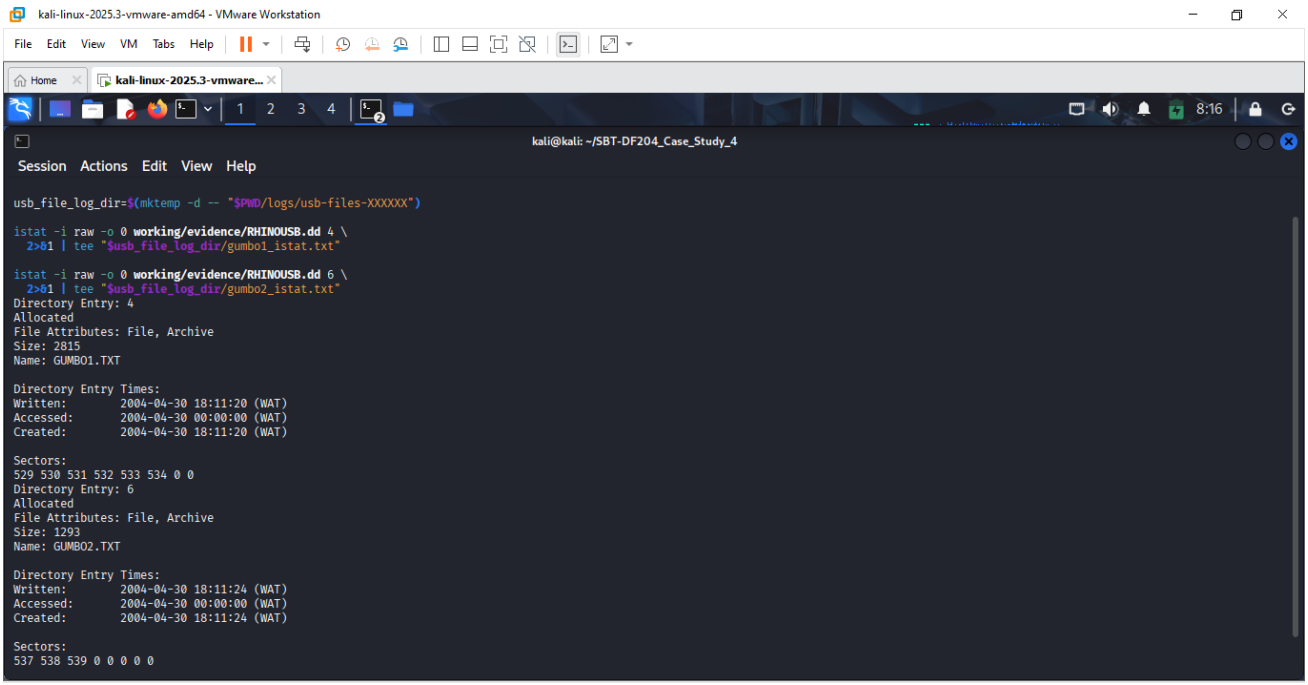


Fig 2.2 USB Gumbo File Metadata.

Explanation

istat displays filesystem metadata for the Gumbo text files, including their sizes, timestamps and storage information. This documents their filesystem records before extraction.

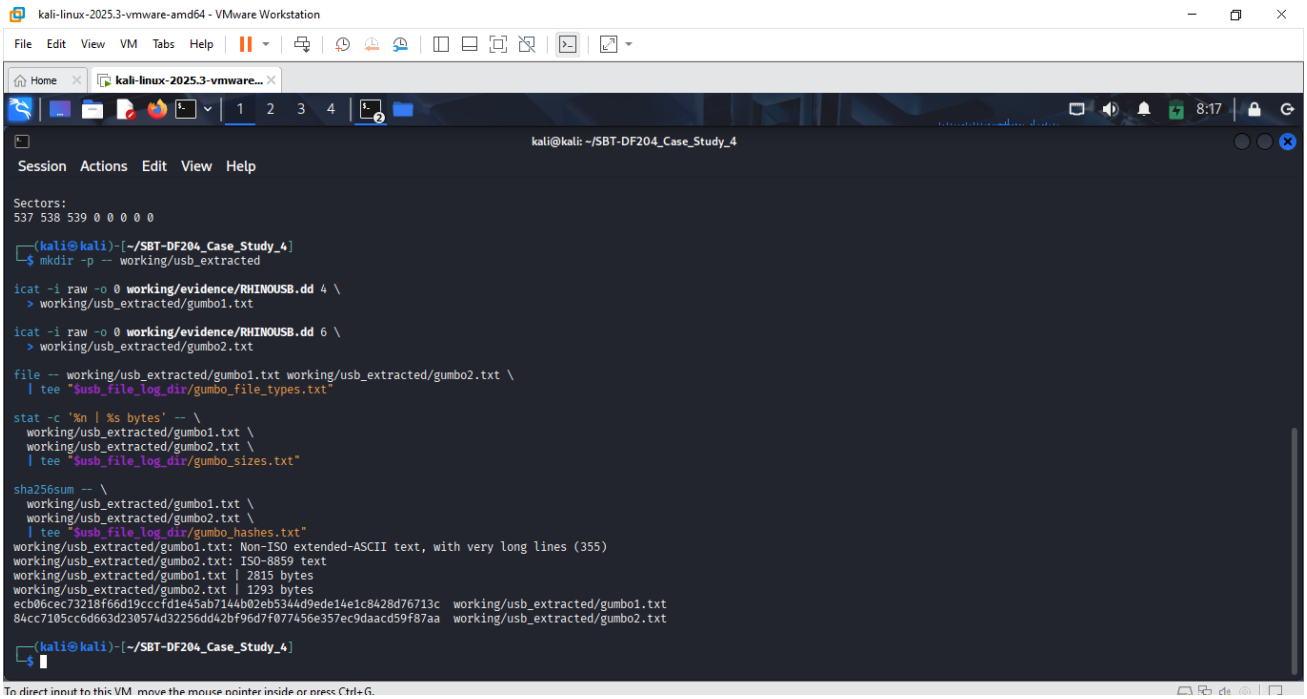


Fig 2.3 Recovered Gumbo Files and SHA256 Hashes.

Explanation

The Gumbo files were extracted with icat, identified by file type and hashed. This provides traceable recovered copies for analysis.

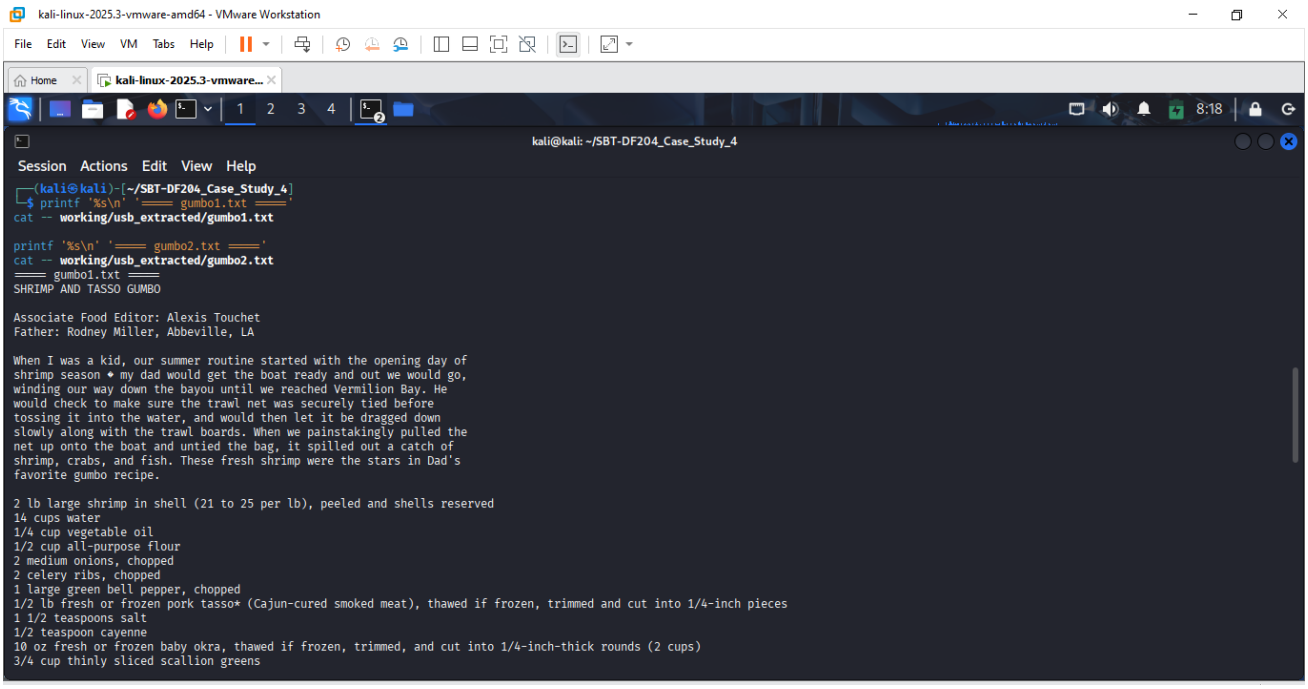


Fig 2.4 Recovered Gumbo Text Examination Page 1.

Explanation

The screenshot displays readable content recovered from one of the Gumbo text files. It confirms that the extracted file contains structured recipe text rather than binary data.

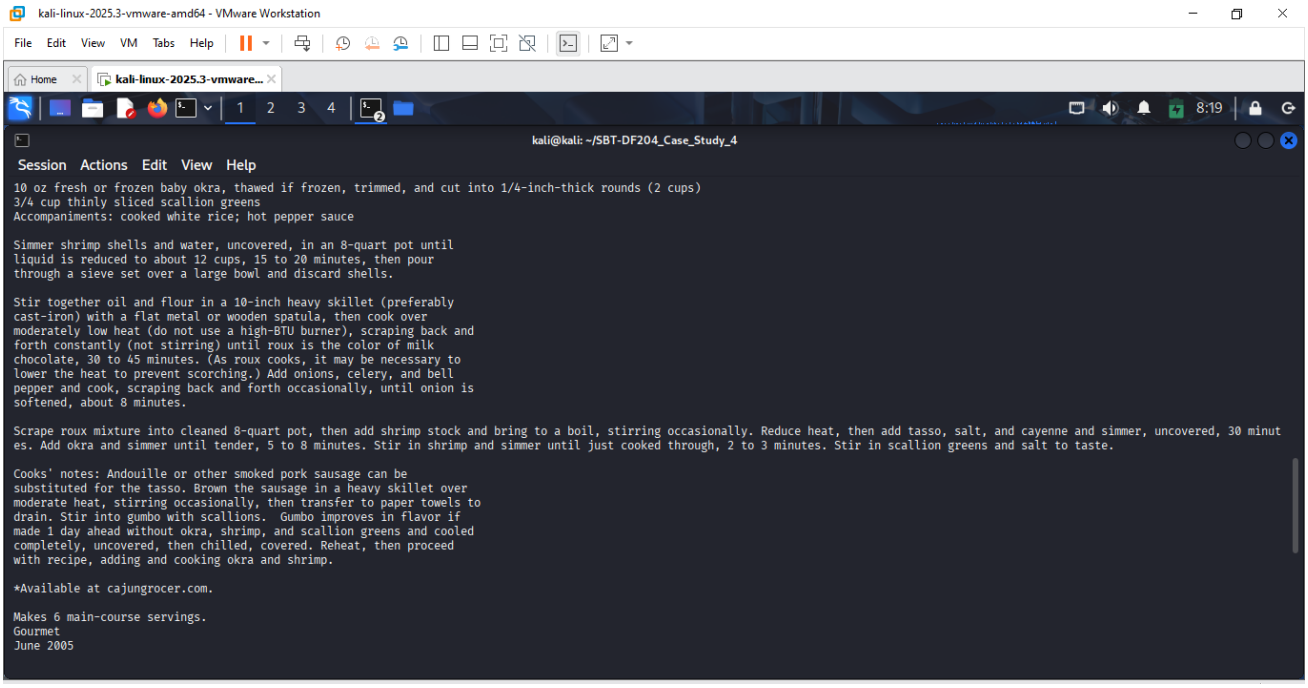


Fig 2.5 Recovered Gumbo Text Examination page 2.

Explanation

This continuation shows further readable content from the recovered Gumbo file. The text remained intact after extraction from the disk image.

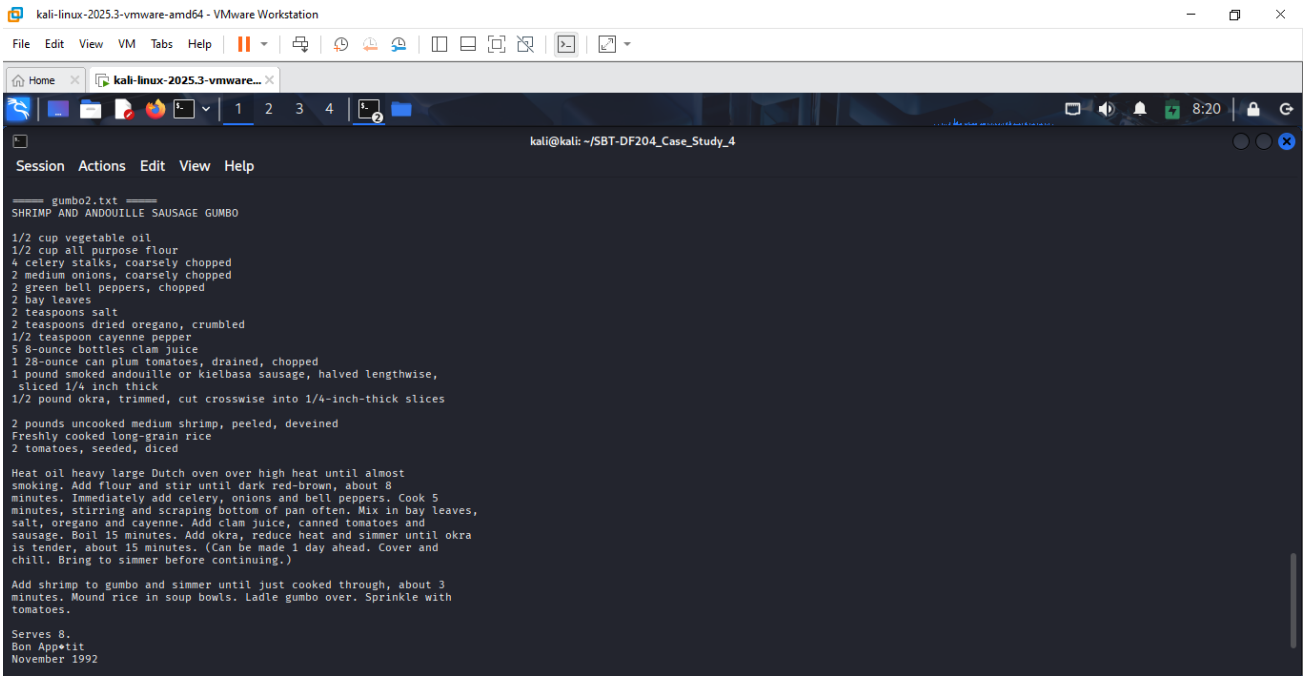


Fig 2.6 Recovered Gumbo Text Examination page 3.

Explanation

The final continuation completes the displayed Gumbo content. It demonstrates that the file could be recovered and read successfully.

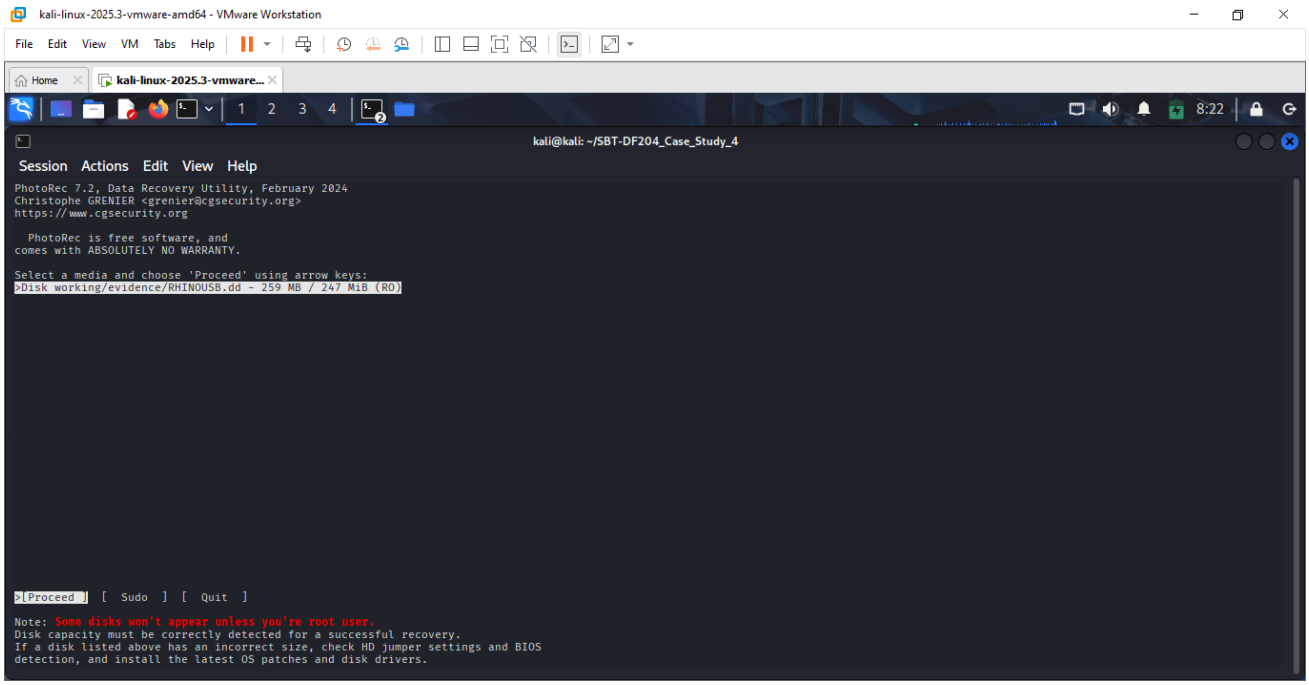


Fig 2.7 PhotoRec Source Disk Selection.

Explanation

PhotoRec displays the USB image selected as the source for recovery. This confirms that recovery was being performed against the working disk image rather than the preserved original.

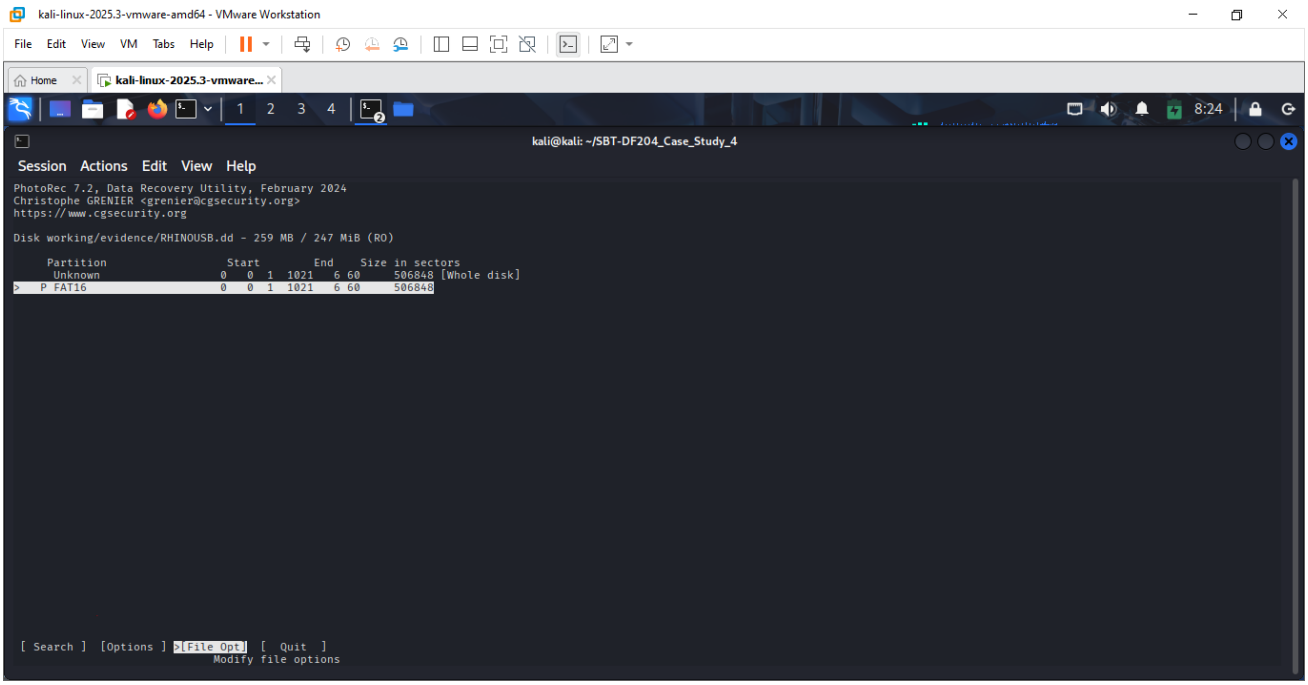


Fig 2.8 PhotoRec Source Disk Selection.

Explanation

The screenshot shows PhotoRec recognising the FAT16 area of the USB image. The FAT16 entry was selected as the recovery target.

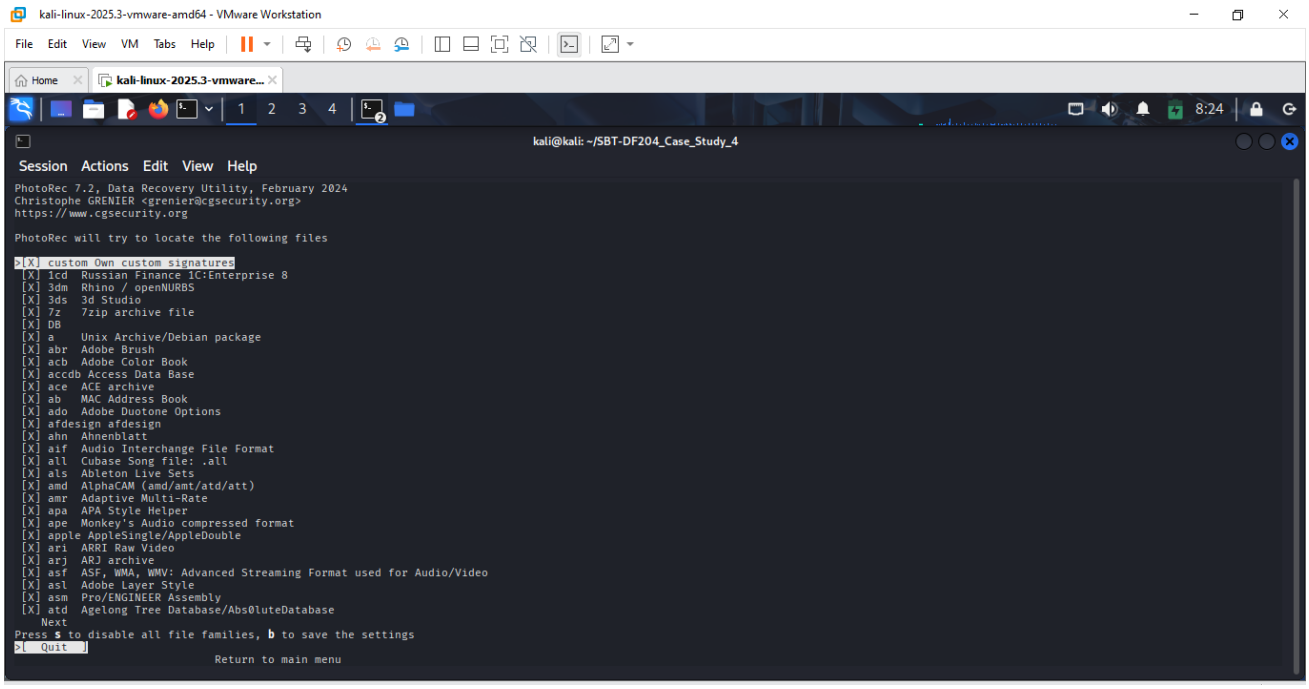


Fig 2.9 PhotoRec File Type Options.

Explanation

PhotoRec displays the recoverable file families enabled for carving. The default selections were retained so the first recovery pass would not unnecessarily exclude potential artefacts.

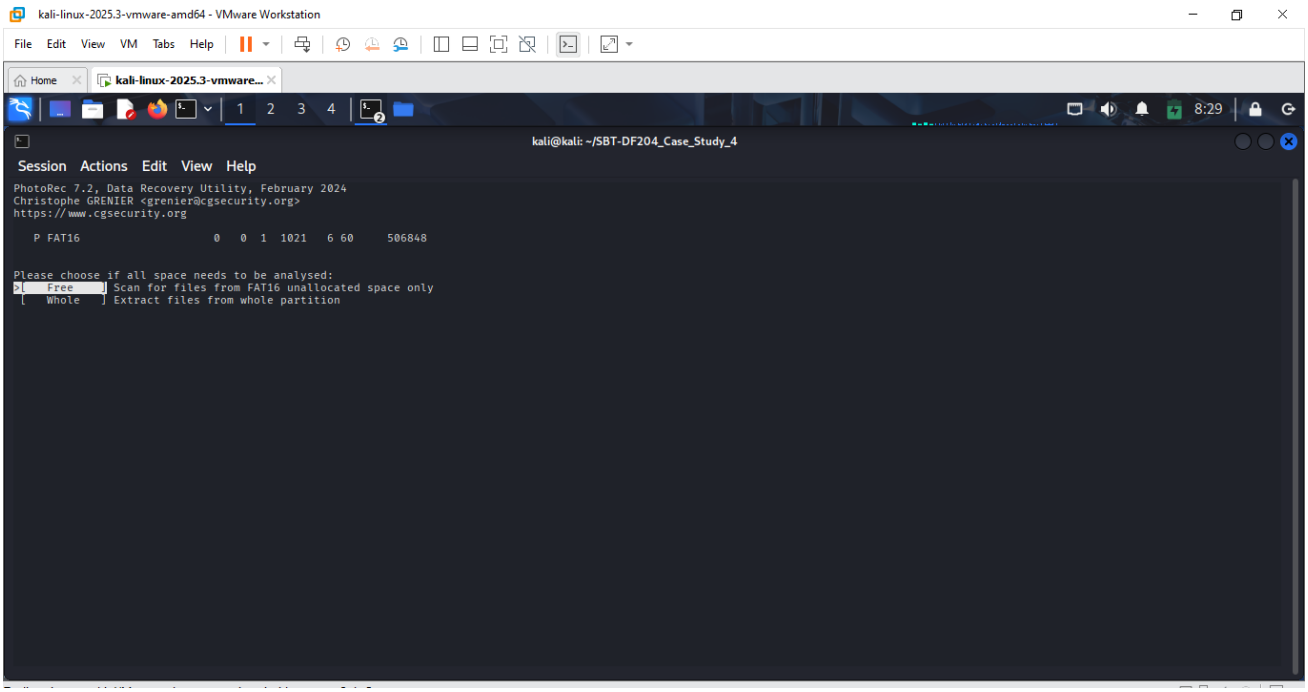


Fig 3.0 PhotoRec Recovery Search Configuration.

Explanation

The screen shows the FAT16 image prepared for the recovery search after the file-type options were reviewed. This documents the recovery configuration before scanning began.

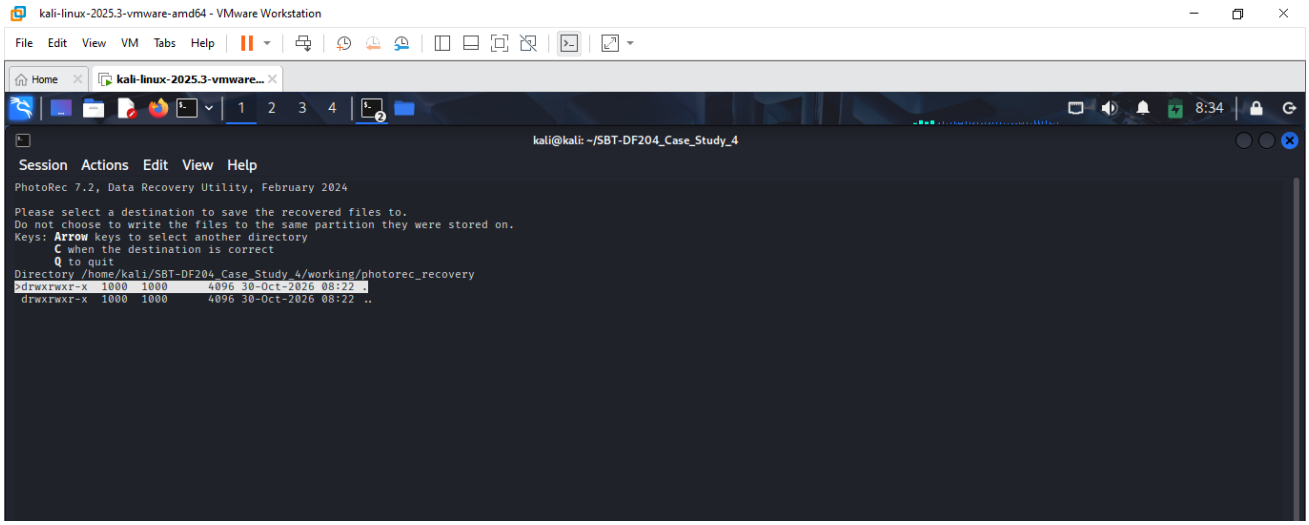


Fig 3.1 PhotoRec Recovery Destination Selection.

Explanation

The chosen destination points to the separate working/photorec_recovery location. Keeping recovered files outside the source image directory protected the evidence from modification.

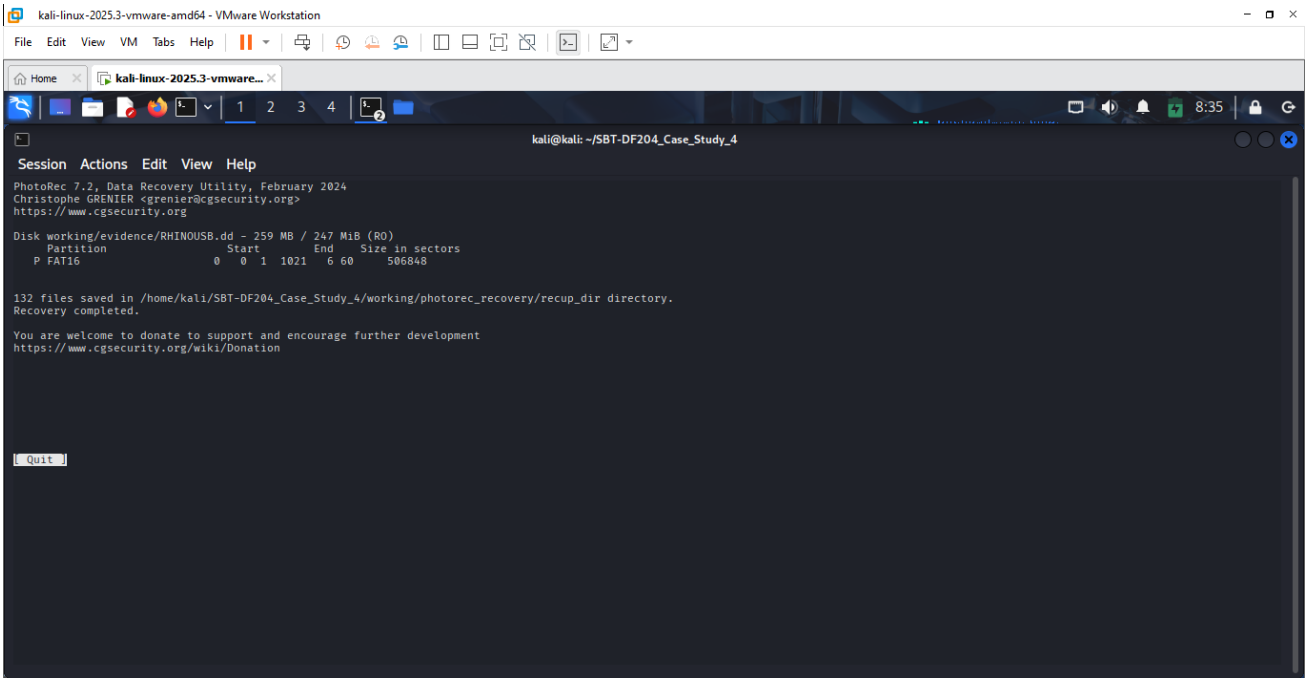


Fig 3.2 PhotoRec USB Recovery Completion.

Explanation

PhotoRec reports that recovery completed successfully and that 132 files were saved. This establishes the total number of carved files produced during the recovery stage.

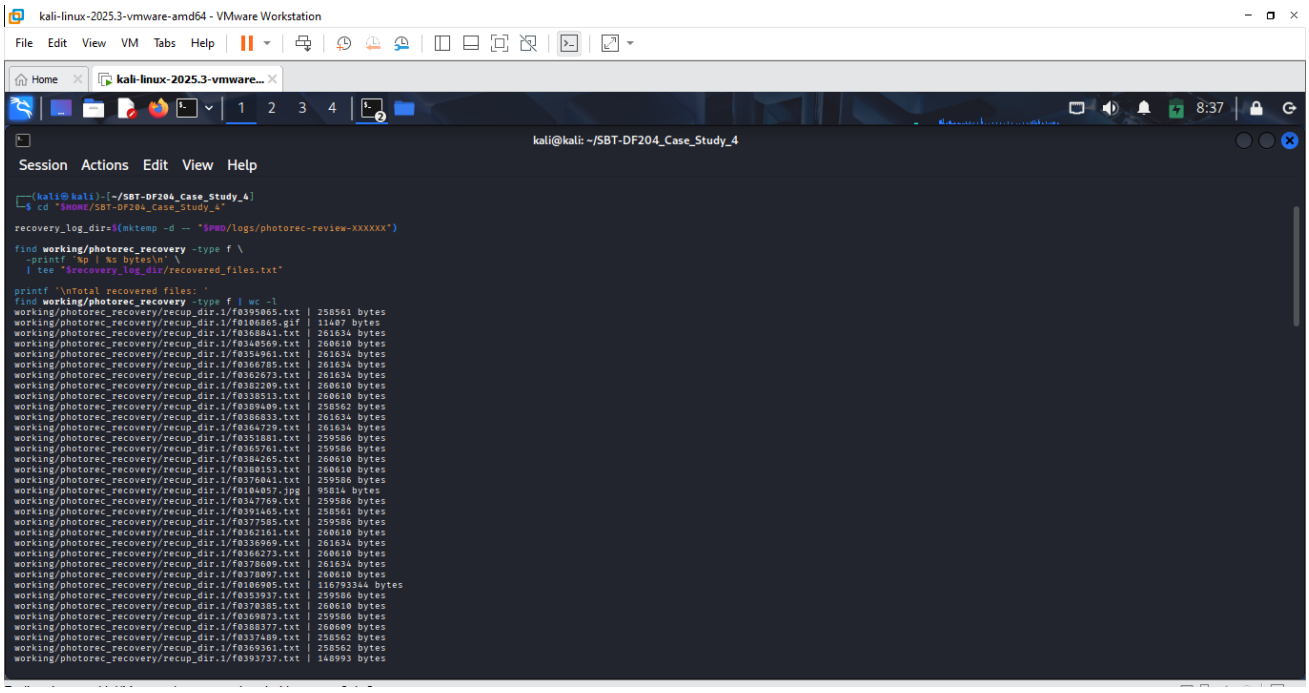


Fig 3.3 Recovered File Inventory Page 1.

Explanation

The screenshot begins the generated list of recovered files and their byte sizes. This inventory provides a record of what PhotoRec produced.

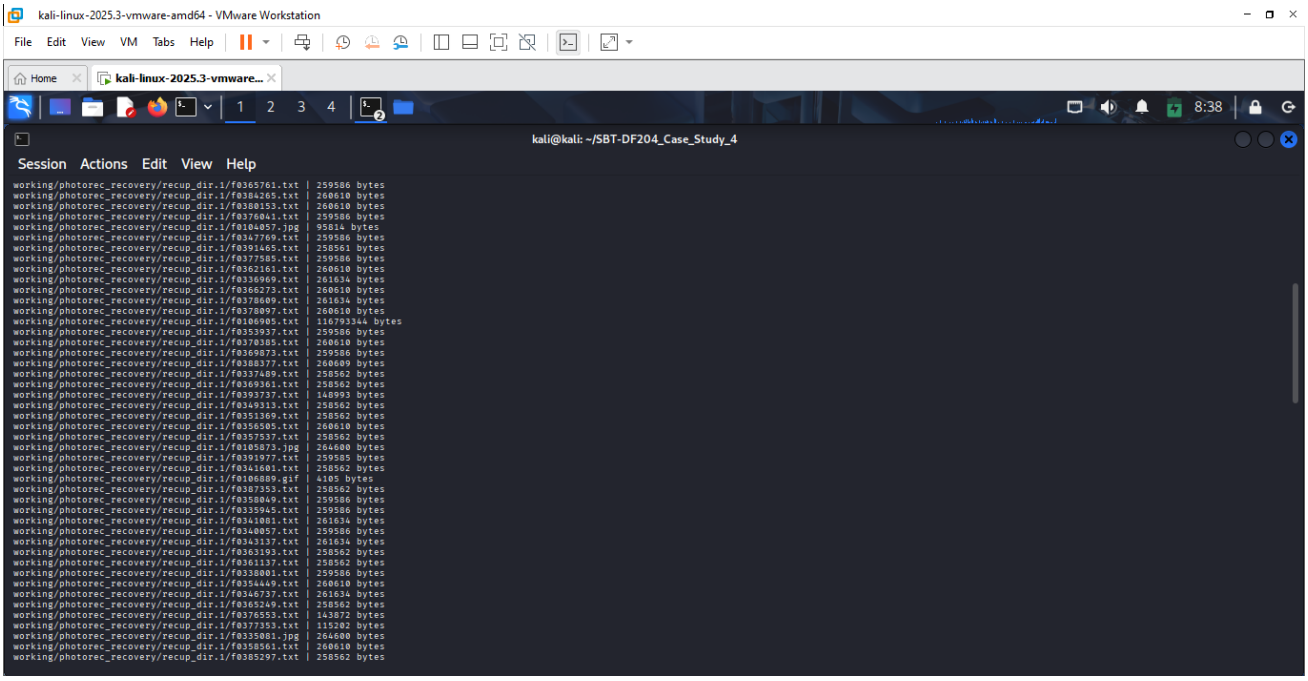


Fig 3.4 Recovered File Inventory Page 2.

Explanation

This page continues the list of recovered filenames and sizes. It demonstrates that the recovery output contained a large number of individual artefacts.

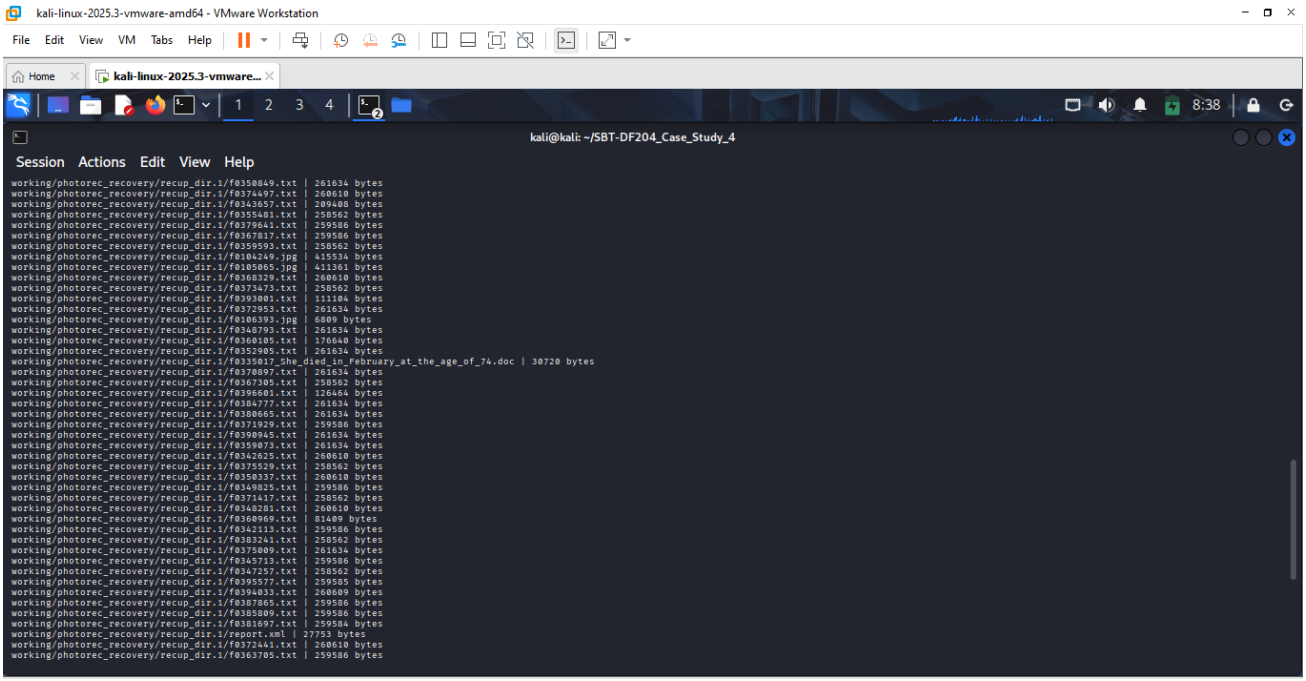


Fig 3.5 Recovered File Inventory Page 3.

Explanation

The screenshot continues the recovered-file inventory. Keeping the complete list supports traceability between later hashes and the recovery output.

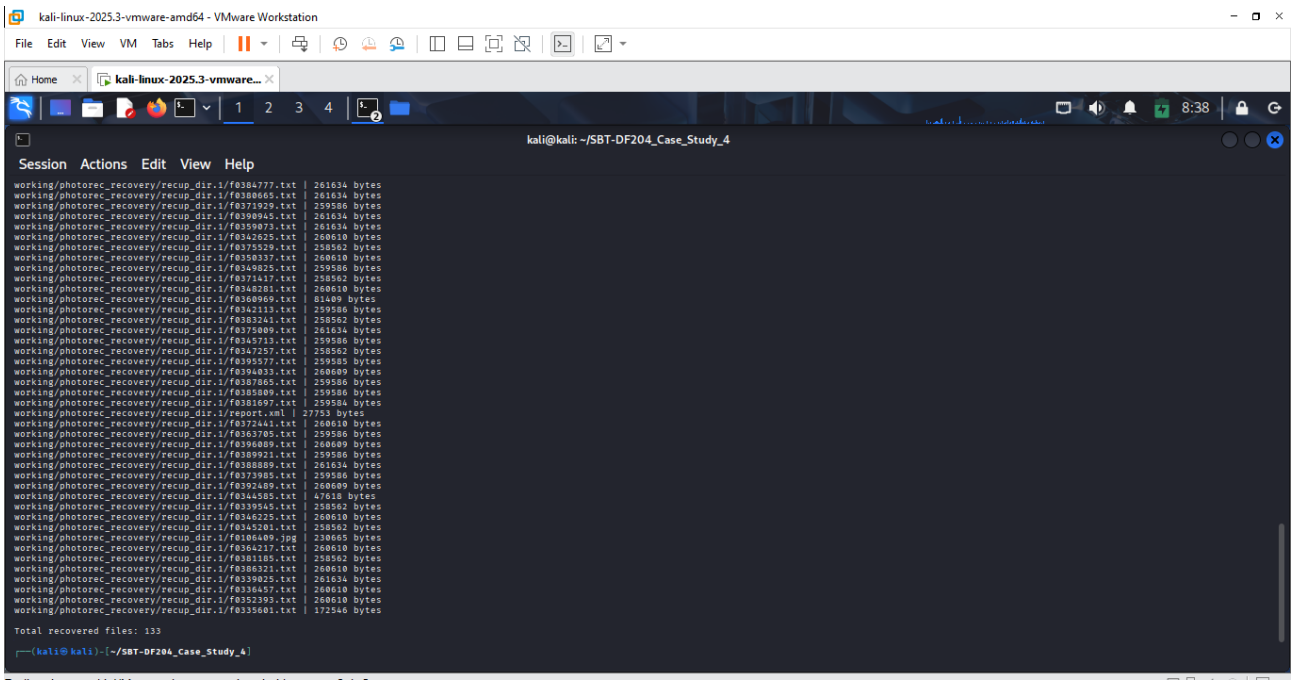


Fig 3.6 Recovered File Inventory Page 4.

Explanation

The final inventory section reaches the end of the recovered-file listing and records the total. This completes the recovery manifest.

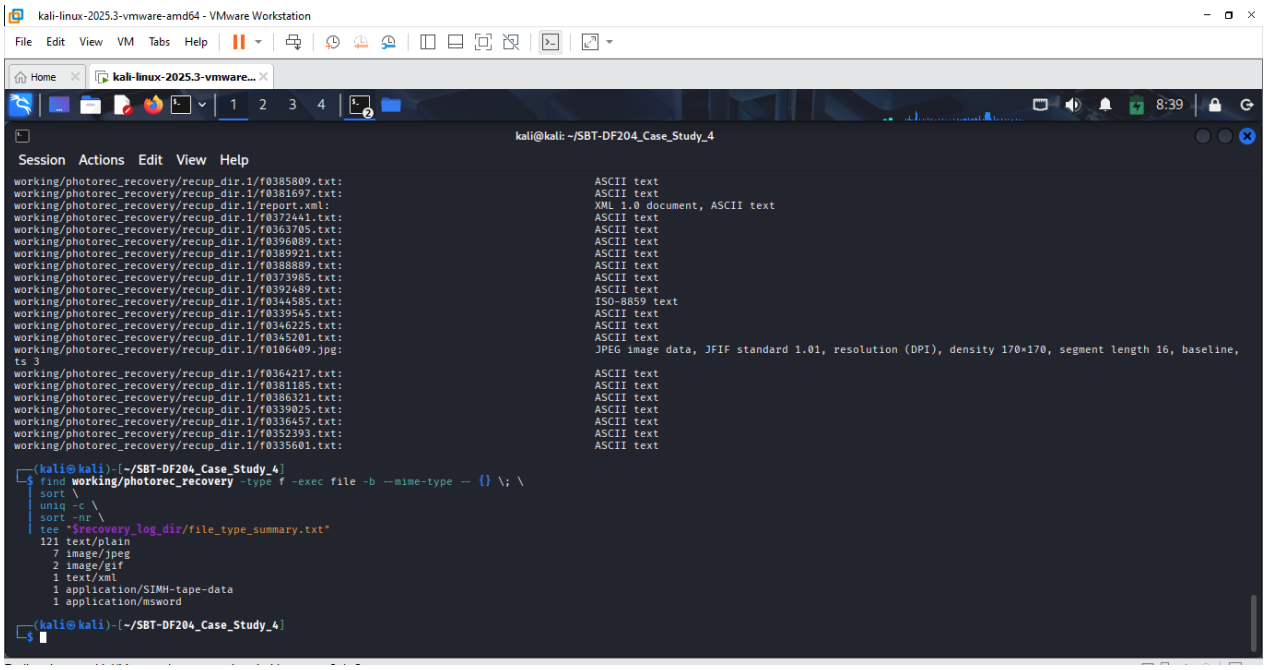


Fig 3.7 Recovered File Type Summary.

Explanation

The recovered material is grouped by MIME/file type, including text, JPEG, GIF and Microsoft Word data. This helped narrow the investigation toward the media and document files relevant to the case.

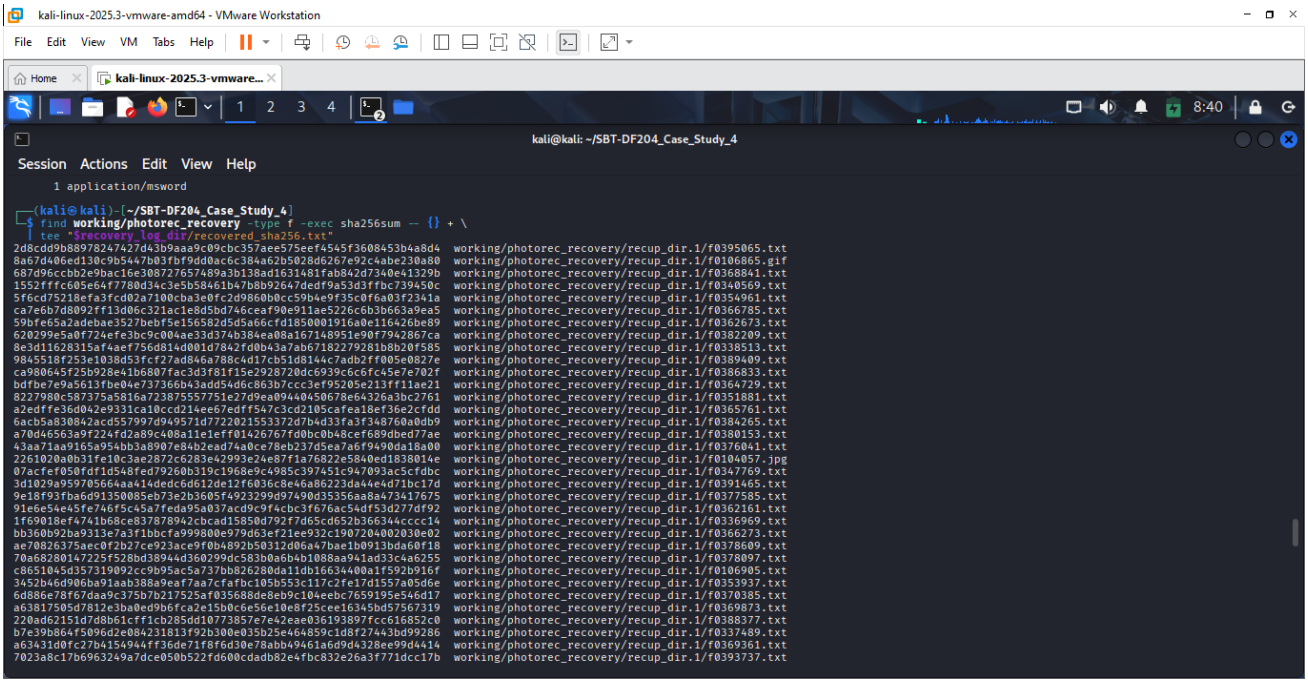


Fig 3.8 Recovered File SHA-256 Hashes Page 1.

Explanation

The screenshot begins the SHA-256 record for all carved files. These hashes allow recovered artefacts to be uniquely compared.

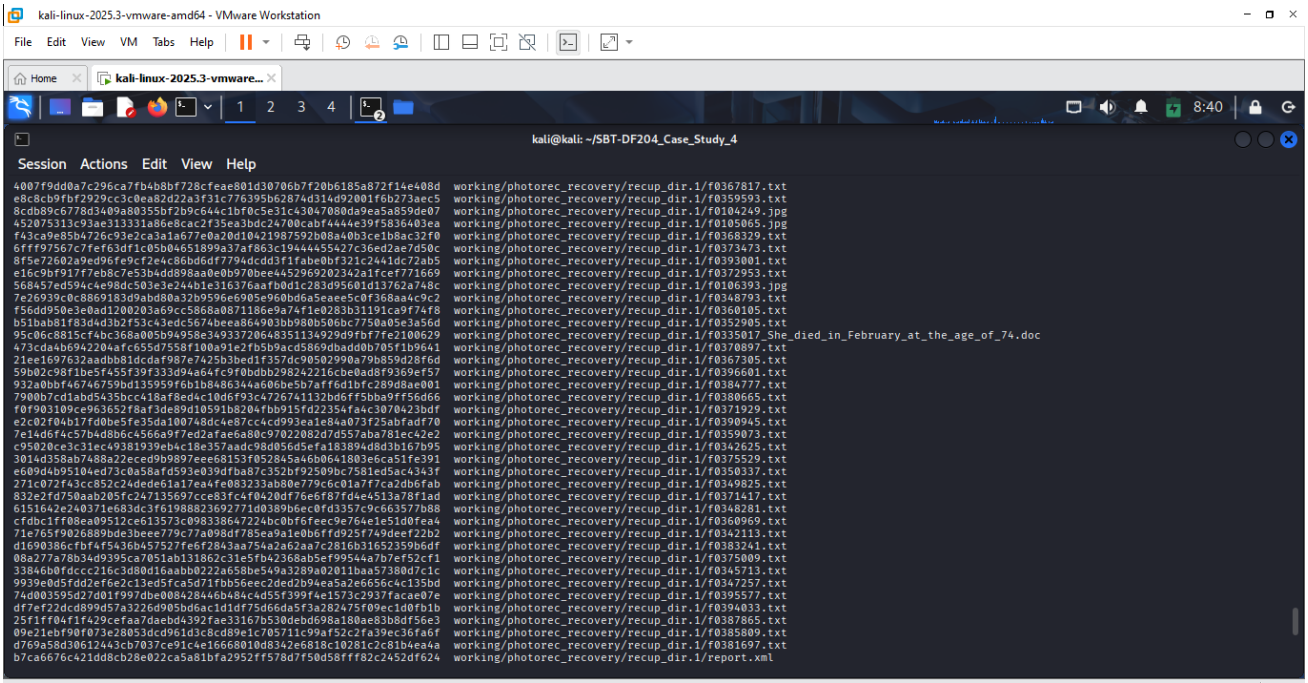


Fig 3.9 Recovered File SHA-256 Hashes Page 2.

Explanation

This continuation records additional hashes from the PhotoRec output. It supports later duplicate detection and cross-source comparison.

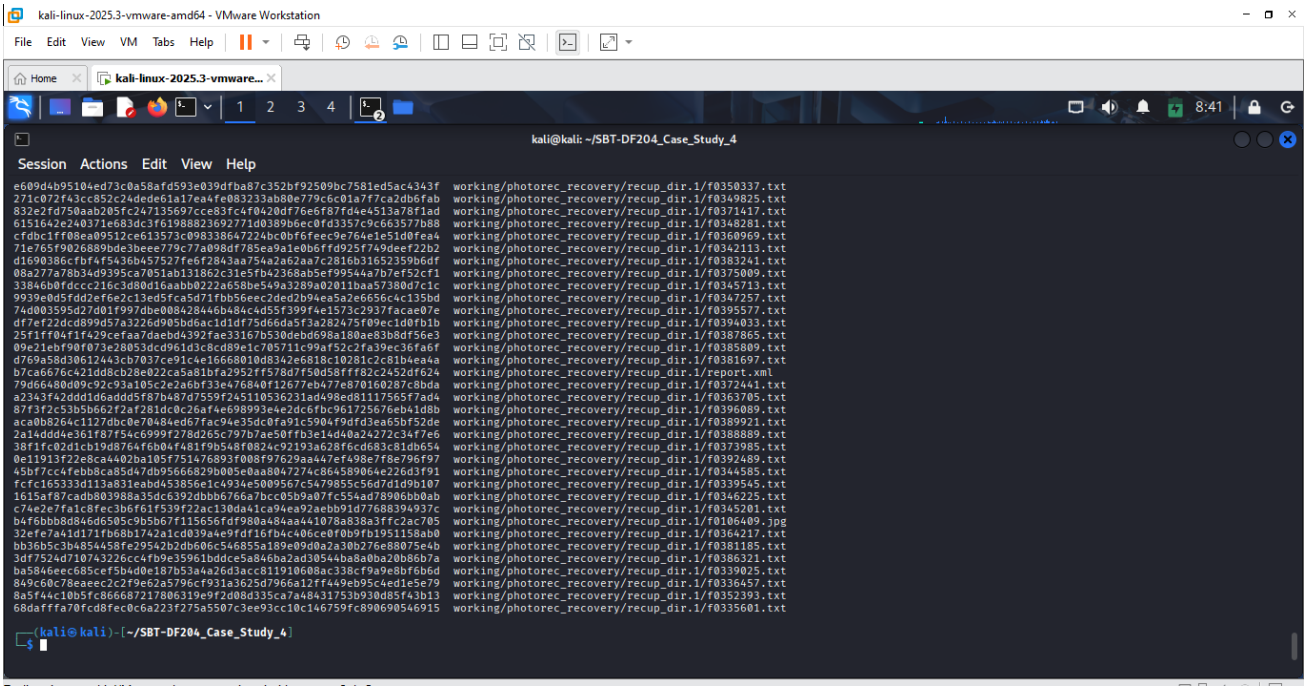


Fig 3.10 Recovered File SHA-256 Hashes Page 3.

Explanation

The remaining recovered-file hashes are shown here. Together, the hash records form the integrity inventory for the recovered material.

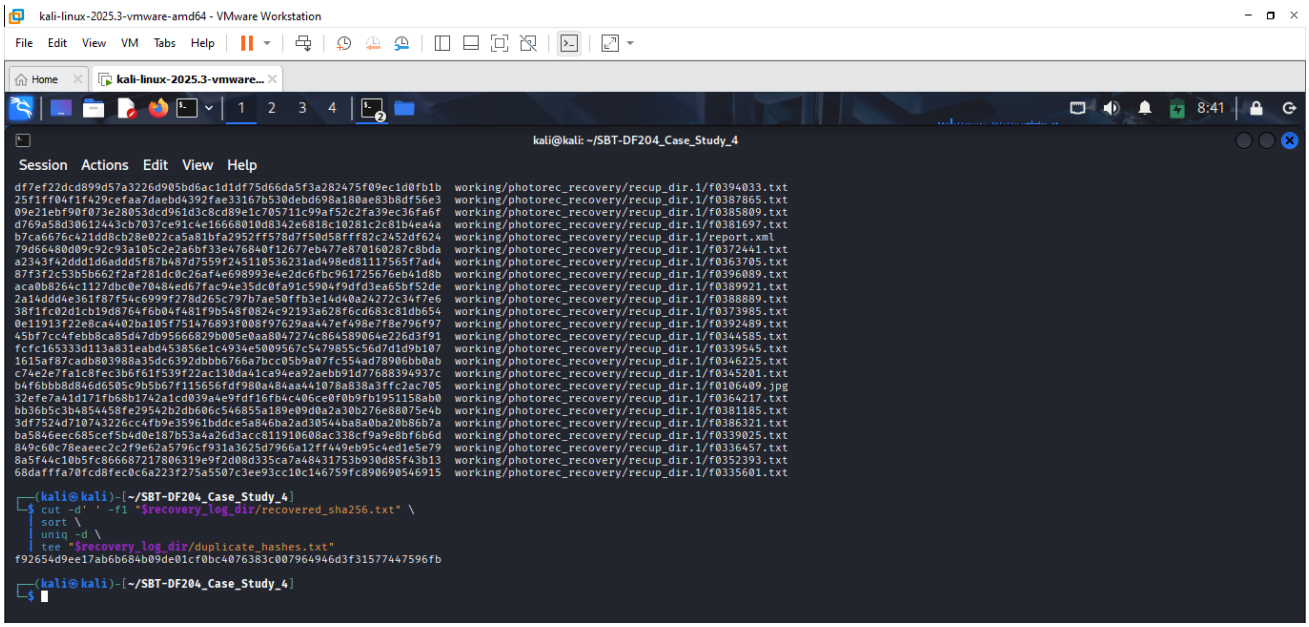


Fig 3.11 Duplicate SHA-256 Hash Identification.

Explanation

The command searches the recovered hash list for repeated values and identifies a duplicate hash. This indicates that at least two recovered filenames contain the same binary content.

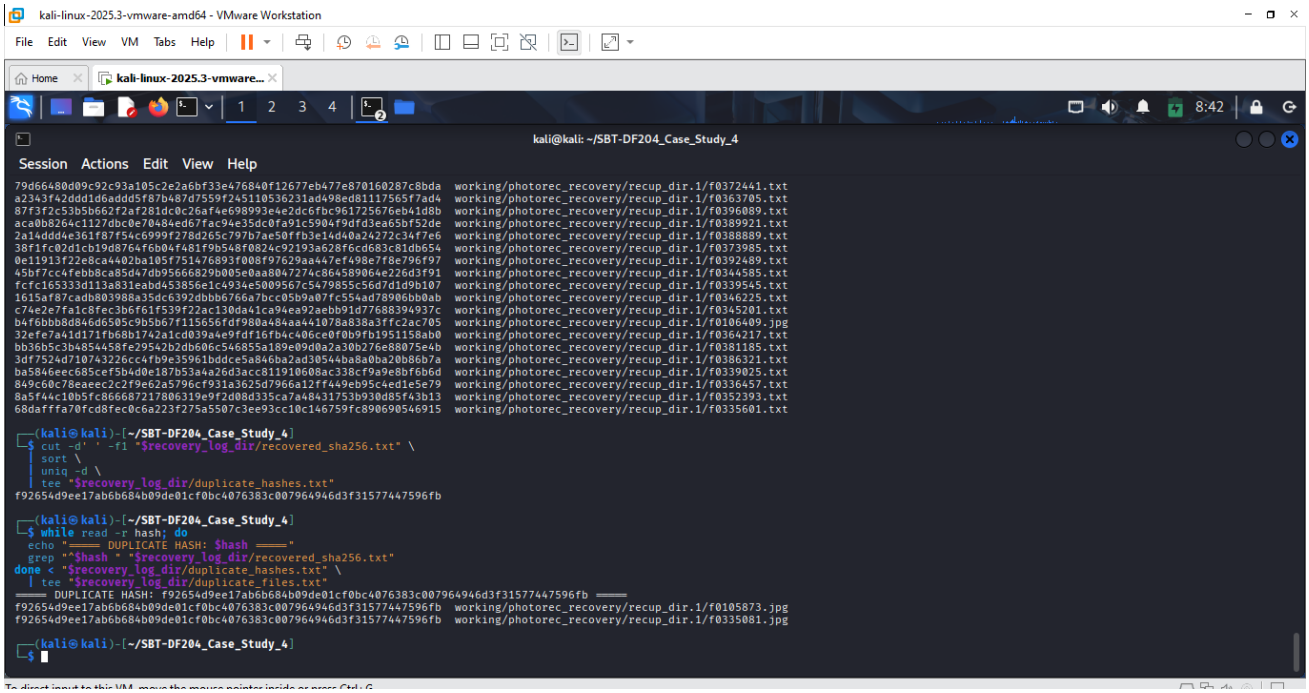


Fig 3.12 Duplicate JPEG Validation.

Explanation

The repeated hash is linked to f0105873.jpg and f0335081.jpg. This establishes that the two recovered filenames represent duplicate JPEG content.

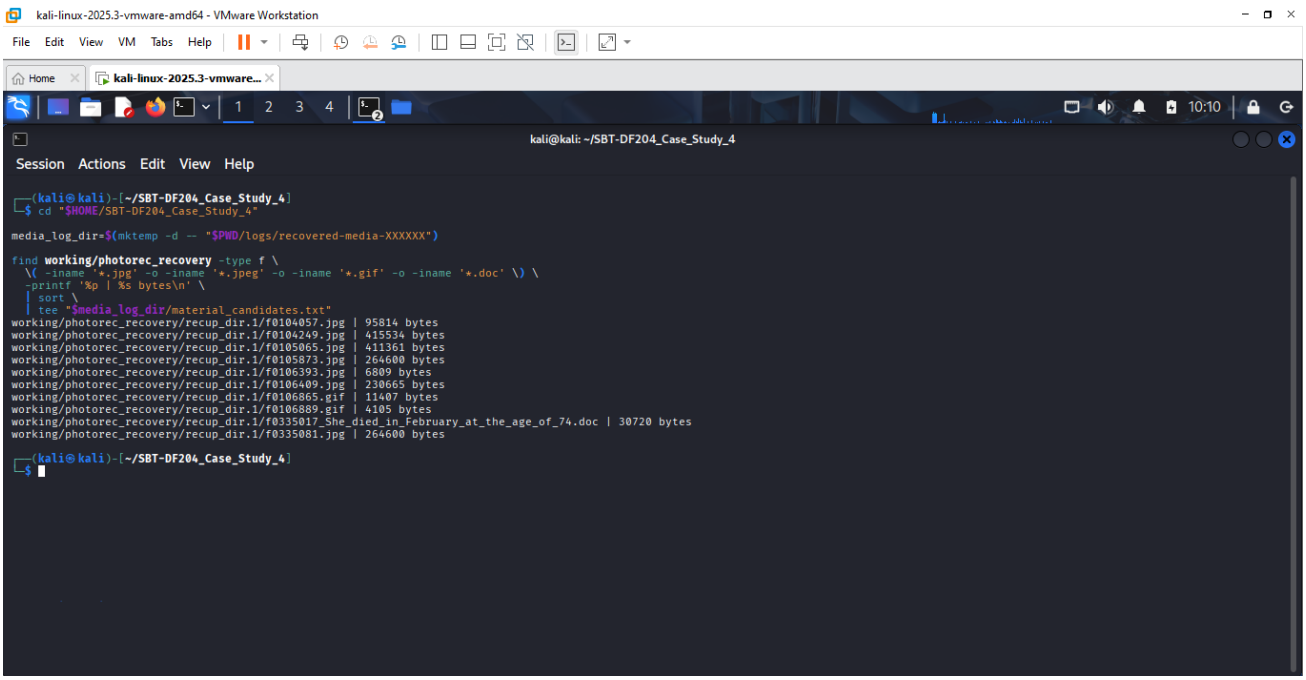


Fig 3.13 Recovered Media Candidate Inventory.

Explanation

The screenshot narrows the recovery results to JPEG, GIF and Word-document candidates and records their sizes. These files became the focus of the metadata and hidden-content review.

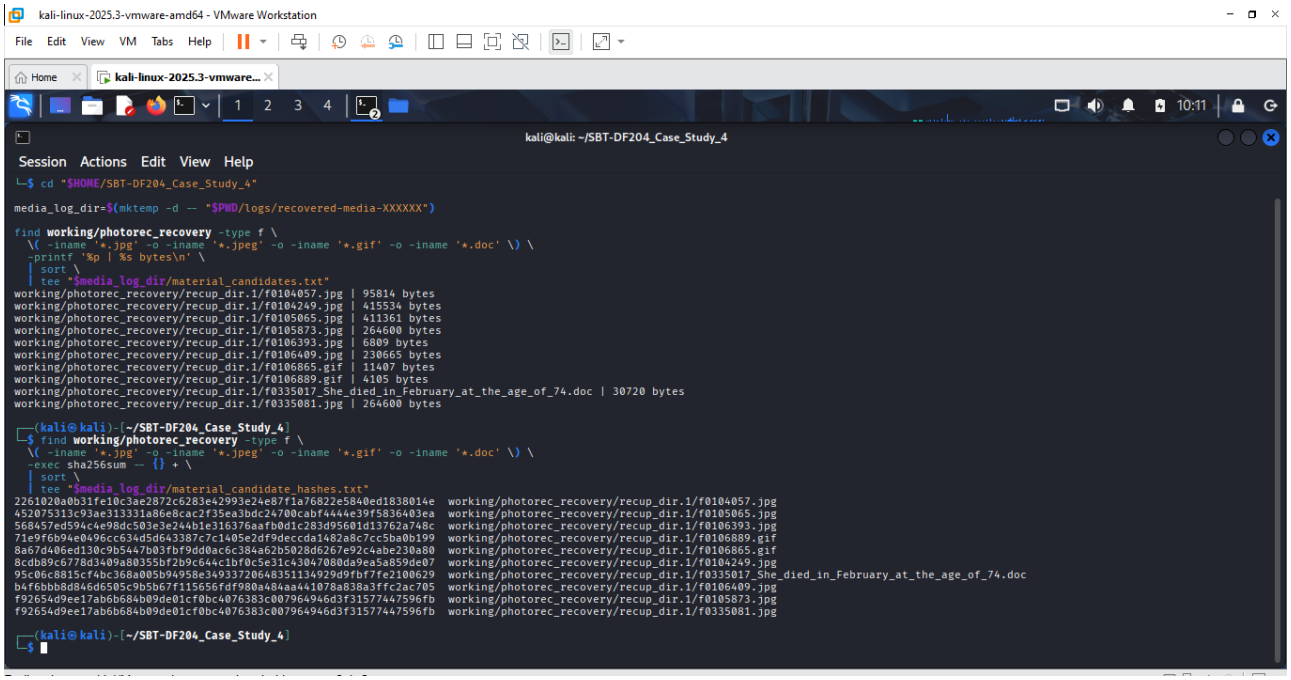


Fig 3.14 Recovered Media and Document Hash Inventory.

Explanation

SHA-256 values are recorded for the selected images and recovered Word document. This created a smaller evidence set for detailed examination.

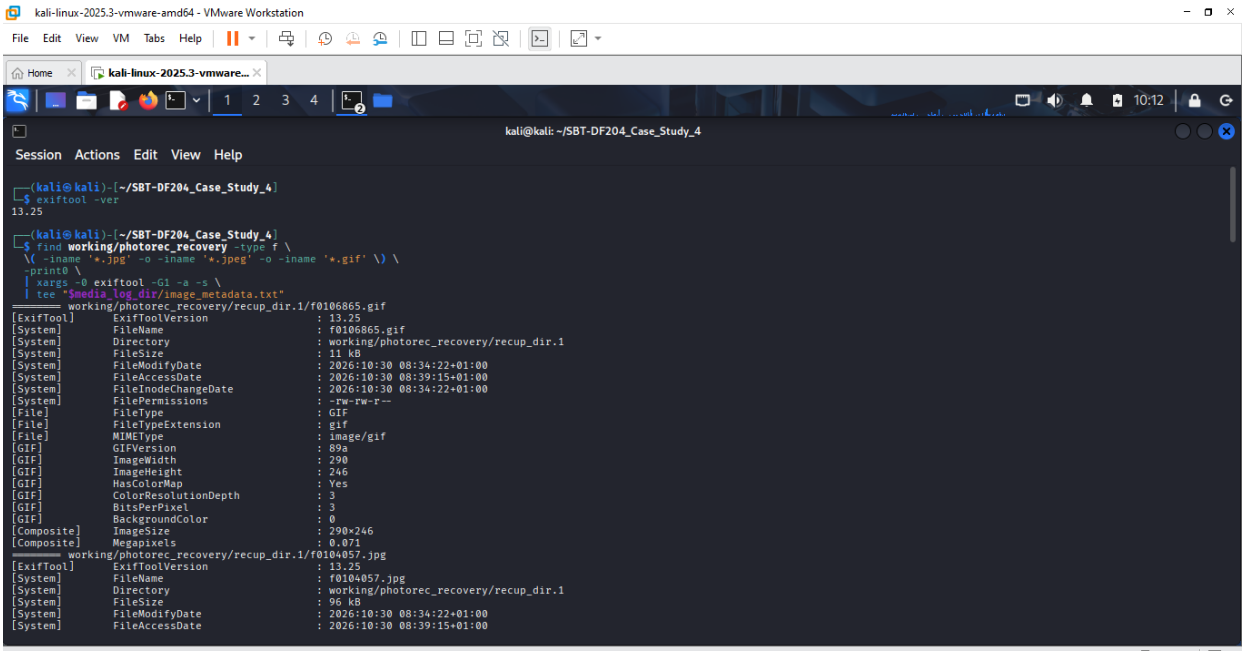


Fig 3.15 Recovered Image Metadata Analysis Page 1.

Explanation

ExifTool begins displaying technical metadata for the recovered images. The output includes file format, dimensions and encoding information.

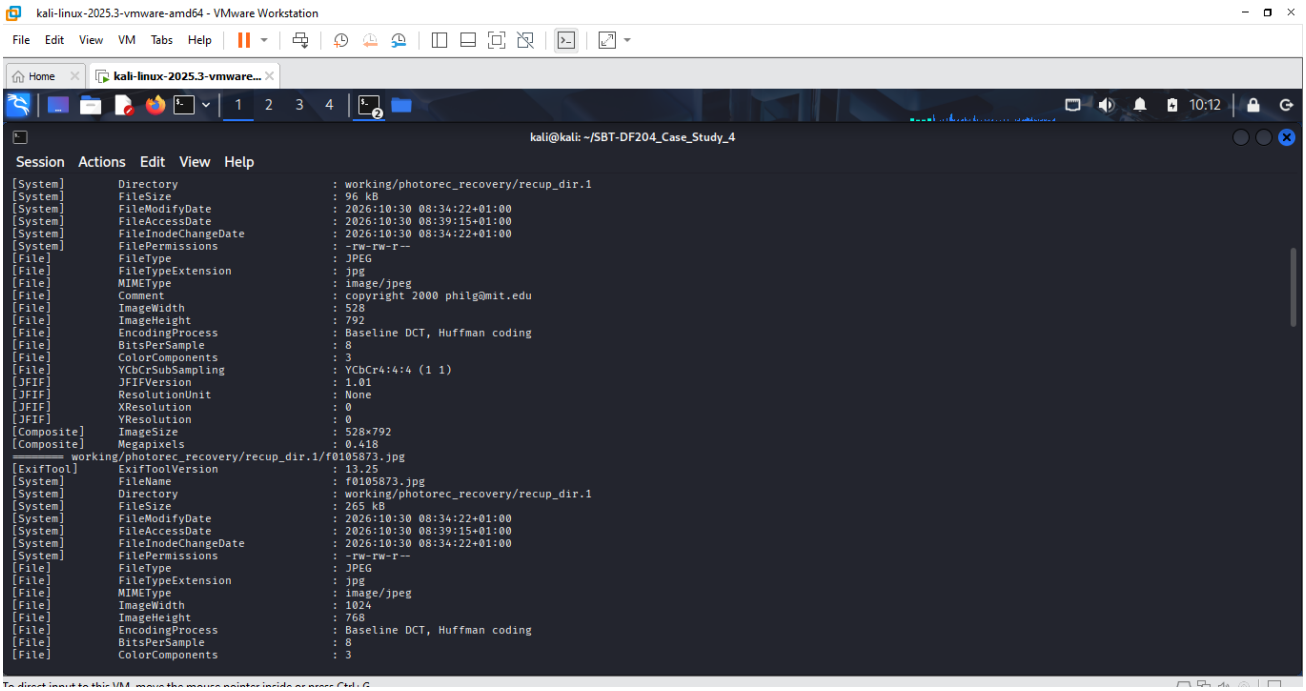


Fig 3.16 Recovered Image Metadata Analysis Page 2.

Explanation

The metadata review continues across additional recovered images. This provides technical characteristics without modifying the original carved files.

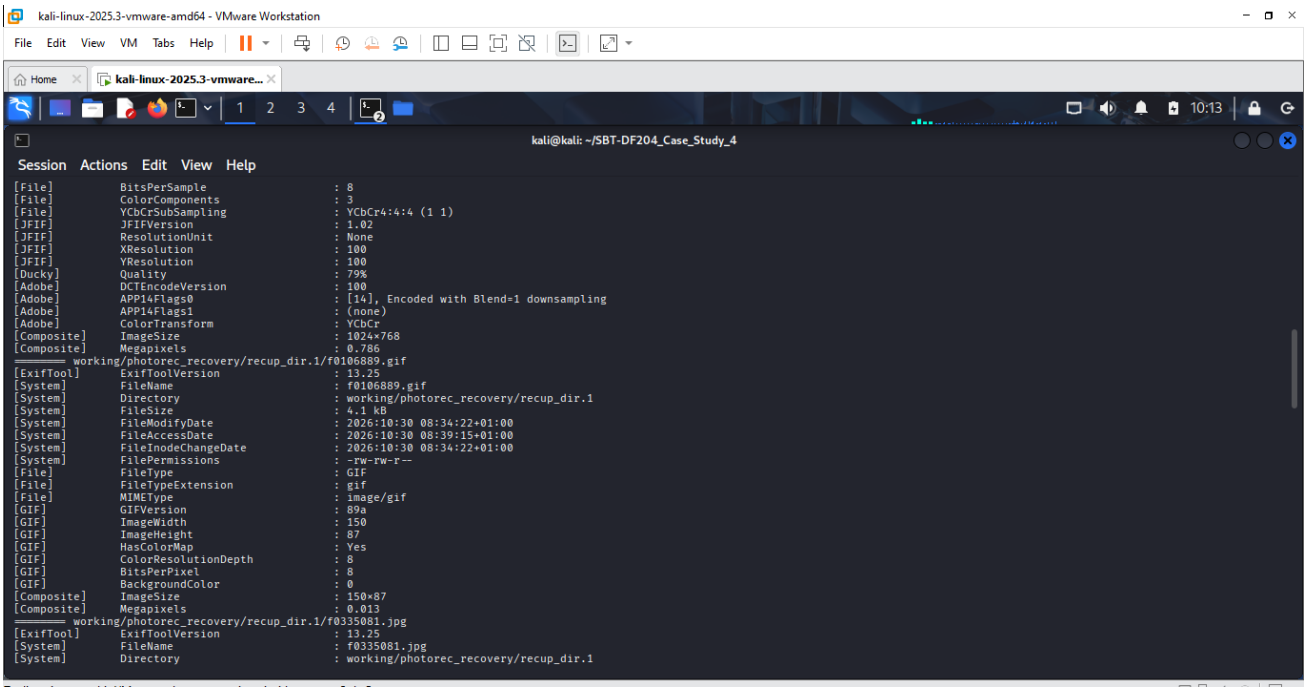


Fig 3.17 Recovered Image Metadata Analysis Page 3.

Explanation

ExifTool displays further image properties, including JPEG or GIF characteristics. The results confirm the recovered media formats.

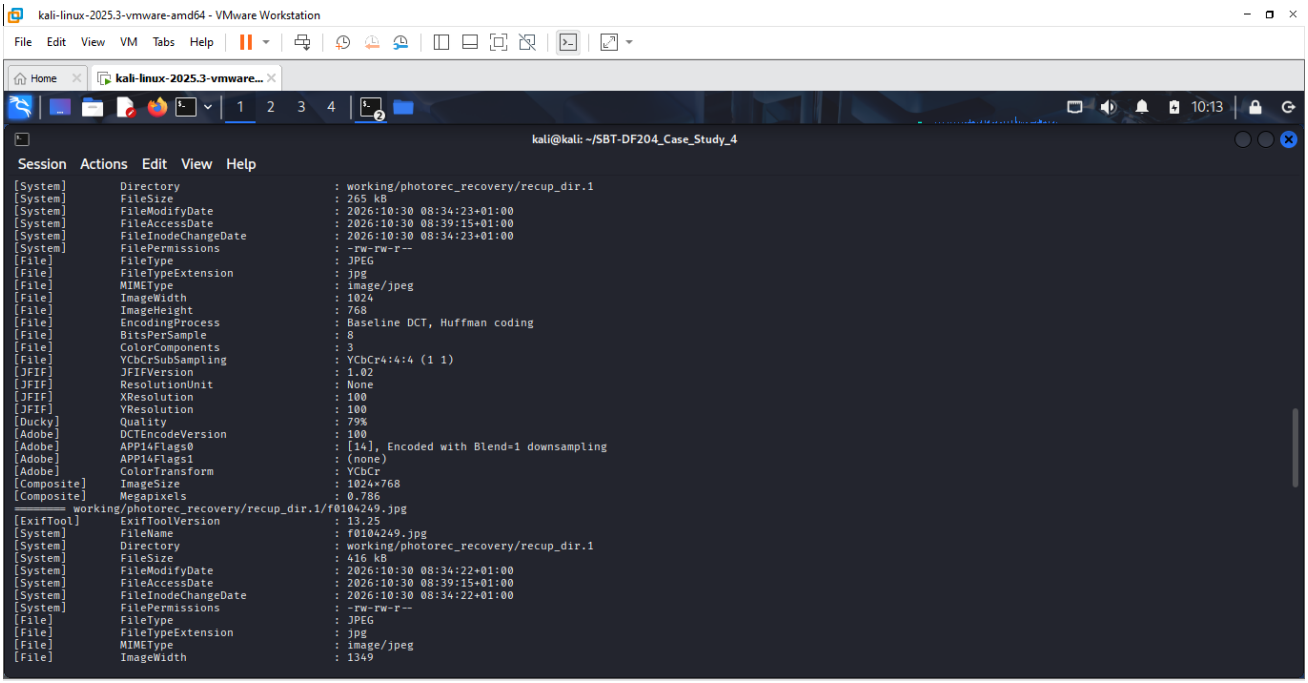


Fig 3.18 Recovered Image Metadata Analysis Page 4.

Explanation

This continuation records additional image dimensions, compression and colour information. These fields contribute to validating the recovered media.

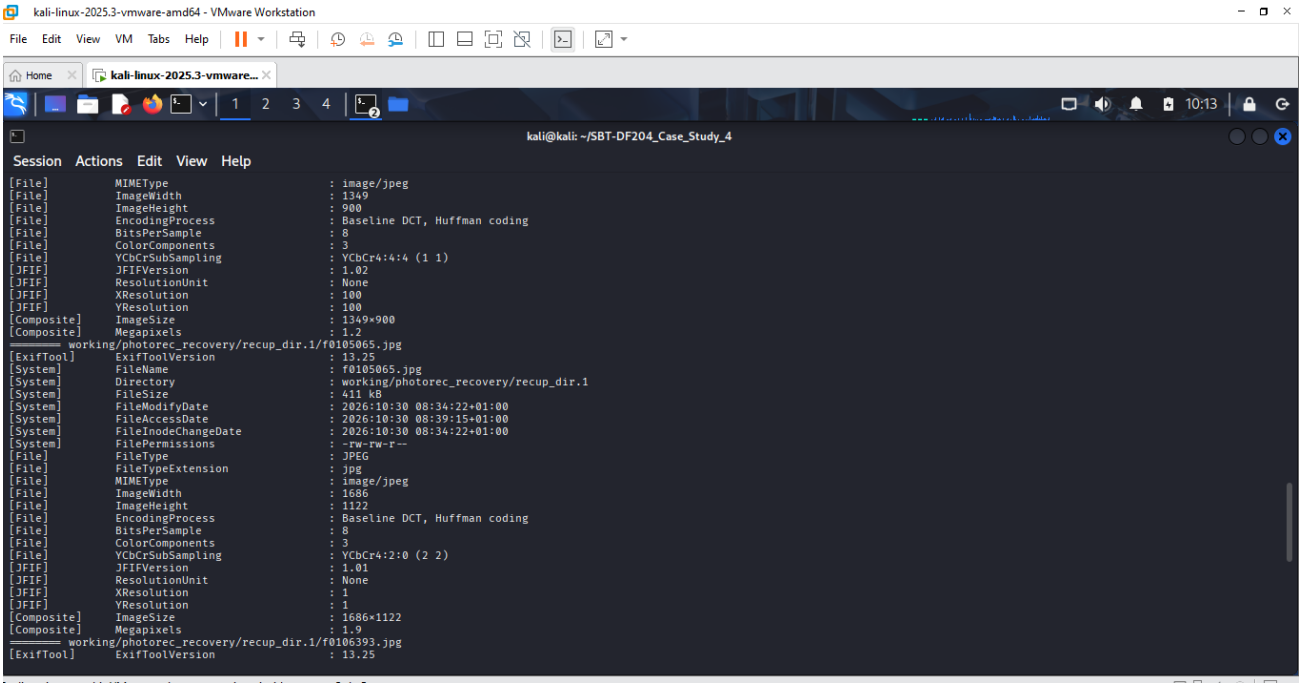


Fig 3.19 Recovered Image Metadata Analysis Page 5.

Explanation

More recovered-image metadata is shown for the remaining files. No interpretation beyond the displayed technical metadata is required.

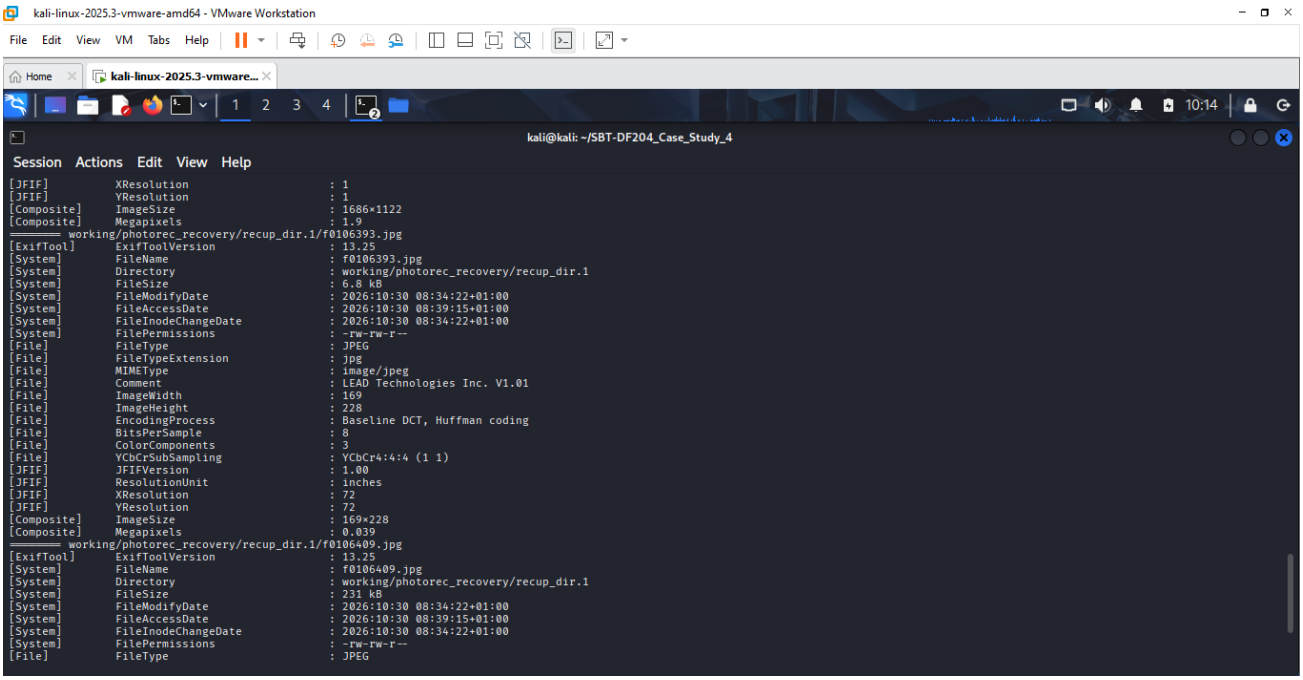


Fig 3.20 Recovered Image Metadata Analysis Page 6.

Explanation

The screenshot continues the ExifTool review of the final image candidates. It preserves the technical results used in the media examination.

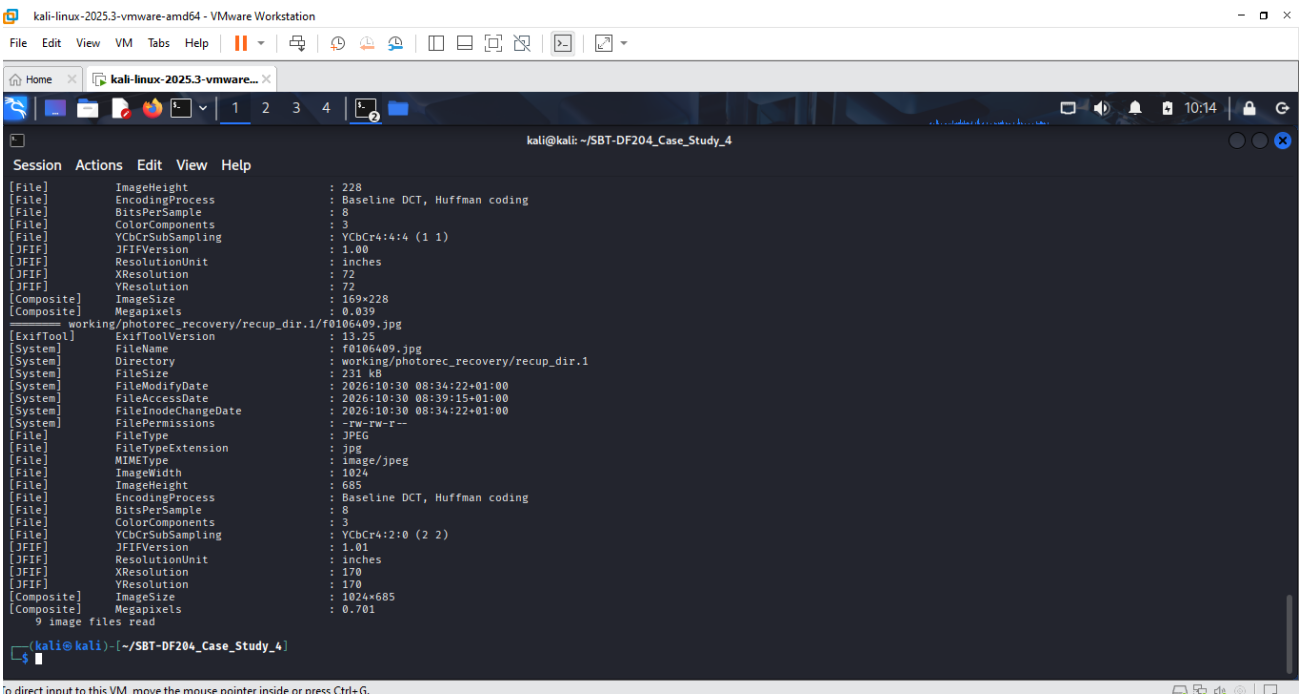


Fig 3.21 Recovered Image Metadata Analysis Page 7.

Explanation

The final image metadata output completes the examination of the selected media files. It confirms that all relevant recovered images were reviewed.

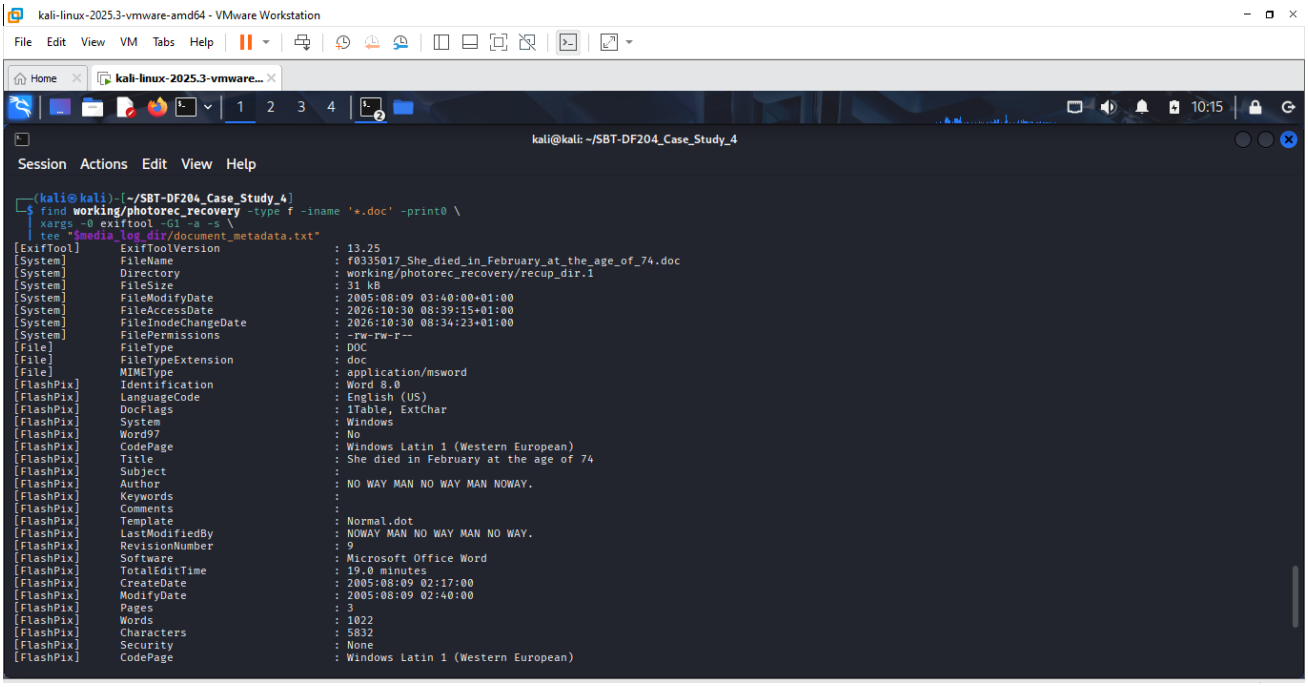


Fig 3.22 Recovered Word Document Metadata Page 1.

Explanation

ExifTool identifies the recovered file as a Microsoft Word document and displays embedded document properties. Relevant fields include its title, author information and application metadata.

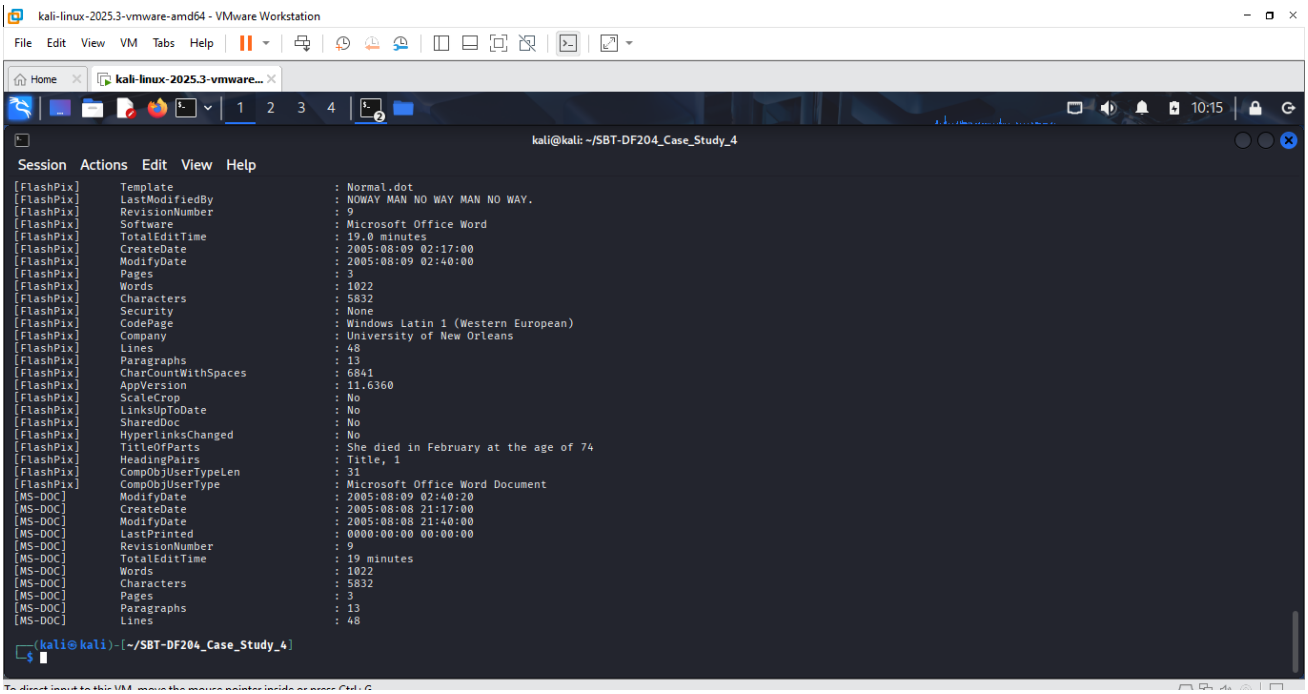


Fig 3.23 Recovered Word Document Metadata Page 2.

Explanation

The continuation records creation/modification information, word count, pages and other document properties. These values are treated as embedded application metadata rather than automatically assumed UTC timestamps.

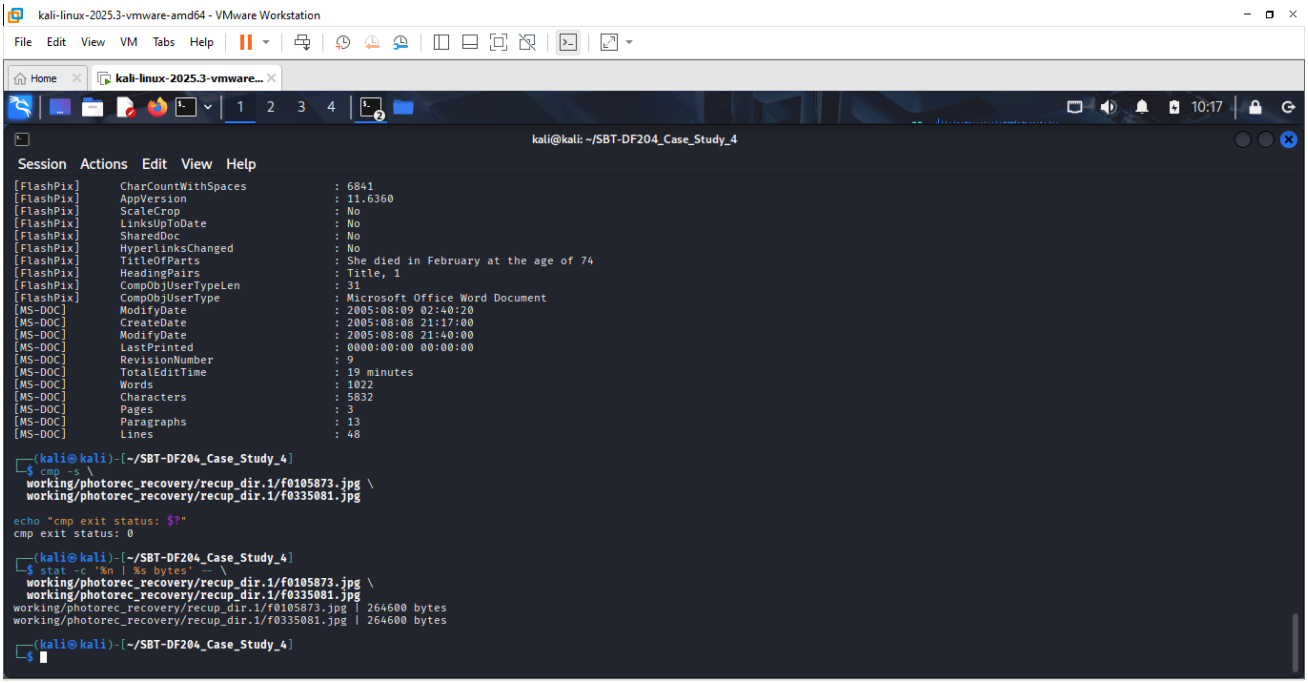


Fig 3.24 Duplicate JPEG Binary and Size Validation.

Explanation

cmp returns exit status 0 for the duplicate JPEG pair and both files show the same byte size. This independently confirms the SHA-256 duplicate finding.

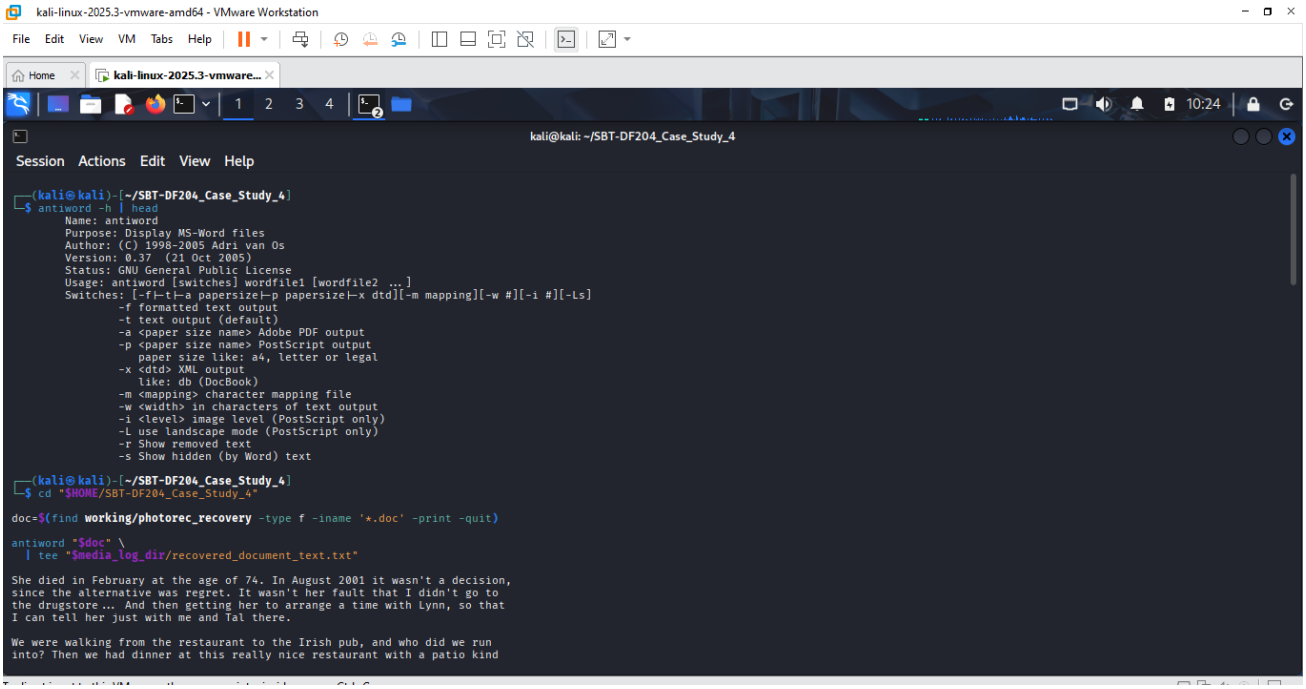


Fig 3.25 Recovered Word Document Text Page 1.

Explanation

Antiword safely extracts readable text from the recovered Word document without opening or executing it. The first section begins the document's narrative content.

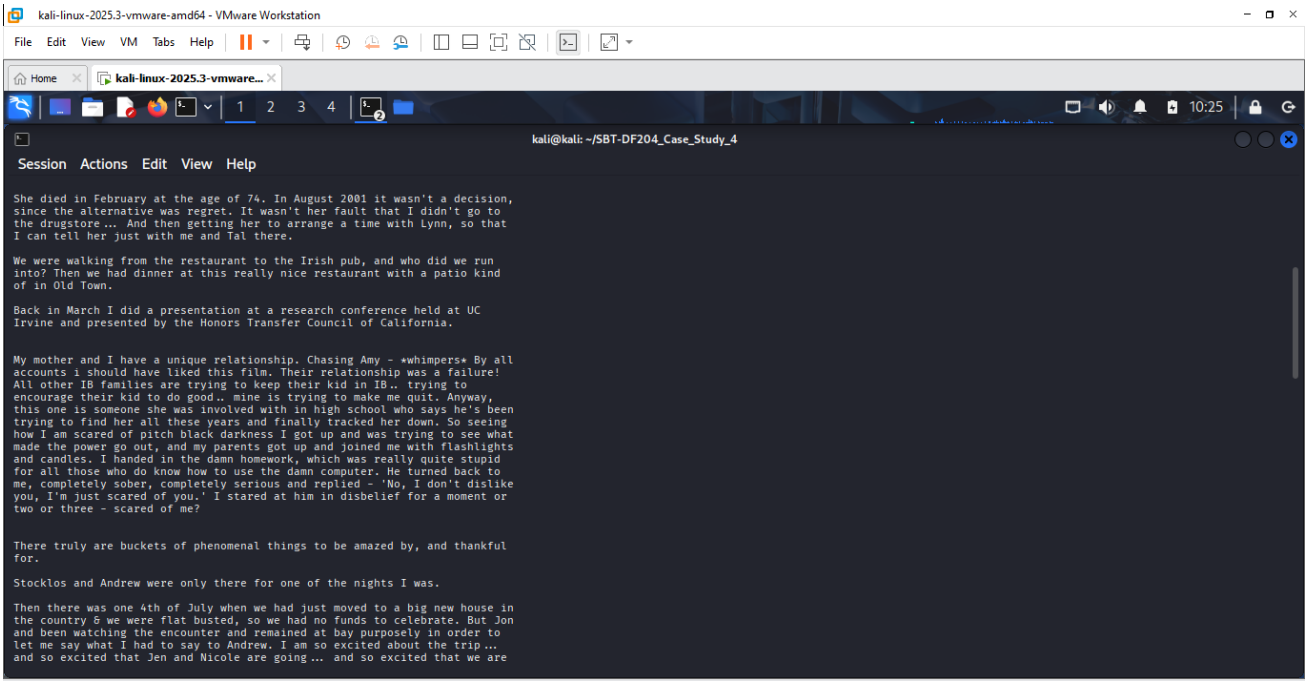


Fig 3.26 Recovered Word Document Text Page 2.

Explanation

This screenshot continues the extracted document text. It demonstrates that substantial readable content survived file recovery.

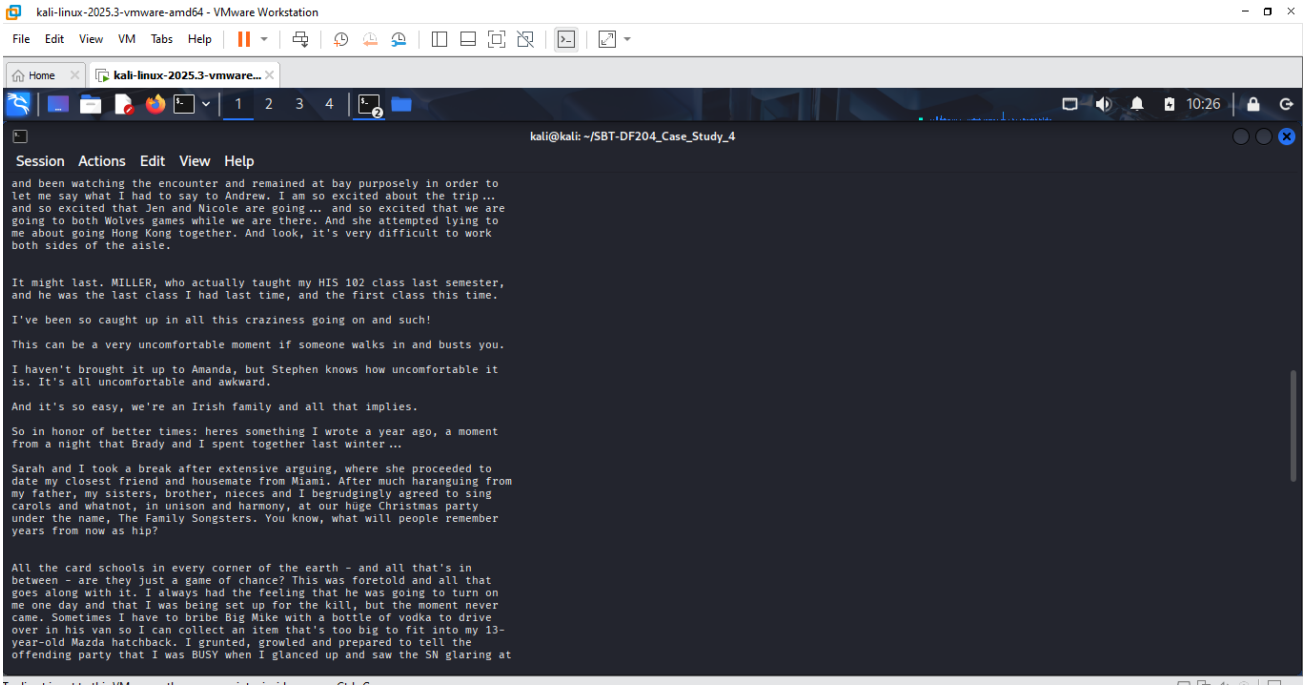


Fig 3.27 Recovered Word Document Text Page 3.

Explanation

The document extraction continues with additional narrative material. The content was reviewed only as recovered evidence.

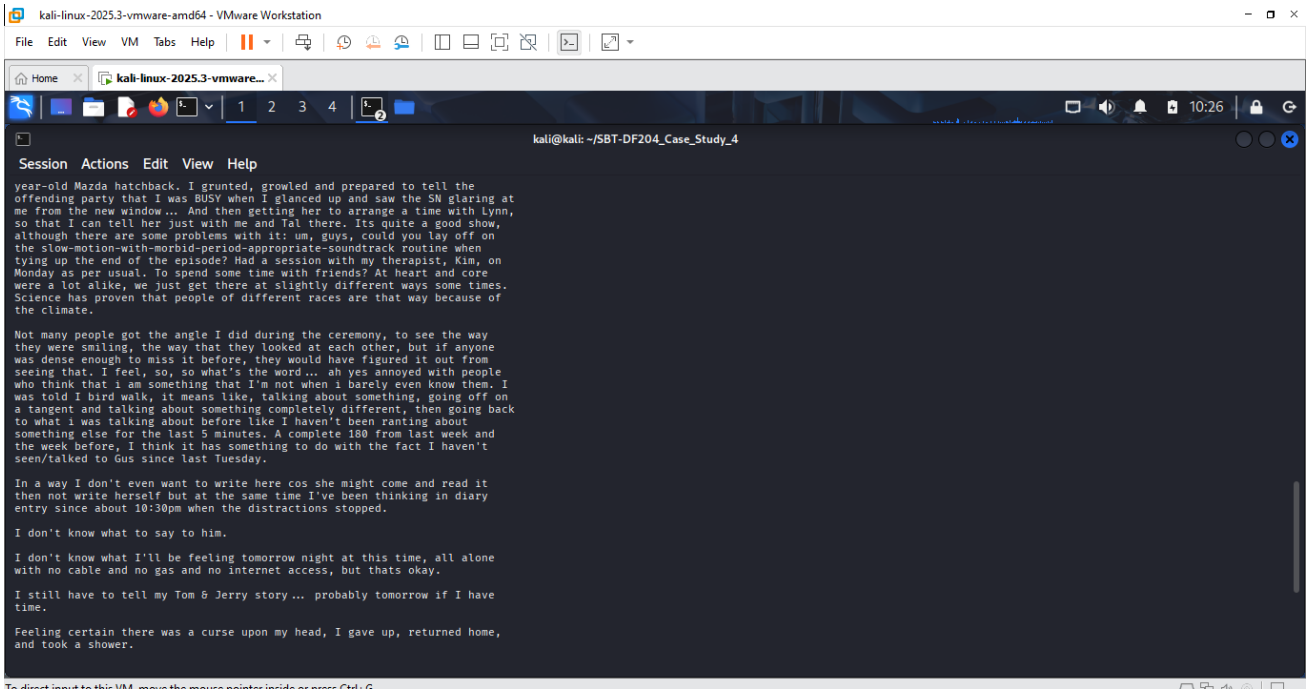


Fig 3.28 Recovered Word Document Text Page 4.

Explanation

Further text from the recovered document is displayed. The continued extraction helps preserve the context surrounding later relevant statements.

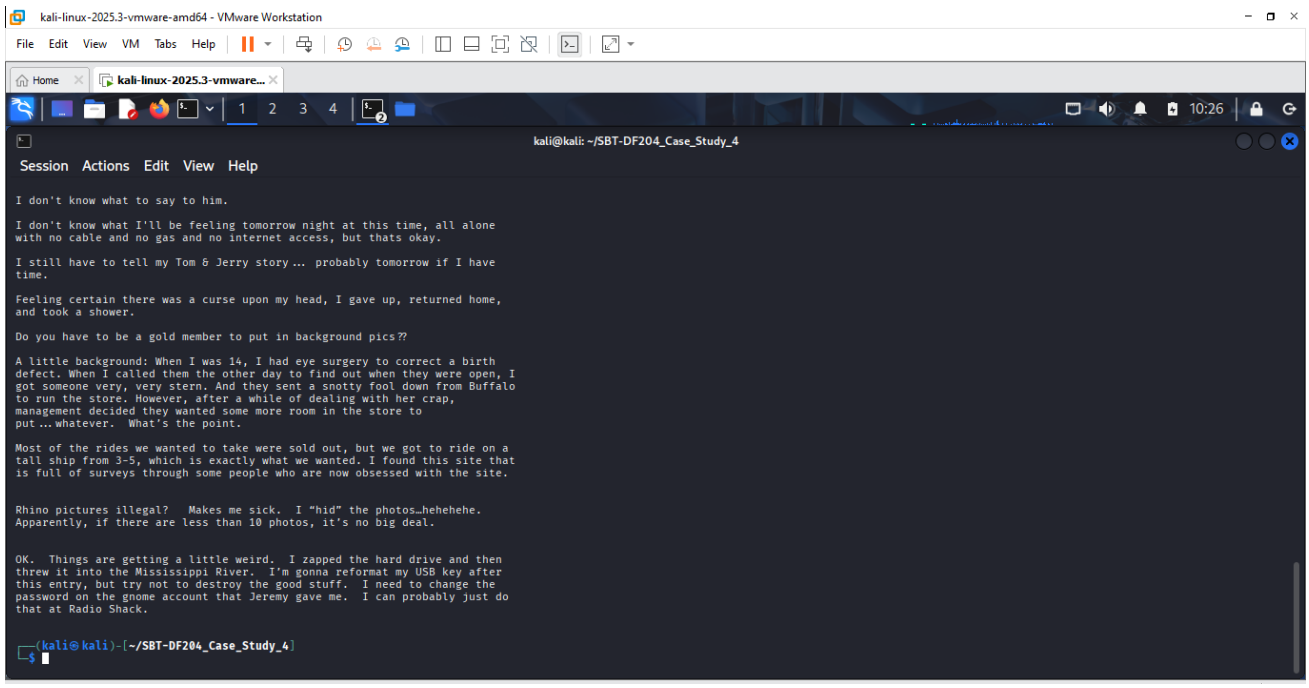


Fig 3.29 Recovered Word Document Text Relevant Final Section.

Explanation

The final section contains references to Rhino photographs, hiding photographs and reformatting a USB device. These statements provide context but do not by themselves establish authorship or prove that the described actions occurred.

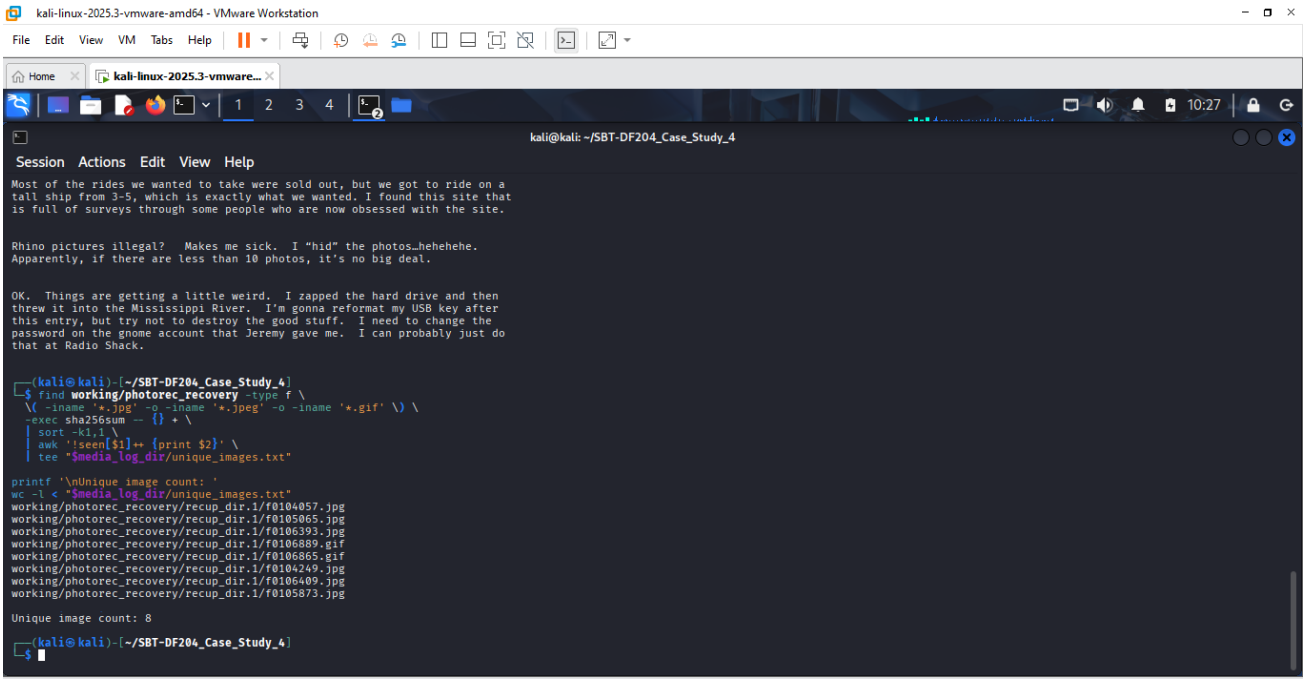


Fig 3.30 Unique Recovered Image List.

Explanation

SHA-256 deduplication reduces the recovered image set to eight unique binaries. This prevents the duplicate JPEG from being counted or tested twice.

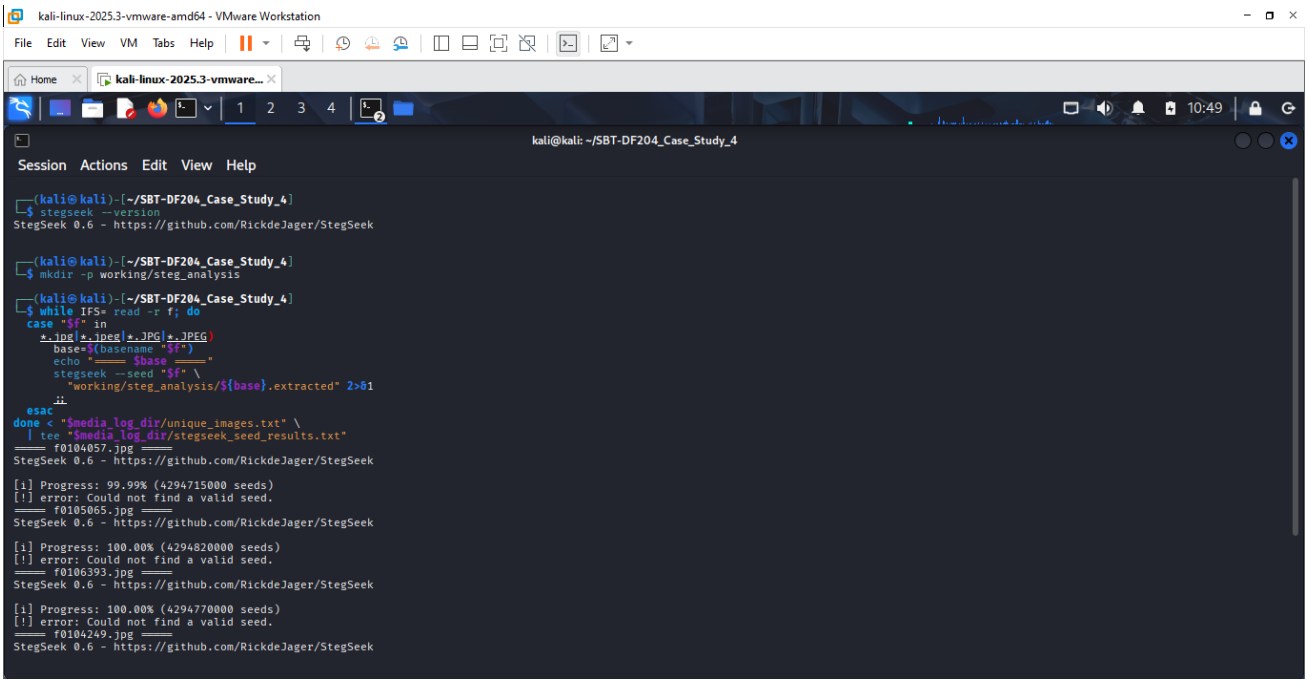


Fig 3.31 Stegseek Hidden-Content Detection Page 1.

Explanation

Stegseek tests the unique JPEG files for a valid Steghide seed. The displayed files return no valid seed.

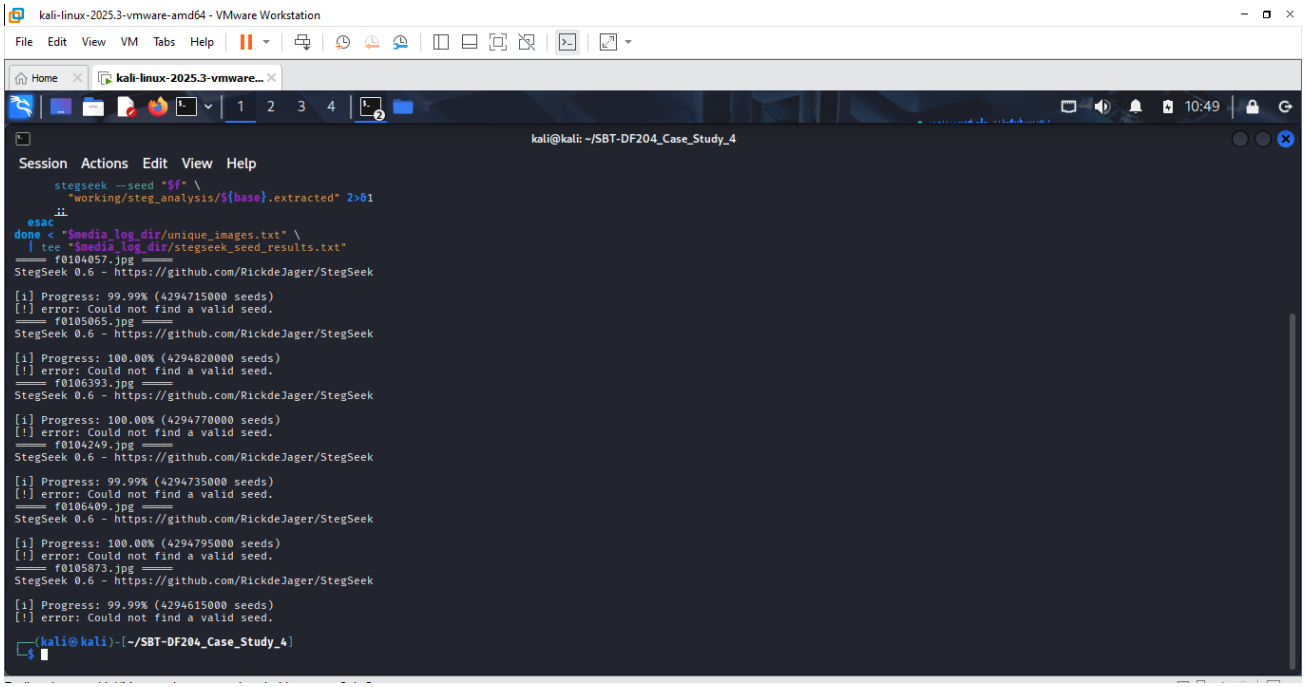


Fig 3.32 Stegseek Hidden-Content Detection Page 2.

Explanation

The remaining unique JPEGs also return no valid seed. No embedded Steghide output was recovered during this test.

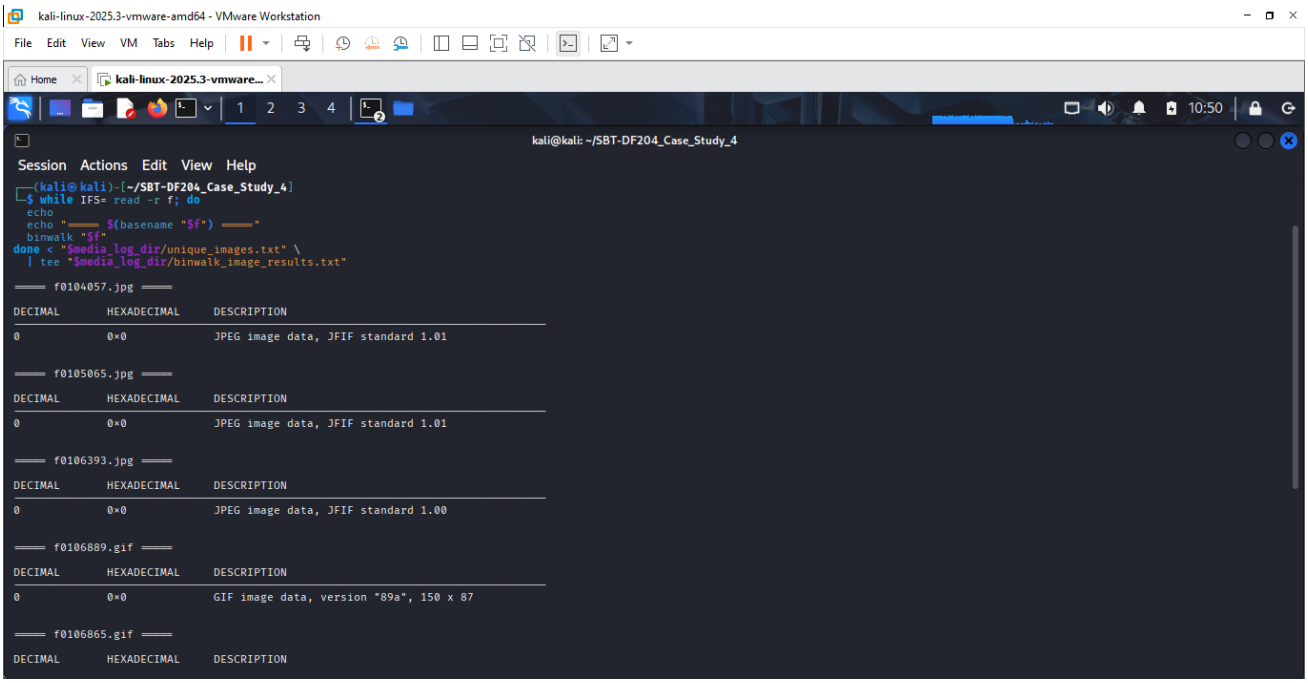


Fig 3.33 Binwalk Image Signature Examination Page 1.

Explanation

Binwalk identifies the normal JPEG and GIF signatures beginning at offset zero. No additional embedded-file signatures are displayed in this part of the output.

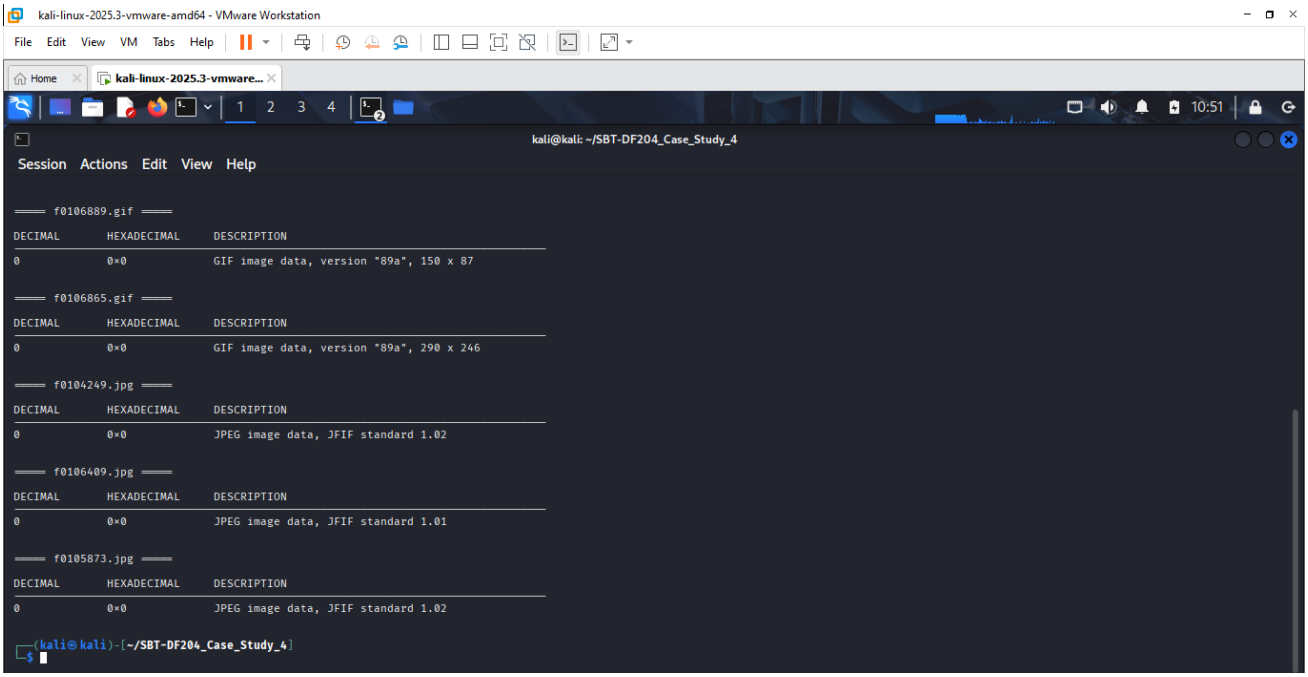


Fig 3.34 Binwalk Image Signature Examination Page 2.

Explanation

The remaining image signatures are shown and again correspond to the expected image formats. The results do not independently confirm appended hidden objects.

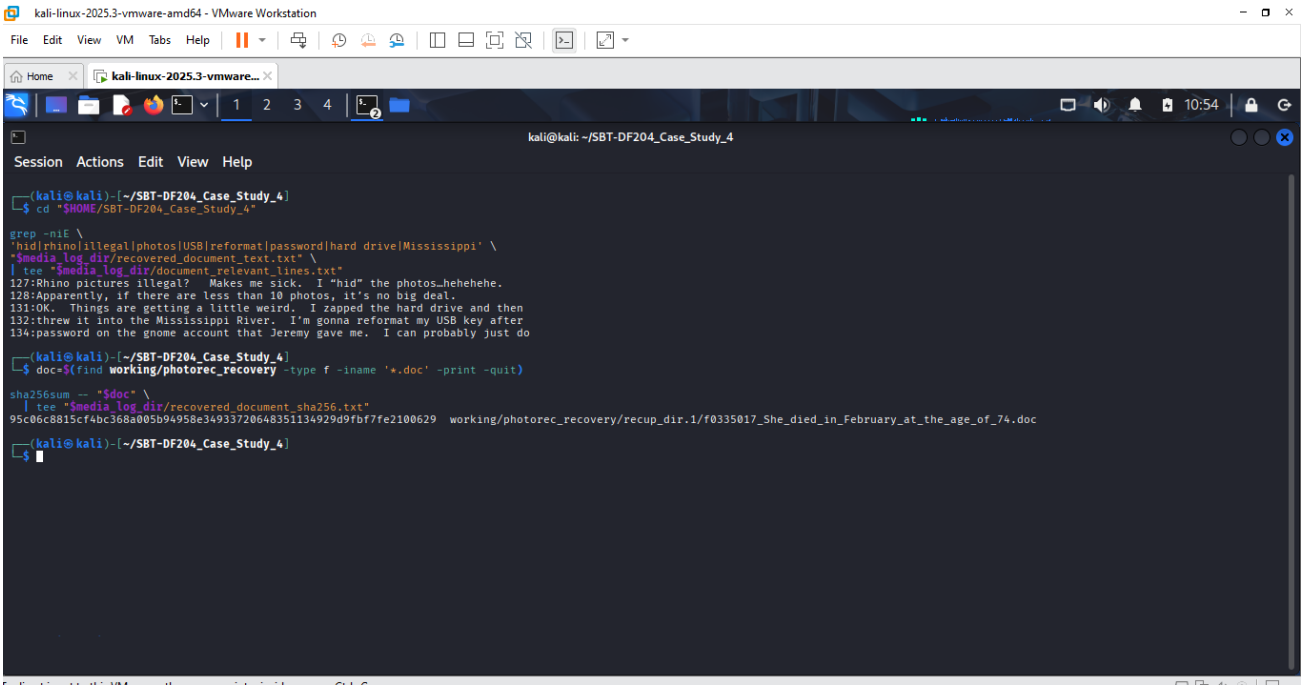


Fig 3.35 Recovered Document Relevant Statements and SHA-256 Hash.

Explanation

The screenshot filters the recovered document for investigation-related terms and records the document's hash. This provides a concise evidential reference for the statements concerning Rhino photographs and USB reformatting.

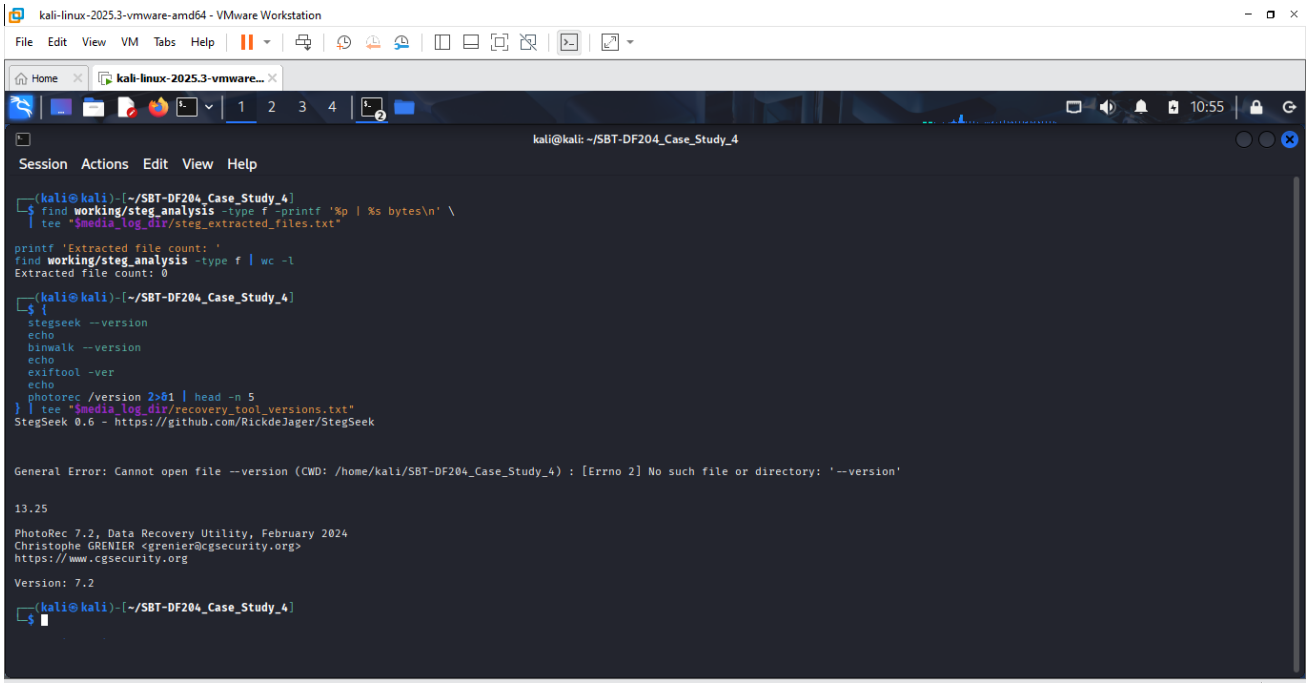


Fig 3.36 Steganography Detection Result and Tool Versions.

Explanation

The output records the hidden-content test result together with tool-version information. No extracted Steghide file was produced by the recorded test.

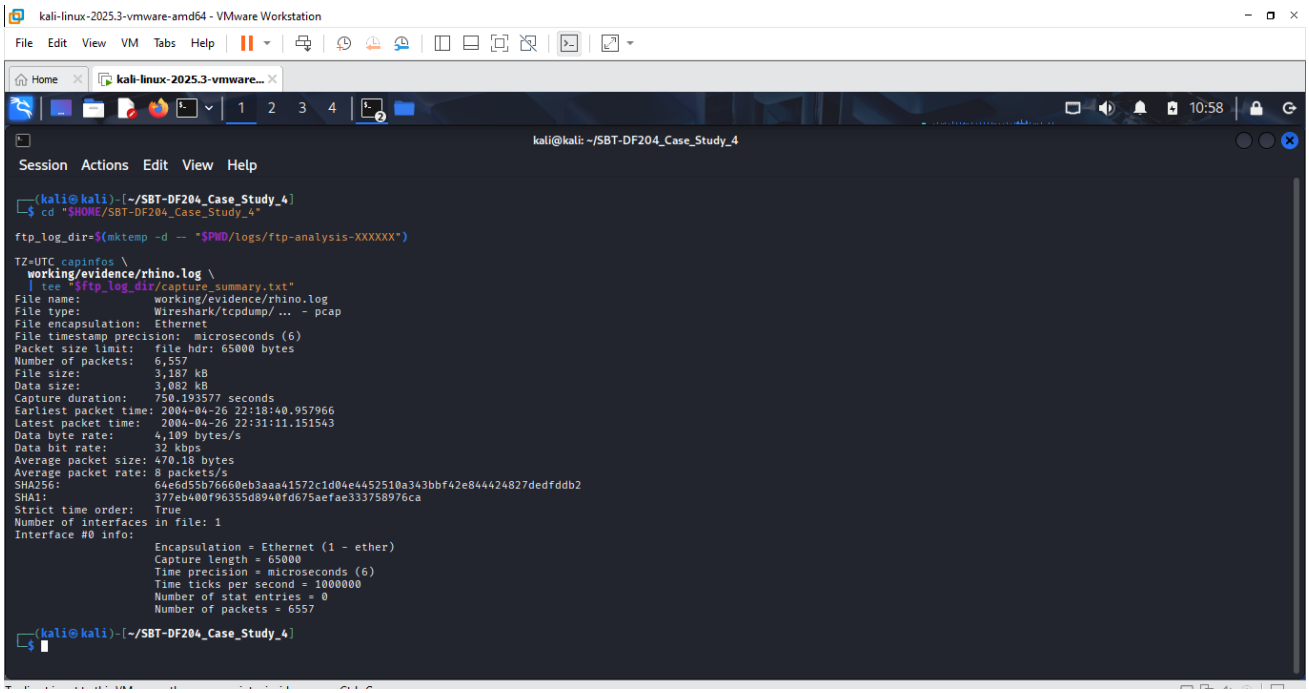


Fig 3.37 FTP Capture Summary.

Explanation

capinfos records the basic properties of rhino.log, including packet count, capture period and file information. This establishes the network evidence being examined.

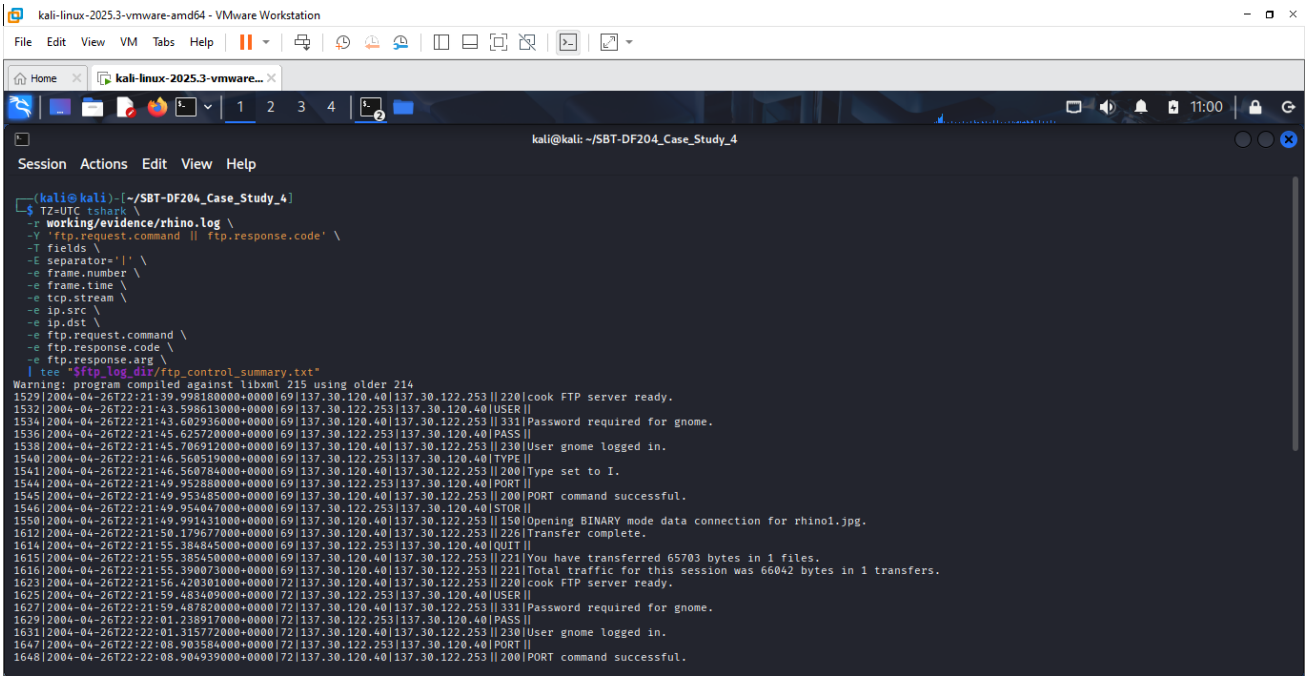


Fig 3.38 FTP Control Session and Server Responses Page 1.

Explanation

The Tshark output shows FTP commands and server replies between the client and server. Successful authentication and transfer-related commands are visible without unnecessarily reproducing the password.

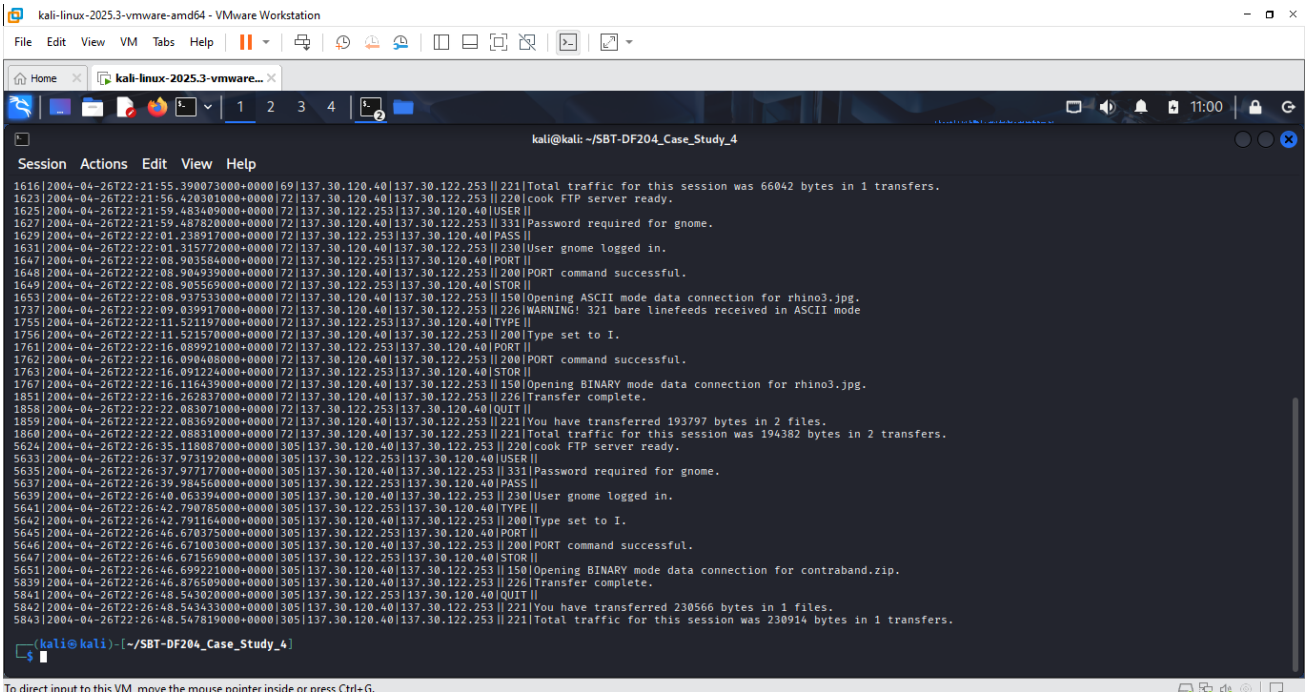


Fig 3.39 FTP Control Session and Server Responses Page 2.

Explanation

This continuation shows later FTP sessions, including transfer-completion responses. STOR activity confirms that the files were uploaded toward the server.

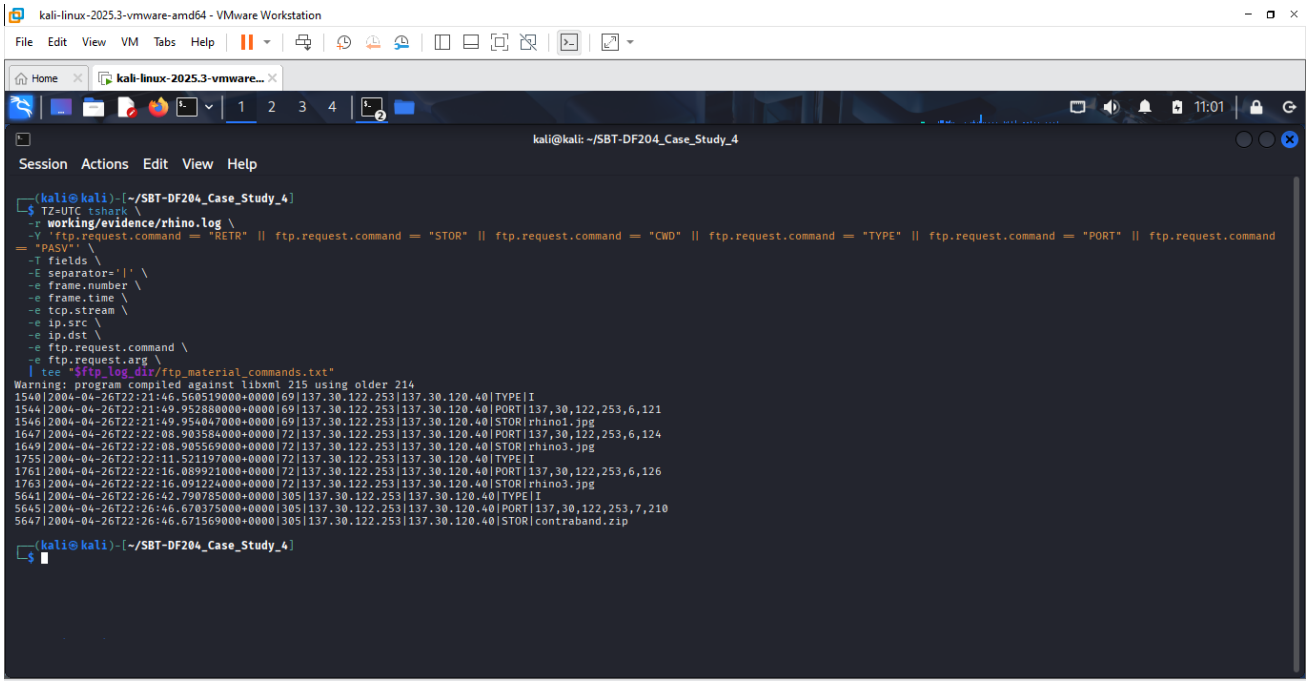


Fig 3.40 FTP Transfer Commands and Filenames.

Explanation

The filtered output identifies STOR rhino1.jpg, STOR rhino3.jpg and STOR contraband.zip. It also shows the transfer modes and active FTP configuration associated with the uploads.

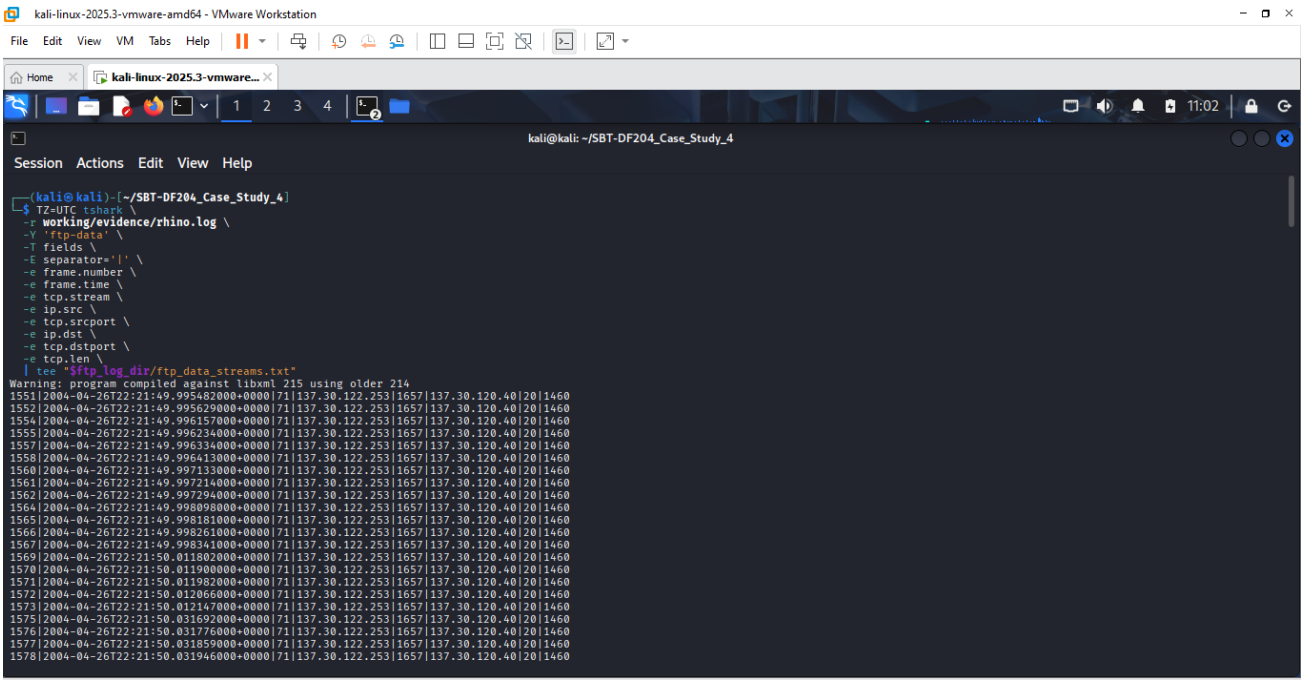


Fig 3.41 FTP Data Stream Identification Page 1.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct the transferred objects.

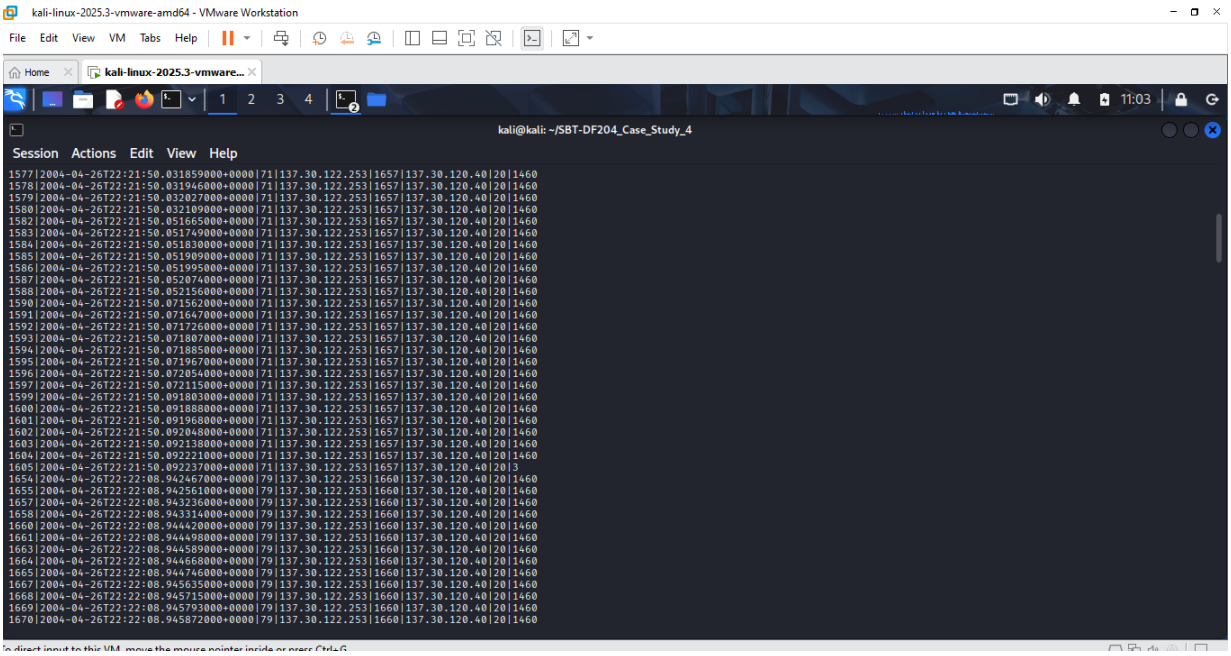


Fig 3.42 FTP Data Stream Identification Page 2.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct the transferred objects.

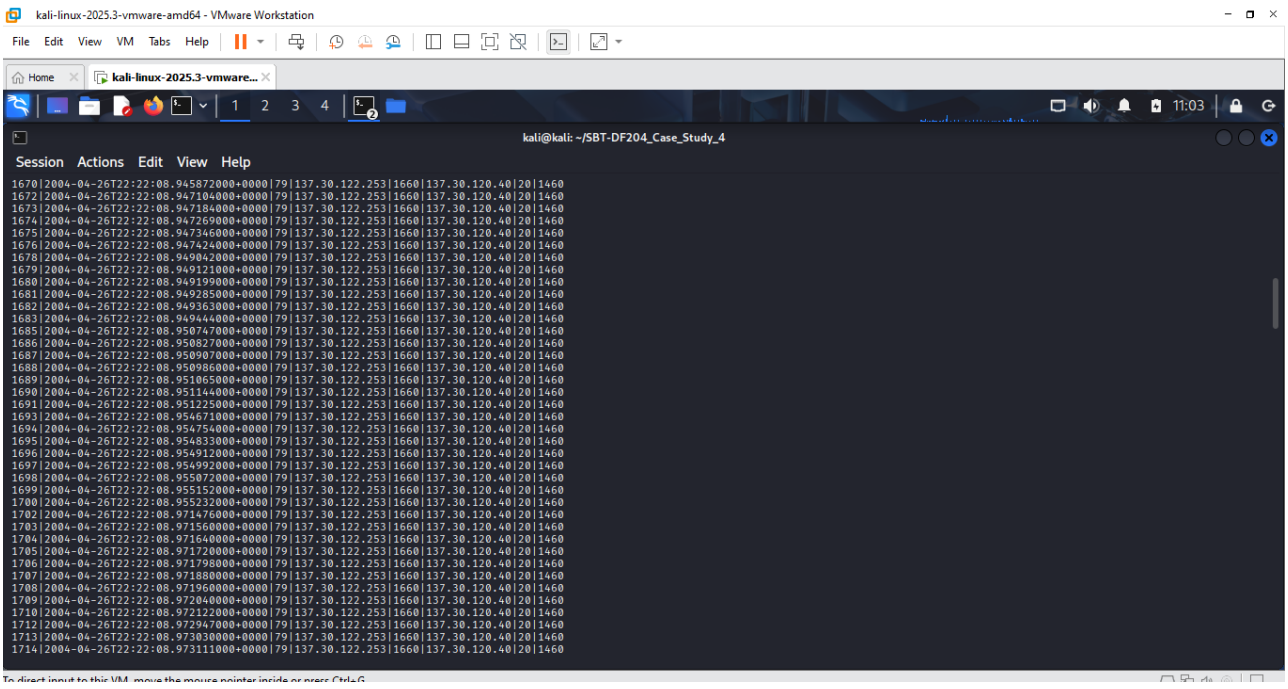


Fig 3.43 FTP Data Stream Identification Page 3.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct

the transferred objects.

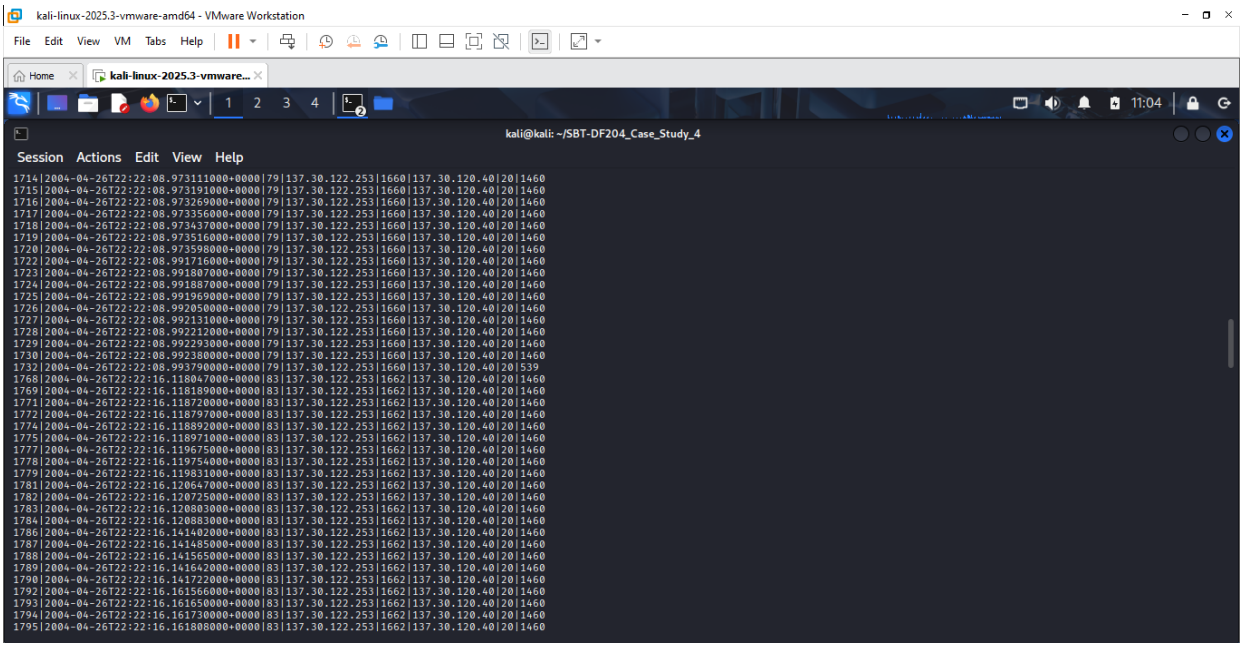


Fig 3.44 FTP Data Stream Identification Page 4.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct the transferred objects**.**

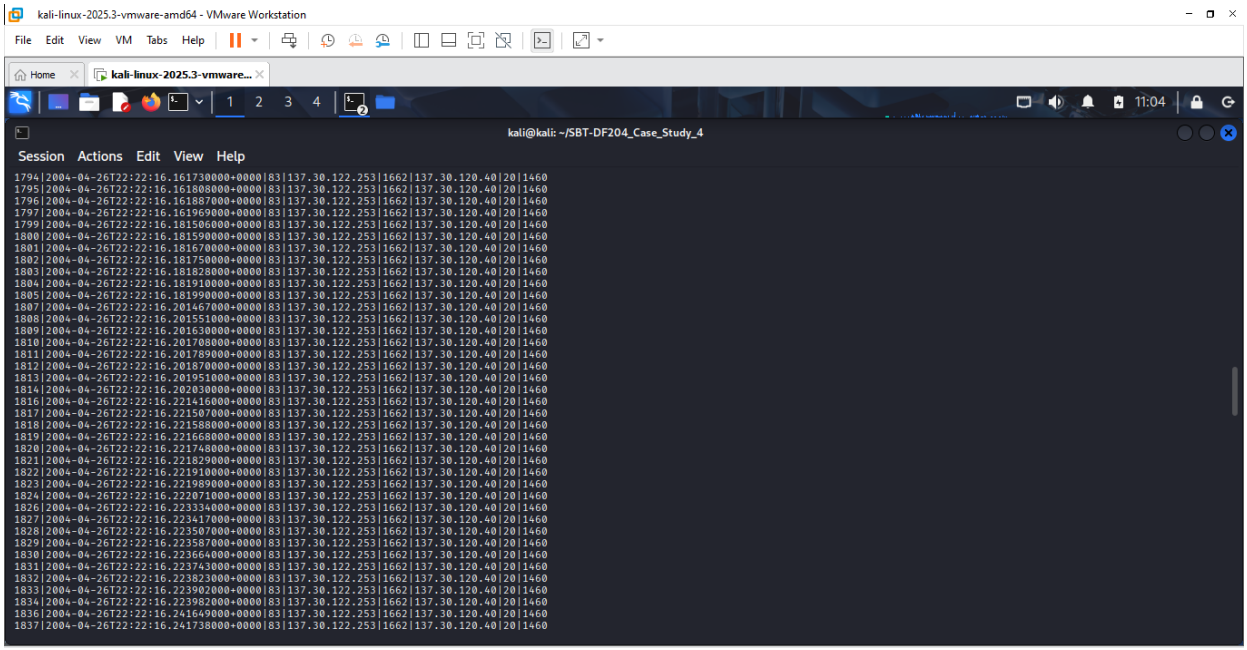


Fig 3.45 FTP Data Stream Identification Page 5.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct

the transferred objects.

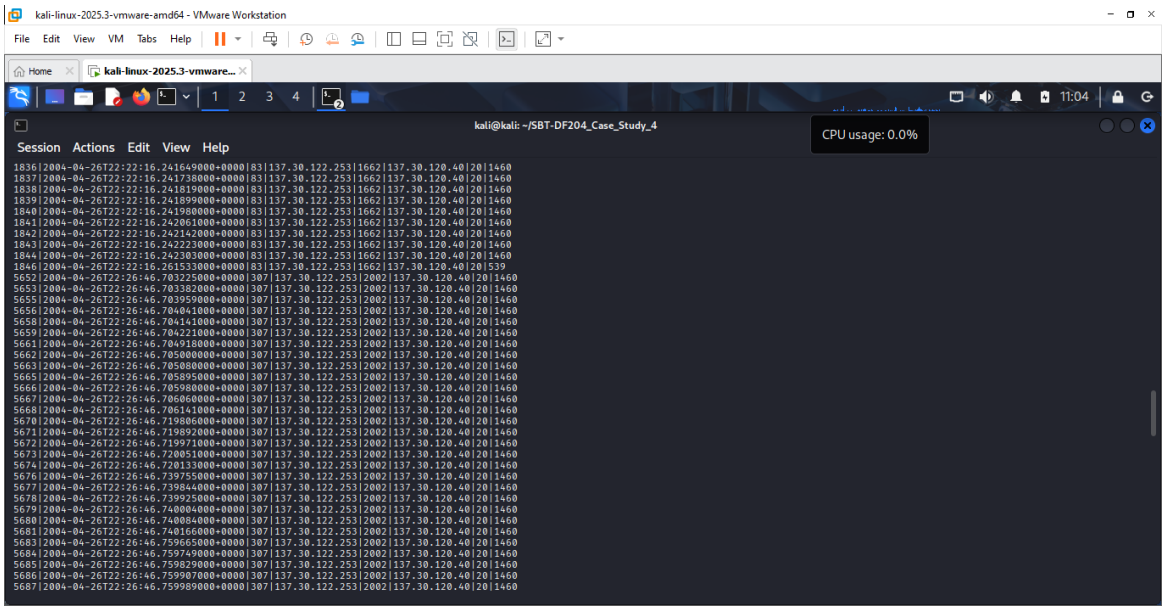


Fig 3.46 FTP Data Stream Identification Page 6.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct the transferred objects.

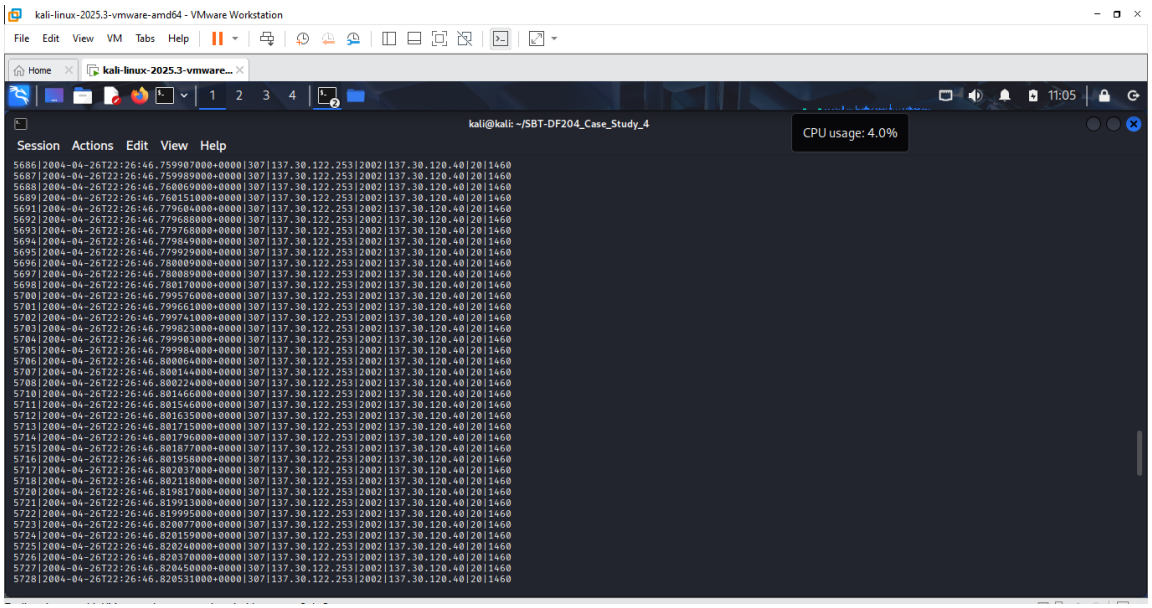


Fig 3.47 FTP Data Stream Identification Page 7.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct the transferred objects.

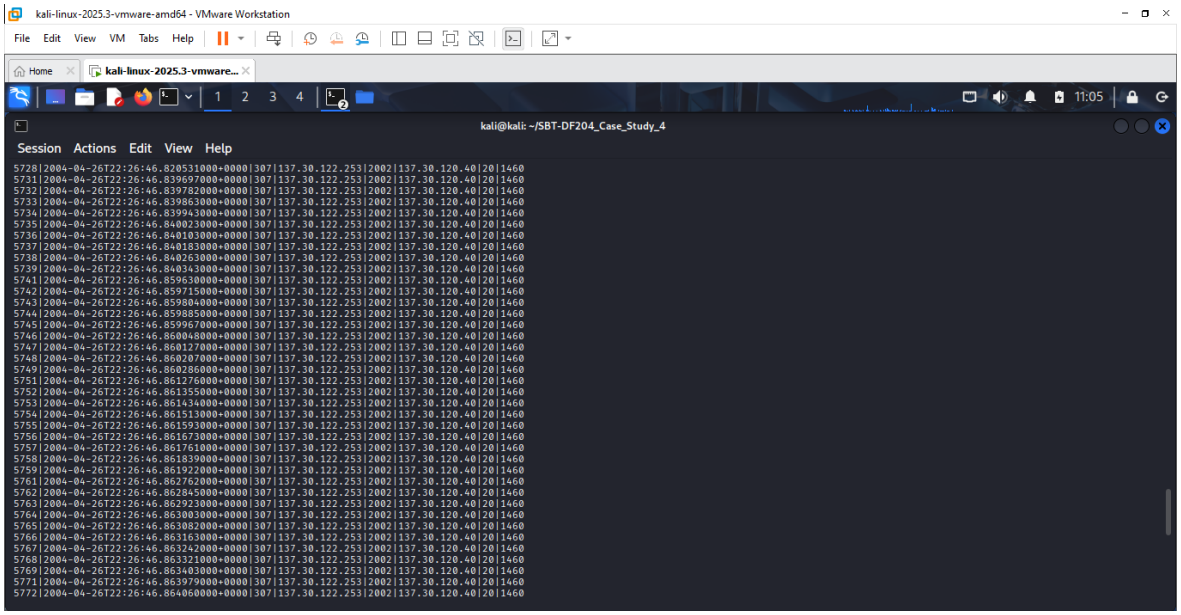


Fig 3.48 FTP Data Stream Identification Page 8.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct the transferred objects.

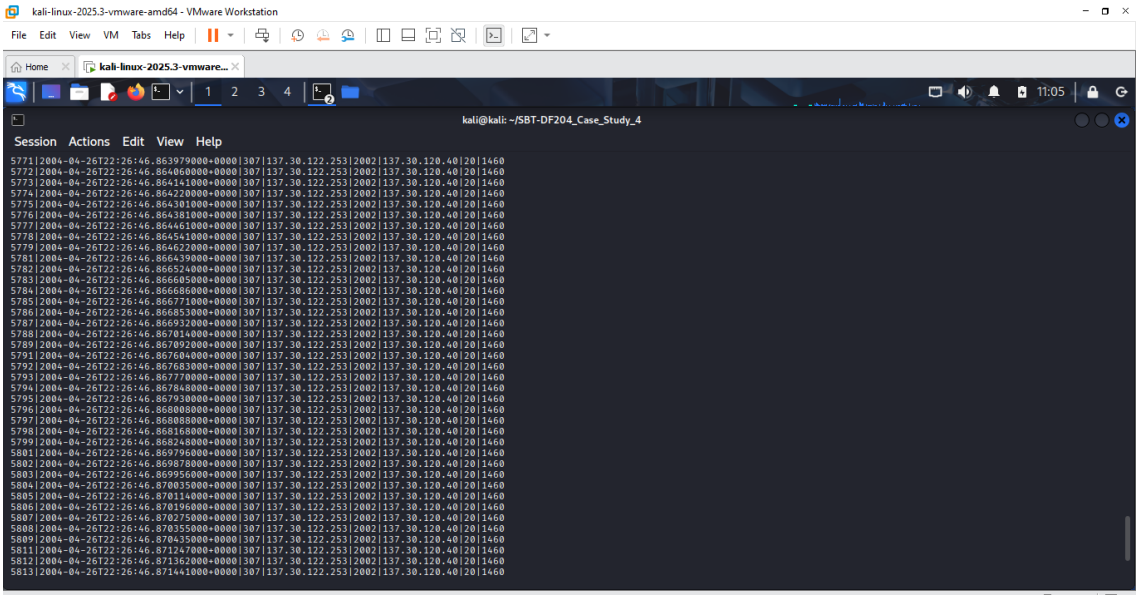


Fig 3.49 FTP Data Stream Identification Page 9.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct the transferred objects.

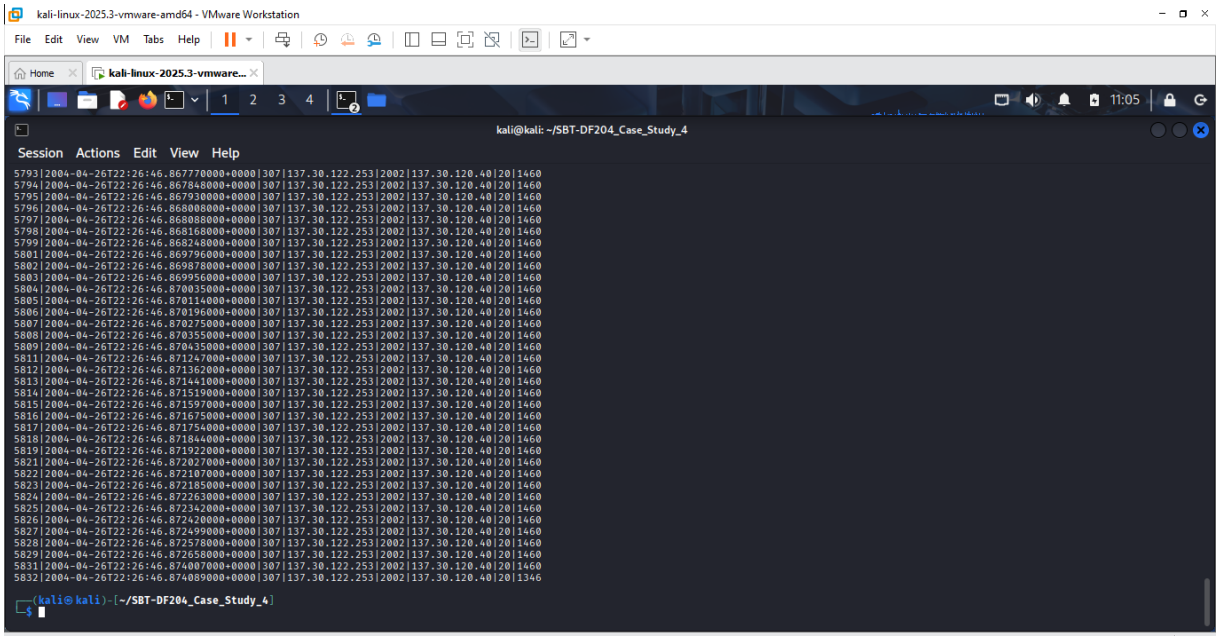


Fig 3.50 FTP Data Stream Identification Page 10.

Explanation

The output continues the packet-level record of the identified FTP data streams. Together with the control-session evidence, these packets document the boundaries, direction and payload data used to reconstruct the transferred objects.

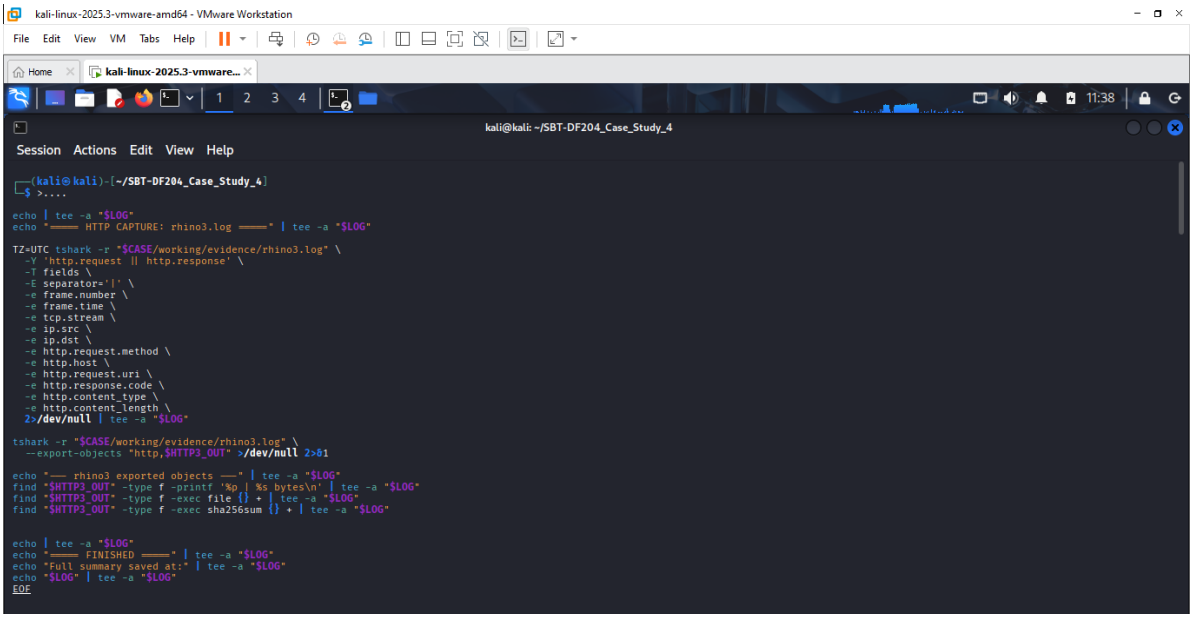


Fig 3.51 FTP and HTTP Reconstruction Script Setup.

Explanation

The screenshot records the commands used to reconstruct FTP objects and export HTTP objects from the supplied packet captures. The workflow operates only on the local working evidence.

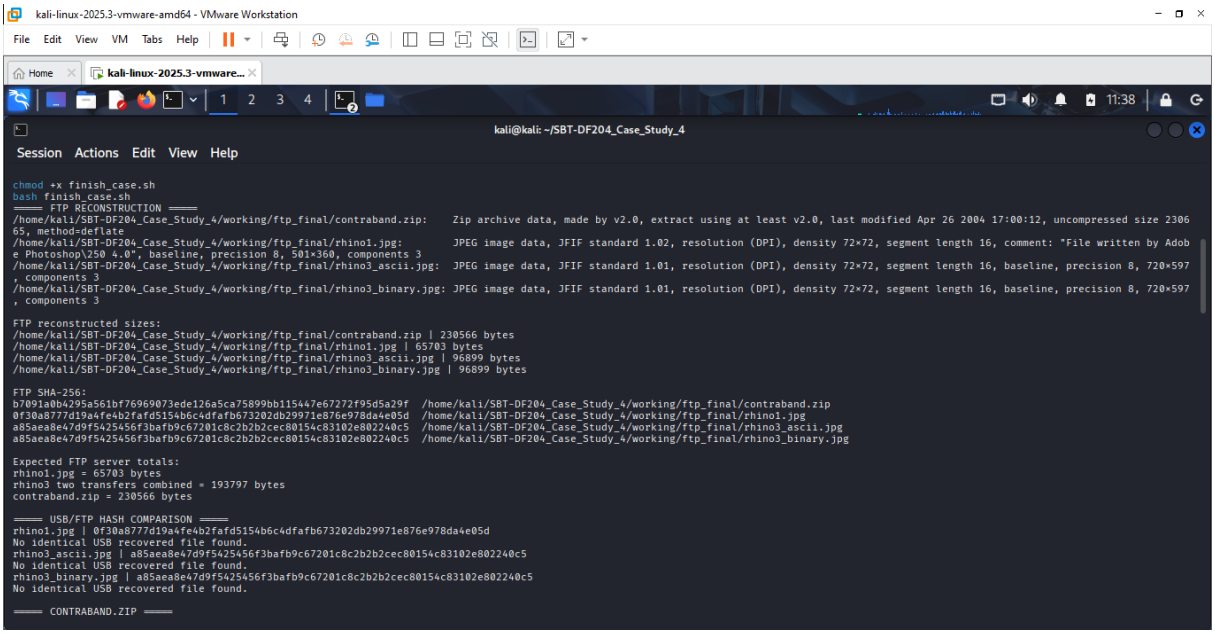


Fig 3.52 FTP Reconstructed Object Size and Hash Validation.

Explanation

The reconstructed rhino1.jpg, both rhino3.jpg transfers and contraband.zip are identified, sized and hashed. The 230,566-byte contraband.zip matches the amount reported by the FTP server.

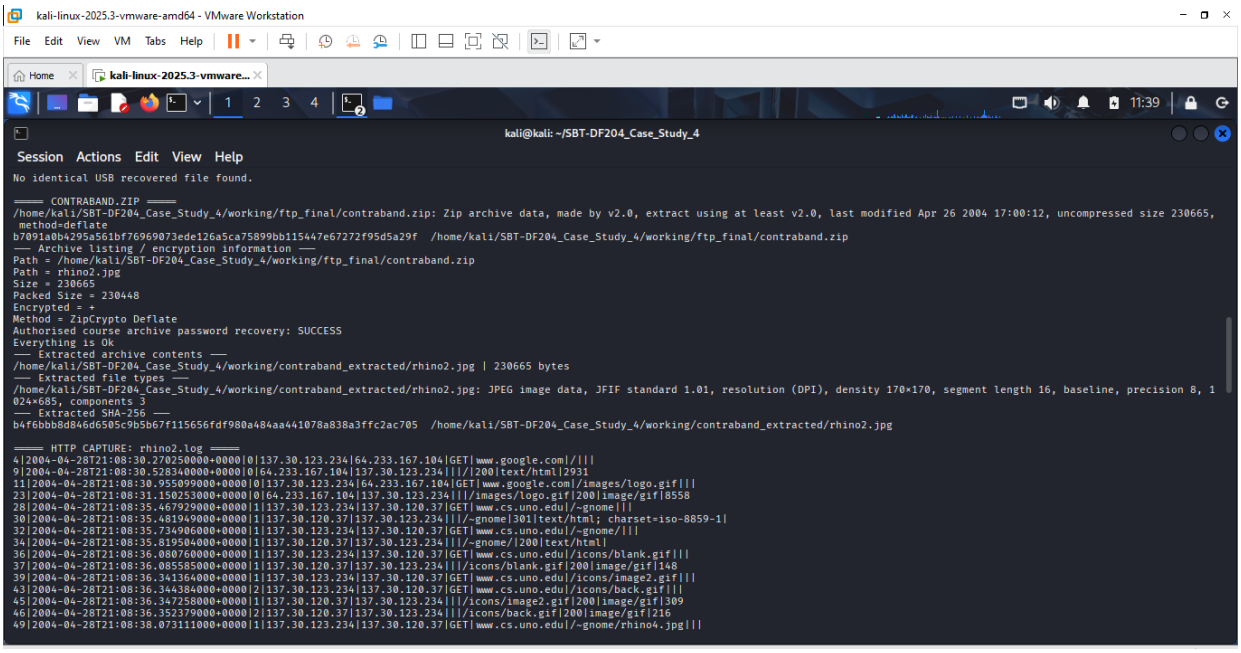


Fig 3.53 Protected Archive Validation and USB Hash Correlation.

Explanation

The reconstructed archive is identified as encrypted, successfully validated and extracted through the authorised procedure. rhino2.jpg from the archive has the same SHA-256 value as the USB-recovered copy, establishing binary equality.

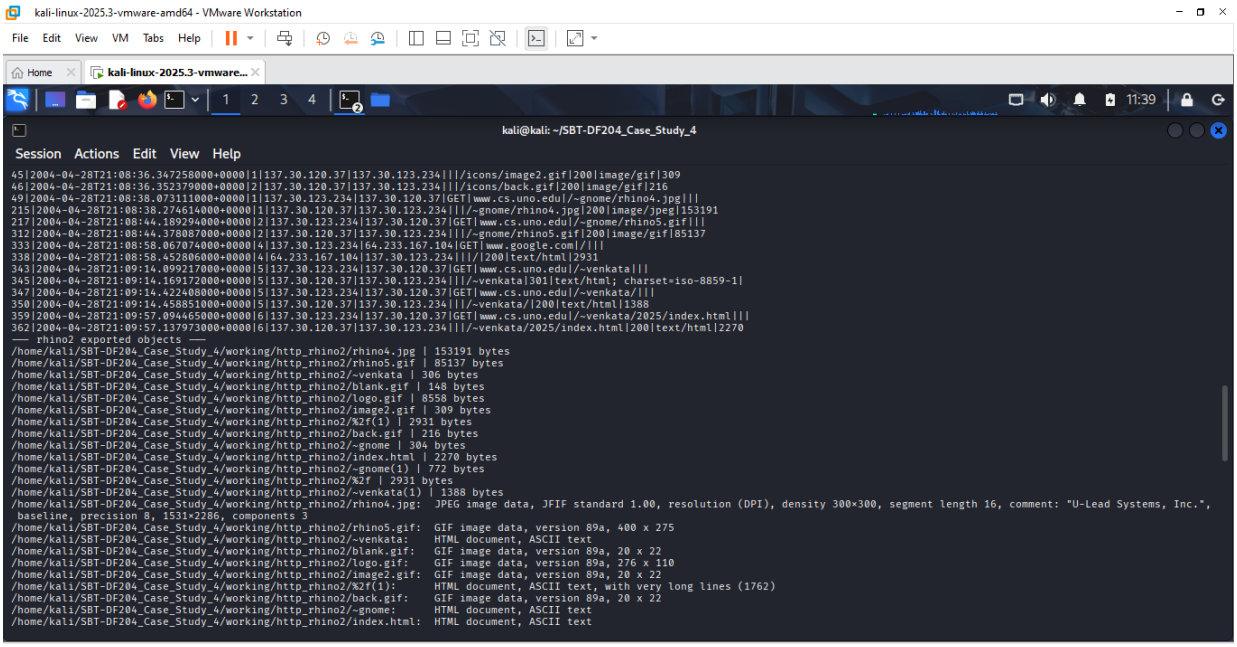


Fig 3.54 HTTP rhino2.log Request and Response Summary.

Explanation

The output records HTTP requests and responses from rhino2.log, including the request for Rhino-related image material. Relevant response codes, content types and object lengths are visible.

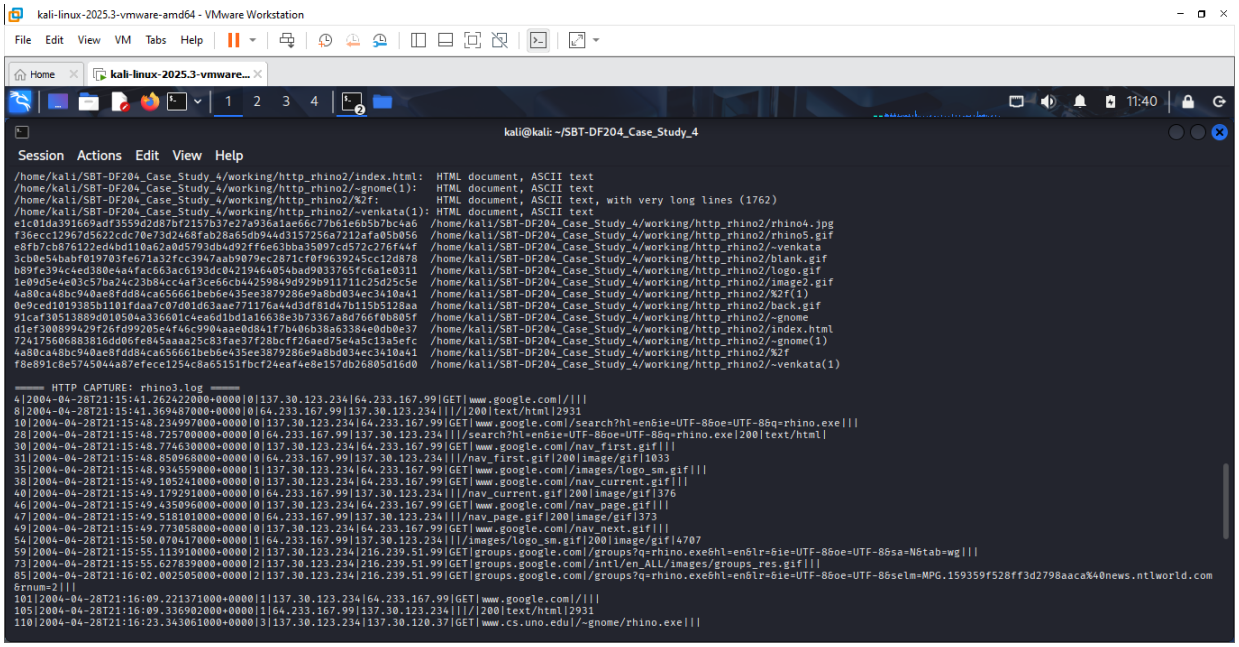


Fig 3.54 HTTP rhino2.log Request and Response Summary.

Explanation

The output records HTTP requests and responses from rhino2.log, including the request for Rhino-related image material. Relevant response codes, content types and object lengths are visible.

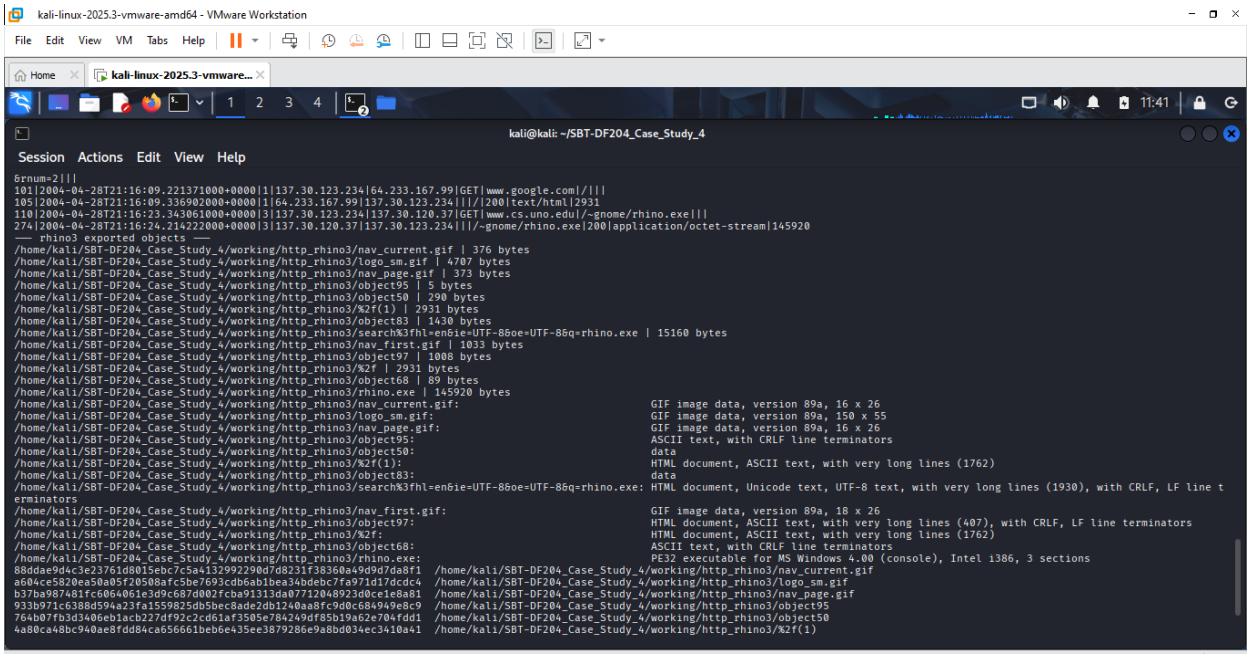


Fig 3.56 HTTP rhino3.log Request and Response Summary.

Explanation

The second HTTP capture records searches and the later request for /~gnome/rhino.exe. The successful server response provides the material used for object reconstruction.

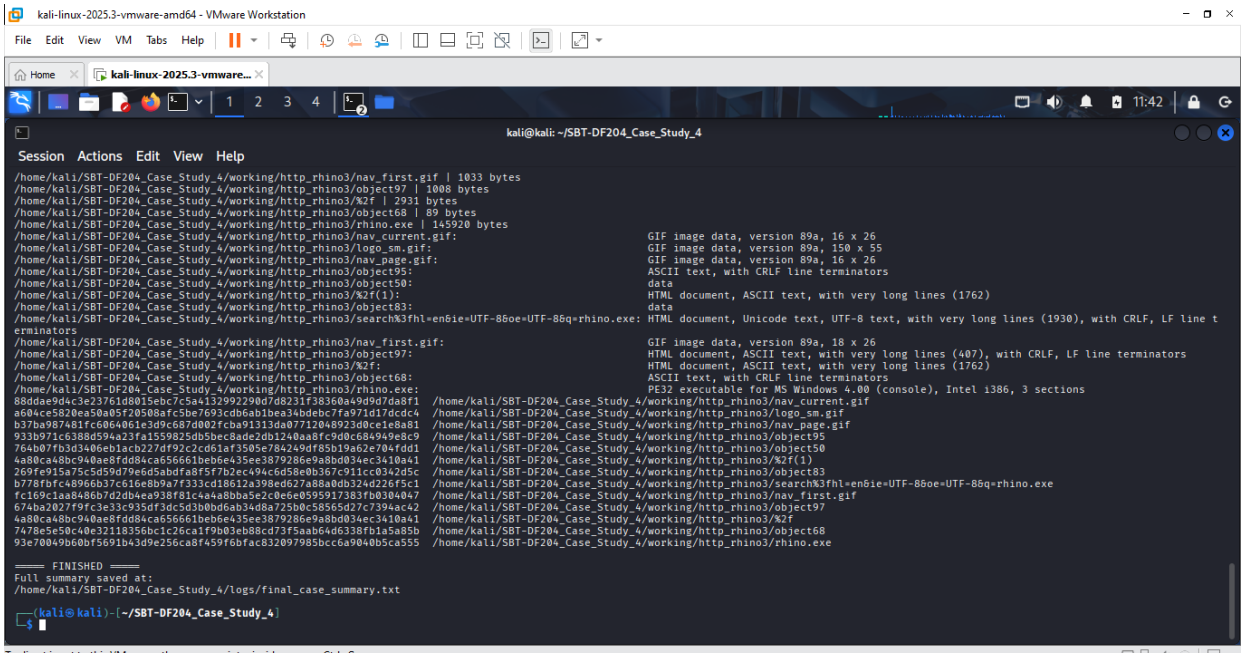


Fig 3.57 Reconstructed rhino.exe File Type and Hash Validation.

Explanation

The exported rhino.exe is identified as a 145,920-byte PE32 Windows executable and is hashed. The file was preserved as evidence and was not executed.

Limitations

Several limitations must be considered when interpreting the findings.

Technical identifiers do not identify a person. IP addresses, usernames, filenames and hashes establish relationships between technical artefacts, but they do not prove which individual was physically using the computer or USB device.

File carving removes some original context. PhotoRec recovered material based on file signatures rather than original filesystem records. As a result, original filenames, directory locations and some timestamps were unavailable for many recovered artefacts.

The recovered document is contextual evidence. Statements concerning hidden photographs and reformatting a USB device were present in the recovered text, but the document alone cannot prove that those actions actually happened or establish its human author.

Negative steganography results are not absolute proof. Stegseek and Binwalk did not confirm hidden content using the methods tested. This reduces support for Steghide or simple embedded-file concealment, but other techniques cannot be completely excluded.

One rhino3.jpg FTP transfer occurred in ASCII mode. The server reported bare-linefeed warnings during the first transfer, introducing some uncertainty around exact byte handling before a second binary-mode transfer was performed.

Successful network transfer does not prove later possession or use. FTP 226 Transfer complete responses support successful completion of the captured transfer, but they do not show what later happened to the server-side files.

Some timestamps have uncertain timezone context. Application metadata from the recovered Word document could not be confidently normalised to UTC and therefore should not be treated as equivalent to packet-capture UTC timestamps.

The reconstructed executable was not executed. rhino.exe was identified as a PE32 Windows executable, but its runtime behaviour was not examined because executing recovered binaries was outside the permitted scope.

Conclusion

The investigation established a clear technical relationship between recovered USB artefacts and the supplied network captures. The USB image contained a FAT16 filesystem from which PhotoRec recovered 132 files, including images and a Word document. The document contained references to Rhino photographs, hiding photographs and reformatting a USB device, while duplicate analysis showed that some recovered image filenames represented the same binary content.

FTP analysis showed that rhino1.jpg, rhino3.jpg and contraband.zip were uploaded from 137.30.122.253 to 137.30.120.40. The protected archive was successfully reconstructed and validated, and its contained rhino2.jpg had the same SHA-256 value as USB-recovered f0106409.jpg. This provides strong cross-source evidence that the same binary image existed in both the USB recovery and the FTP archive.

The HTTP captures further documented access to rhino4.jpg and rhino.exe. The executable was reconstructed and identified but was not run.

Taken together, the evidence supports a high-confidence technical correlation between the recovered media and network activity. However, the available artefacts do not by themselves prove the identity of the person who created, transferred, concealed or physically possessed the files.

Screenshots

Fig 1.0 Forensic Investigation Directory Structure.
Fig 1.1 Original Archive Hash and Working Copy Verification.
Fig 1.2 Evidence Archive Contents.
Fig 1.3 Extracted Evidence Hashes and Working Copy Verification.
Fig 1.4 Extracted Evidence Hashes and Working Copy Verification page 1.
Fig 1.5 Evidence File Type Identification.
Fig 1.6 Evidence File Type Identification page 1.
Fig 1.7 USB Disk Image Details and Partition Layout.
Fig 1.8 USB FAT16 File System Details.
Fig 1.9 USB FAT16 File System Details page 1.
Fig 2.0 USB Root Directory Listing.
Fig 2.1 Deleted File and Directory Entries.
Fig 2.2 USB Gumbo File Metadata.
Fig 2.3 Recovered Gumbo Files and SHA256 Hashes.
Fig 2.4 Recovered Gumbo Text Examination Page 1.
Fig 2.5 Recovered Gumbo Text Examination page 2.
Fig 2.6 Recovered Gumbo Text Examination page 3.
Fig 2.7 PhotoRec Source Disk Selection.
Fig 2.8 PhotoRec Source Disk Selection.
Fig 2.9 PhotoRec File Type Options.
Fig 3.0 PhotoRec Recovery Search Configuration.
Fig 3.1 PhotoRec Recovery Destination Selection.
Fig 3.2 PhotoRec USB Recovery Completion.
Fig 3.3 Recovered File Inventory Page 1.
Fig 3.4 Recovered File Inventory Page 2.
Fig 3.5 Recovered File Inventory Page 3.
Fig 3.6 Recovered File Inventory Page 4.
Fig 3.7 Recovered File Type Summary.
Fig 3.8 Recovered File SHA-256 Hashes Page 1.
Fig 3.9 Recovered File SHA-256 Hashes Page 2.
Fig 3.10 Recovered File SHA-256 Hashes Page 3.
Fig 3.11 Duplicate SHA-256 Hash Identification.
Fig 3.12 Duplicate JPEG Validation.
Fig 3.13 Recovered Media Candidate Inventory.
Fig 3.14 Recovered Media and Document Hash Inventory.
Fig 3.15 Recovered Image Metadata Analysis Page 1.
Fig 3.16 Recovered Image Metadata Analysis Page 2.
Fig 3.17 Recovered Image Metadata Analysis Page 3.
Fig 3.18 Recovered Image Metadata Analysis Page 4.
Fig 3.19 Recovered Image Metadata Analysis Page 5.
Fig 3.20 Recovered Image Metadata Analysis Page 6.
Fig 3.21 Recovered Image Metadata Analysis Page 7.
Fig 3.22 Recovered Word Document Metadata Page 1.
Fig 3.23 Recovered Word Document Metadata Page 2.
Fig 3.24 Duplicate JPEG Binary and Size Validation.
Fig 3.25 Recovered Word Document Text Page 1.

Fig 3.26 Recovered Word Document Text Page 2.
Fig 3.27 Recovered Word Document Text Page 3
Fig 3.28 Recovered Word Document Text Page 4.
Fig 3.29 Recovered Word Document Text Relevant Final Section.
Fig 3.30 Unique Recovered Image List.
Fig 3.31 Stegseek Hidden-Content Detection Page 1.
Fig 3.32 Stegseek Hidden-Content Detection Page 2.
Fig 3.33 Binwalk Image Signature Examination Page 1.
Fig 3.34 Binwalk Image Signature Examination Page 2.
Fig 3.35 Recovered Document Relevant Statements and SHA-256 Hash.
Fig 3.36 Steganography Detection Result and Tool Versions.
Fig 3.37 FTP Capture Summary.
Fig 3.38 FTP Control Session and Server Responses Page 1.
Fig 3.39 FTP Control Session and Server Responses Page 2.
Fig 3.40 FTP Transfer Commands and Filenames.
Fig 3.41 FTP Data Stream Identification Page 1.
Fig 3.42 FTP Data Stream Identification Page 2.
Fig 3.43 FTP Data Stream Identification Page 3.
Fig 3.44 FTP Data Stream Identification Page 4.
Fig 3.45 FTP Data Stream Identification Page 5.
Fig 3.46 FTP Data Stream Identification Page 6.
Fig 3.47 FTP Data Stream Identification Page 7.
Fig 3.48 FTP Data Stream Identification Page 8.
Fig 3.49 FTP Data Stream Identification Page 9.
Fig 3.50 FTP Data Stream Identification Page 10.
Fig 3.51 FTP and HTTP Reconstruction Script Setup.
Fig 3.52 FTP Reconstructed Object Size and Hash Validation.
Fig 3.53 Protected Archive Validation and USB Hash Correlation.
Fig 3.54 HTTP rhino2.log Request and Response Summary.
Fig 3.55 HTTP rhino2.log Exported Objects and Hash Validation.
Fig 3.56 HTTP rhino3.log Request and Response Summary.
Fig 3.57 Reconstructed rhino.exe File Type and Hash Validation.

References

ICDFA Materials.